

Hochschule Aalen

Bachelorarbeit
Studiengang: Informatik

FleetControl

Entwurf und Implementierung eines Softwaresystems zur Überwachung von Fahrzeugflotten

von

Fabian Strauß

3003662

Betreuender Mitarbeiter: Dr. Marc Hermann

Einreichungsdatum: 15. August 2026

Eidesstattliche Erklärung

Hiermit erkläre ich, **Fabian Strauß**, dass ich die vorliegenden Angaben in dieser Arbeit wahrheitsgetreu und selbständig verfasst habe.

Weiterhin versichere ich, keine anderen als die angegebenen Quellen und Hilfsmittel benutzt zu haben, dass alle Ausführungen, die anderen Schriften wörtlich oder sinngemäß entnommen wurden, kenntlich gemacht sind und dass die Arbeit in gleicher oder ähnlicher Fassung noch nicht Bestandteil einer Studien- oder Prüfungsleistung war.

Ort, Datum Unterschrift (Student)

Kurzfassung

Die effiziente Verwaltung und Überwachung von Firmenfahrzeugflotten stellt insbesondere bei größeren Firmenfuhrparks eine Herausforderung dar. Eine zentrale Erfassung von Fahrzeug-, Fahrer- und Fahrtdaten ermöglicht eine verbesserte Nachvollziehbarkeit von Fahrten und schafft eine Grundlage für die Auswertung und Optimierung der Fahrzeugnutzung.

Im Rahmen dieser Projektarbeit wurde ein System zur Verwaltung und Überwachung von Fahrzeugflotten entworfen und als Minimum Viable Product (MVP) umgesetzt. Die entwickelte Lösung basiert auf einer Microservice-Architektur und beinhaltet die Umsetzung einer Telemetrieeinheit zur Erfassung von Fahrtdaten, mehrere Backend-Komponenten zur Verarbeitung und Speicherung sowie eine browserbasierte Benutzeroberfläche zur Verwaltung und Visualisierung.

Das im Rahmen dieser Arbeit umgesetzte System ermöglicht die Anlage, Verwaltung und Verknüpfung von Fahrzeugen, Fahrern und Telemetrieinheiten sowie die Erfassung und Darstellung aufgezeichneter Fahrten und Fahrzeugpositionen. Die entworfene Systemarchitektur wurde implementiert, getestet und als funktionsfähiges System bereitgestellt.

Das entwickelte System kann als Grundlage für zukünftige Erweiterungen dienen, um beispielsweise automatisierte Fahrtenbücher zu verfassen und detaillierte Auswertungen sowie Analysen hinsichtlich Sicherheit, Nutzung und Fahrzeugauslastung vorzunehmen.

Inhaltsverzeichnis

Eidesstattliche Erklärung i

Kurzfassung ii

Abbildungsverzeichnis vi

Abkürzungsverzeichnis ix

1 Einleitung 1
- 1.1 Motivation . . . 1
- 1.2 Problemstellung und -abgrenzung . . . 2
- 1.3 Ziel der Arbeit . . . 3
- 1.4 Vorgehen . . . 3

2 Grundlagen 5
- 2.1 Software-Architektur . . . 5
  - 2.1.1 Microservices . . . 5
  - 2.1.2 Monolithische Architektur . . . 6
  - 2.1.3 Domain-Driven Design . . . 6
  - 2.1.4 Schichtenarchitektur . . . 7
  - 2.1.5 Asynchrone Programmierung . . . 7
- 2.2 Mobile and Embedded Software Development . . . 7
  - 2.2.1 Publisher-Subscriber-Modell, MQTT . . . 7
- 2.3 Cloud and Distributed Systems . . . 8
  - 2.3.1 gRPC . . . 8
  - 2.3.2 REST . . . 8
  - 2.3.3 Containerisierung . . . 9
- 2.4 Embedded Systems . . . 9
  - 2.4.1 Echtzeitbetriebssystem . . . 9
- 2.5 Objektorientierte Modellierung / Rechnerarchitektur . . . 10
  - 2.5.1 Zustandsautomaten . . . 10
- 2.6 Datenbanken / Data Engineering . . . 10
  - 2.6.1 Relationale Datenbanken . . . 10
  - 2.6.2 Objektrelationales Mapping (ORM) . . . 11
  - 2.6.3 Zeitseriendatenbanken . . . 11

3 Problemanalyse **12**
3.1 Problem . . . 12
3.1.1 Stakeholder . . . 12
3.2 Anforderungen . . . 13
3.2.1 Funktionale Anforderungen . . . 13
3.2.2 Nicht funktionale Anforderungen . . . 16
3.2.3 Anwendungsfälle . . . 16
3.2.4 Abnahmekriterien . . . 19

4 Lösungskonzept **20**
4.1 Systemarchitektur . . . 20
4.2 Kommunikation . . . 21
4.3 Komponenten . . . 25
4.3.1 TelemetryUnit . . . 25
4.3.2 IngestionService . . . 36
4.3.3 DriveDataService . . . 45
4.3.4 FleetControlApi . . . 46
4.3.5 FleetControlUI . . . 57

5 Implementierung **67**
5.0.1 TelemetryUnit . . . 67
5.0.2 IngestionService . . . 82
5.0.3 DriveDataService . . . 89
5.0.4 FleetControlApi . . . 91
5.0.5 FleetControlUI . . . 100
5.0.6 RabbitMQ Eventbus, InfluxDB, PostgreSQL, Mosquitto(MQTT) 106
5.1 Tests . . . 107
5.1.1 TelemetryUnit . . . 107
5.1.2 IngestionService . . . 108
5.1.3 FleetControlApi . . . 109
5.1.4 FleetControlUI, DriveDataService, Gesamtanwendung . . . 109
5.1.5 Probleme und Lösungen im Verlauf der Projektumsetzung . . . 109

6 Inbetriebnahme **113**

7 Evaluation **116**
7.1 Erfüllung der Anforderungen . . . 116
7.1.1 Funktionale Anforderungen . . . 116
7.1.2 Nichtfunktionale Anforderungen . . . 117
7.1.3 Bekannte Probleme . . . 118

8 Ausblick **120**
8.1 Erweiterbarkeit der Ergebnisse . . . 120
8.2 Übertragbarkeit der Ergebnisse . . . 121

9 Zusammenfassung und Fazit **122**

Literatur **125**

Abbildungsverzeichnis

1.1 Vorgehen bei der Bearbeitung 4

4.1 Architekturdiagramm der endgültigen Komponenten und ihrer Kommunikationswege 24
4.2 STM32F103RB Nucleo Entwicklungsplatine 25
4.3 IMU-Sensoreinheit MPU6050 26
4.4 Waveshare SIM7600E-H 4G HAT Entwicklungsplatine 27
4.5 GPS-Antenne 27
4.6 LTE-Antenne 27
4.7 Verdrahtete Telemetrieeinheit ohne MPU6050 29
4.8 Zustandsautomat der Telemetry-FSM 31
4.9 Zustandsautomat der Network-FSM 32
4.10 Zustandsautomat der GNSS-FSM 33
4.11 Klassendiagramm der Entwurfsebene von *TelemetryUnit* 35
4.12 Zustandsautomat für laufende Fahrten 39
4.13 Klassendiagramm für MQTT-Eingang und Nachrichtenverarbeitung . 41
4.14 Klassendiagramm des Zustandsautomatenbereichs 42
4.15 Klassendiagramm für Zuordnungssynchronisations-, Persistenz- und Benachrichtigungsbereich 44
4.16 Klassendiagramm der Entwurfsebene von *DriveDataService* 46
4.17 Entity-Relationship-Diagramm von *FleetControlApi* 49
4.18 Klassendiagramm der *API*-Schicht von *FleetControlApi* 52
4.19 Klassendiagramm der *Service*-Schicht von *FleetControlApi* 54
4.20 Klassendiagramm der *Data*-Schicht von *FleetControlApi* 55
4.21 Klassendiagramm der *Domain*-Schicht von *FleetControlApi* 56
4.22 Darstellung des Navigationskonzept 58
4.23 Grobe Darstellung der Fahrzeugeansicht 61
4.24 Grobe Darstellung der Fahrtenansicht 62
4.25 Grobe Darstellung der Personenansicht 63
4.26 Grobe Darstellung der Telemetrieeinheitenansicht, in rot das Auslösen des Broadcast-Befehls zum Verbindungsaufbau mit einer angeschlossenen Hardware-Einheit 64
4.27 Unvollständiges Klassendiagramm der Entwurfsebene von *FleetControlUI* 66

5.1 Hardwarekonfiguration des Projekts in STM32CubeMX 68

6.1 Fahrtenansicht von *FleetControl* 115

Listings

5.1 mutexgesicherte UART-Nutzung in Uart-Klasse (Uart.hpp) 70
5.2 Mutex-Kapselung innerhalb von `Uart::write` (Uart.cpp) 70
5.3 Byteweises Einlesen und Befehlserkennung (check_for_device_id_-request_command) 72
5.4 Öffentliche Schnittstelle der Klasse Sim7600 (sim7600eh.hpp) 74
5.5 AT-Befehl senden und Antwort auswerten (Sim7600::check_network) . 75
5.6 JSON-Serialisierung eines Datenpunkts (TelemetryPoint::to_json) . . 76
5.7 Beschleunigung aus GPS-Geschwindigkeitsdifferenz (TelemetryPoint::calculate_accel_in_ms2) 77
5.8 Zustandsübergang Idle -> Publishing (TelemetryFsm::step) 78
5.9 Vollständiger Sendevorgang (TelemetryFsm::publish_timestamp) . . . 78
5.10 C/C++-Vermittlung am Beispiel von NetworkFSM (Network/interface.cpp) 79
5.11 RTOS-Tasks mit tastergesteuertem Moduswechsel in freertos.c 81
5.12 RPC-Definition und Nachrichtenformat in `assignments.proto` . . . 83
5.13 Retry-Schleife für den MQTT-Verbindungsaufbau in MqttClientService.cs 84
5.14 Kernprüfung in GpsPlausibilityFilter.IsPlausible 86
5.15 Erkennung des Fahrtbeginns in TripReactor.DispatchAsync 87
5.16 Anreichern der Datenpunkte mit Informationen zur Zuordnung in InfluxWriteWorker.cs vor der Persistierung 88
5.17 Service-Definition der RPCs in `telemetry.proto` 90
5.18 Implementierung des RPCs `GetTelemetryPoints` in `TelemetryQueryService.cs` 91
5.19 Auszug aus VehicleController.cs: Controller-Muster und Routendefinition 93
5.20 Service-Definition in `assignments.proto` 94
5.21 Service-Definition in `trip.proto` 94
5.22 Service-Definition in `asssignments_updates.proto` 94
5.23 Service-Definition für Abfrage von *DriveDataService* in `telemetry.proto` 95
5.24 Unvollständige Service-Klasse `VehicleService` 96
5.25 Schemadefinition und Konfiguration in `AppDbContext` 98
5.26 Implementierung `RabbitMqTelemetryConsumer` als Background Service 100
5.27 Reiter-Auswahl im HTML-Template von MainPageComponent 101
5.28 Definition der vier Reiter in der Klasse MainPageComponent 101
5.29 Theme-Variablen in styles.scss (Auszug) 102

5.30 Verwendung der Variablen 102
5.31 Alle Fahrzeuge laden in VehicleService.ts 103
5.32 Verbindungsaufbau in VehicleLiveService 104
5.33 Start-, Ziel- und Pulsier-Marker (CardWidgetComponent) 105
5.34 Achsenkonfiguration für Geschwindigkeit und Beschleunigung in TripChartComponent 106
5.35 Einbindung des Liniendiagramms in trip-chart.component.html 106
5.36 Docker-Compose-Konfiguration des RabbitMQ-Eventbus 107
5.37 Debug-Ausgaben in GnssFSM der Telemetrieeinheit 108
5.38 Verwendung der Logger-Instanz im Fahrtzustandsautomaten (TripReactor.DispatchAsync) 109
5.39 Reverse-Proxy-Konfiguration (Caddyfile) 112

Abkürzungsverzeichnis

MQTT Message Queuing Telemetry Transport
gRPC gRPC Remote Procedure Calls
RPC Remote Procedure Call
GNSS Global Navigation Satellite System
GPS Global Positioning System
REST Representational State Transfer
HAL Hardware Abstraction Layer
RTOS Real Time Operating System
UART Universal Asynchronous Receiver Transmitter
JSON JavaScript Object Notation
UUID Universally Unique Identifier
DSGVO Datenschutz-Grundverordnung
API Application Programming Interface
HTTP Hypertext Transfer Protocol
HTTPS Hypertext Transfer Protocol Secure
TLS Transport Layer Security
SPA Single Page Application
NMEA National Marine Electronics Association
UTC Coordinated Universal Time
FSM Finite State Machine
FIN Fahrzeug-Identifikationsnummer
GPIO General Purpose Input Output
LTE Long Term Evolution
CRUD Create, Read, Update, Delete
ACID Atomicity, Consistency, Isolation, Durability
DDD Domain-Driven Design
VM Virtuelle Maschine
IoT Internet of Things
URL Uniform Resource Locator

1 Einleitung

1.1 Motivation

Die Verwaltung eines größeren Fuhrparks ist mit einer Vielzahl organisatorischer und administrativer Aufgaben verbunden. Neben der Verwaltung von Fahrzeugen und Fahrern müssen insbesondere Informationen über die Nutzung der Fahrzeuge erfasst, zugeordnet und nachvollziehbar dokumentiert werden. Ohne eine zentrale Datengrundlage können solche Informationen auf unterschiedliche Systeme, Dokumente oder manuelle Prozesse verteilt sein. Dies erschwert die Übersichtlichkeit und Nachvollziehbarkeit und erhöht gleichzeitig das Risiko von Inkonsistenzen durch menschliche Fehler.
Die Automatisierung wiederkehrender Tätigkeiten kann administrativen Aufwand und damit verbundene Personalkosten reduzieren.

Von einem Fuhrparkmanagementsystem können darüber hinaus weitere betriebliche Abläufe profitieren. Die kontinuierliche Erfassung von Fahrzeug- und Fahrtdaten ermöglicht eine bessere Nachvollziehbarkeit der Fahrzeugnutzung und kann dazu beitragen, Unregelmäßigkeiten oder eine unsachgemäße Nutzung von Fahrzeugen zu erkennen. Es besteht auch die Möglichkeit der Überwachung des Umgangs von Mitarbeitern mit den ihnen zugewiesenen Fahrzeugen, in welchem Umfang sowas zulässig ist soll an der Stelle außen vor bleiben, gemeint ist eine Überwachung im Rahmen der Legalität mit Kenntnis und Zustimmung der betreffenden Mitarbeiter. Die Erfassung von Positionsdaten kann zudem bei der Nachverfolgung gestohlener oder unbefugt genutzter Fahrzeuge unterstützen.

Auch im Zusammenhang mit Unfällen können aufgezeichnete Fahr- und Positionsdaten einen Mehrwert bieten. Durch die zeitliche und räumliche Nachvollziehbarkeit von Fahrten können Ereignisabläufe nachträglich besser rekonstruiert und objektiv nachvollzogen werden. Dies kann beispielsweise bei der Klärung von Unfallhergängen und der Bewertung von Verantwortlichkeiten unterstützend herangezogen werden.

Die zentral erfassten Daten bieten zudem eine Grundlage für weiterführende Anwendungen. Dazu zählen beispielsweise die automatisierte Erstellung von Fahrtenbüchern, die Analyse der Fahrzeugauslastung und des Fahrverhaltens sowie die Untersuchung von Verschleiß und sicherheitsrelevanten Ereignissen. Langfristig

können die gewonnenen Erkenntnisse auch für die Optimierung der Fuhrparkplanung und zukünftige Beschaffungsentscheidungen genutzt werden.

Aus diesen Anforderungen entsteht die Motivation für die Entwicklung einer Systemlösung, die relevante Fahrzeug-, Fahrer- und Fahrtdaten erfasst, miteinander verknüpft und für unterschiedliche Anwendungsfälle zugänglich macht. Die vorliegende Arbeit untersucht die Konzeption und praktische Umsetzung einer solchen Lösung und setzt diese als funktionsfähiges Minimum Viable Product um.

Neben dem fachlichen Nutzen der entwickelten Lösung war auch die technische Vielseitigkeit des Projekts ein wesentlicher Grund für dessen Wahl. Das Projekt verbindet die Entwicklung einer eingebetteten, fahrzeugseitigen Telemetrieeinheit mit der Entwicklung eines mehrkomponentigen, auf Microservices basierenden Full-Stack-Systems inklusive einer browserbasierten Benutzeroberfläche. Dadurch können unterschiedliche Bereiche der Software- und Systementwicklung sowie deren Zusammenspiel innerhalb einer durchgängigen Lösung praktisch erprobt werden. Die Kombination dieser Bereiche entspricht zugleich den technischen Tätigkeitsfeldern, in denen eine zukünftige berufliche Tätigkeit angestrebt wird, und bietet somit die Möglichkeit, praktische Erfahrungen in diesen Bereichen zu vertiefen.

1.2 Problemstellung und -abgrenzung

Will man Daten für Fahrzeuge erheben, muss in jedem Fahrzeug eine Telemetrieeinheit verbaut werden, die mittels geeigneter Sensoren Fahrtdaten erfassen und diese während der Fahrt übertragen kann. Die Telemetrieeinheit sollte fest und möglichst unsichtbar im Fahrzeug verbaut werden, um bei der Fahrzeugnutzung nicht zu stören und die Einheit selbst vor Beschädigung zu schützen. Eine Bedienung durch den Fahrer, beispielsweise zum Ein- oder Ausschalten der Einheit, sollte nicht erforderlich sein, damit die Daten unabhängig ohne Zutun des Fahrers erfasst und übertragen werden können bzw. die Präsenz keine zusätzliche Belastung erzeugt oder die Aktivierung nicht vergessen werden kann.

Um die erfassten Daten für das System nutzbar zu machen, muss es eine oder mehrere Komponenten geben, die die von vielen Fahrzeugen eingehenden Datenpunkte empfangen und gegebenenfalls aufbereiten und persistieren können. Dabei müssen die Daten so verarbeitet werden, dass sie für die weitere Nutzung im System zur Verfügung stehen und den entsprechenden Fahrzeugen zugeordnet werden können.

Um einen Fuhrpark schließlich verwalten und die erfassten Daten nutzen zu können, müssen diese in geeigneter Form dargestellt werden. Hierfür ist eine grafische Benutzeroberfläche erforderlich, die den Aufgaben der Fuhrparkverwaltung gerecht wird und einen übersichtlichen Zugriff auf die relevanten Informationen ermöglicht.

Die Problemstellung ergibt sich somit aus dem Zusammenspiel dieser drei Bereiche: der Erfassung und Übertragung der Fahrtdaten, der Verarbeitung und Speicherung der eingehenden Daten sowie deren Darstellung und Nutzung durch den Anwender.

1.3 Ziel der Arbeit

Ziel ist die Planung und Umsetzung aller benötigten Komponenten des Systems mit den Minimalfunktionen, um das System im Ganzen betreiben zu können.

Dazu gehört die Auswahl geeigneter Hardware-Komponenten sowie die Verdrahtung dieser zu einer Telemetrieeinheit für eine Nutzung im Fahrzeug. Die benötigte Software, mit der eine Telemetrieeinheit Fahrtdaten wie Geschwindigkeit und Ortsposition kontinuierlich erfassen und an das restliche System senden kann, wird geplant und umgesetzt.

Die Arbeit umfasst außerdem alle nötigen Komponenten, um von vielen Fahrzeugen gleichzeitig Fahrtdaten zu empfangen und diese, wenn nötig aufbereitet, abzuspeichern, sodass die aktuelle oder letzte Position eines Fahrzeugs sowie eine Rekonstruktion der Strecken mit allen zugehörigen Informationen (Fahrzeug, Fahrer, Datenpunkte) möglich ist.

Da sie für den Betrieb sowie für die Visualisierung der Daten nötig ist, ist die grafische Benutzeroberfläche ebenfalls Teil der Arbeit. Die Bereitstellung der Anwendung ist auch Teil der Arbeit.

1.4 Vorgehen

Die Vorgehensweise wird in den folgenden Kapiteln beschrieben. Kapitel 2 vermittelt das benötigte Grundlagenwissen zu Architekturmustern, eingebetteten Systemen, Datenbanktechnologien (sowie weiteren Themenbereichen), das bei der Erstellung der Arbeit zum Einsatz gekommen ist und im Rahmen des Studiums erworben wurde. So können Designentscheidungen nachvollzogen werden.

In Kapitel 3 wird untersucht, was die Anwendung überhaupt können soll, das heißt was konkret umgesetzt werden muss. Dazu werden Stakeholder identifiziert sowie funktionale und nichtfunktionale Anforderungen definiert, die eine Richtung bzw. eine Möglichkeit geben, später den Erfolg des Projekts objektiv überprüfen zu können. Die Anforderungsanalyse kann als Basis für weitere Implementierungen außerhalb dieses Projekts dienen.
Das Kapitel 4 geht auf die konkrete Aufteilung des Projekts in Unterprojekte, deren Entwurf sowie das Zusammenspiel und die Kommunikation zwischen den Kompo-

nenten ein. Es wird ein Architekturkonzept erarbeitet sowie ein Lösungskonzept für die einzelnen Komponenten.
Im Kapitel 5 wird die technische Umsetzung des Lösungskonzepts vorgestellt, wobei auf Abweichungen vom Lösungskonzept und auf dabei aufgetretene Probleme eingegangen wird. Ebenfalls behandelt wird dabei, wie die Software getestet wurde.
Kapitel 6 beschreibt die Überführung der Lösung in den Produktivbetrieb.

Abschließend bewertet das Kapitel 7, inwieweit die eingangs gestellten Anforderungen erfüllt wurden, und benennt bekannte Probleme und Grenzen der Lösung.
Kapitel 8 ordnet die Erweiterbarkeit und Übertragbarkeit der Ergebnisse ein.
Zum Abschluss fasst das Kapitel 9 die erzielten Ergebnisse zusammen und zieht ein Fazit.

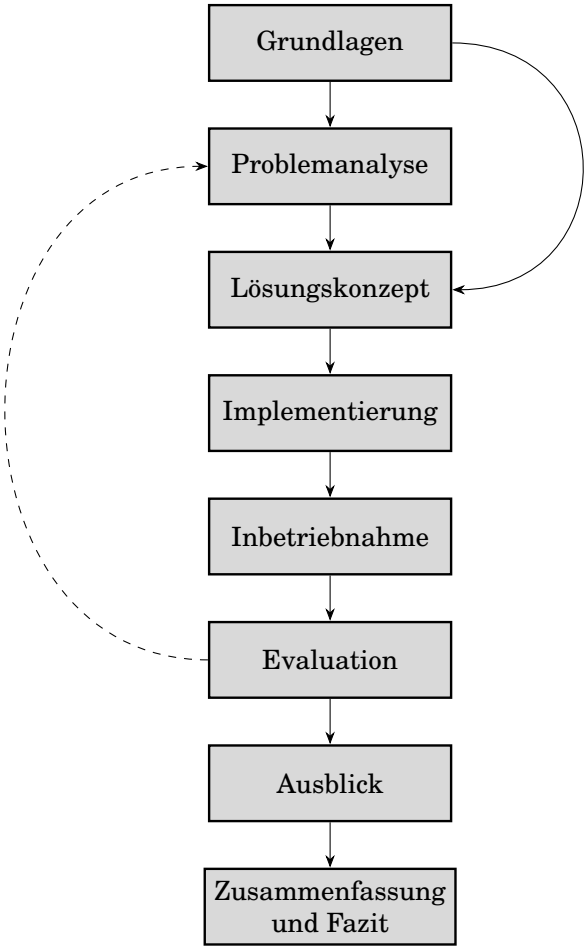


Abbildung 1.1: Vorgehen bei der Bearbeitung

2 Grundlagen

Bei der Erstellung der Arbeit eingeflossenes Wissen, welches im Rahmen des Studiums erworben wurde, wird nachfolgend dargestellt.

2.1 Software-Architektur

2.1.1 Microservices

Eine Microservice-Architektur ist ein verteilter Architekturstil, bei dem eine Anwendung in mehrere fachlich abgegrenzte und möglichst lose gekoppelte Services aufgeteilt wird. Die einzelnen Services bilden dabei jeweils einen eigenständigen fachlichen Verantwortungsbereich ab und können unabhängig voneinander betrieben und weiterentwickelt werden [12, S. 330].

Um die Eigenständigkeit zu erreichen, laufen die Komponenten (Services) in einem eigenen Prozess und kommunizieren über Schnittstellen miteinander.

Microservices partitionieren das System fachlich, d. h. eine Komponente enthält sämtliche Funktionalität, die zur Erledigung einer Aufgabe oder eines Aufgabenbereichs notwendig ist [12, S. 331]. Durch diese Eigenschaft lassen sich einzelne Services leicht unabhängig und damit auch parallel entwickeln.

Außerdem eignet sich dieser Architekturstil gut für die Skalierung bzw. Elastizität des Systems, da Instanzen der Services beliebig hoch- und heruntergefahren werden können und damit erweitert und bei Bedarf der Last angepasst werden kann. Die Fehlertoleranz steigt ebenfalls, da im Falle eines Absturzes nicht die gesamte Anwendung abstürzt, sondern nur eine Service-Instanz, die sofort durch Hochfahren einer neuen ersetzt werden kann.

Die Silbe „Micro“ im Namen des Architekturstils gibt dabei nicht vor, dass die Services zwingend sehr klein sein müssen [12, S. 332].

Ein Service kann selbst im Inneren weiter partitioniert werden, z. B. nach technischen Aufgaben. Aufrufe erfolgen über das Netzwerk, was zusätzliche Komplexität und Entwicklungsaufwand erfordert. Wenn die Anwendung zu klein ist, können die Kosten der Microservice-Architektur den Nutzen überwiegen.

2.1.2 Monolithische Architektur

Beim Monolithen befinden sich alle Funktionen in einer Anwendung, die in einem Prozess läuft. Die Anwendung kann somit nur als Ganzes bereitgestellt werden [12, S. 173].

Innerhalb des Monolithen können Verantwortungsbereiche in Komponenten getrennt werden, wodurch ein modularer Monolith entsteht [12, S. 173f.]. Eine technische Partitionierung, z. B. durch Schichten, kann zusätzlich auf oberster Ebene oder innerhalb der fachlichen Komponenten erfolgen [12, S. 174].

Da die Anwendung nur als Ganzes bereitgestellt werden kann, ist ihre Fehlertoleranz gering. Im Falle eines Absturzes fällt die gesamte Anwendung aus. Da die Komponenten innerhalb der Anwendung durch die gemeinsame Ausführung stark aneinander gekoppelt sind, erfordern Änderungen eine erneute Bereitstellung der gesamten Anwendung, was der Verfügbarkeit schadet. Skalierbarkeit und Elastizität sind ebenfalls eingeschränkt, da nur Instanzen des gesamten Systems hoch- und heruntergefahren werden können.

Ein Vorteil des Monolithen ist, dass er sich als geschlossene Einheit einfach entwickeln und bereitstellen lässt. Entwickelt man die Anwendung als modularen Monolithen, lässt sie sich durch die Komponententrennung auch gut parallel entwickeln. Da sich die einzelnen Komponenten im gleichen Prozess befinden, können Aufrufe ohne Netzwerkkommunikation erfolgen, was die Systemkomplexität reduziert und damit die Entwicklung vereinfacht.

Mit zunehmender Größe steigen jedoch auch die Nachteile. Überschreitet die Anwendung eine gewisse Größe, liegt es nahe, auf eine andere Architektur umzusteigen. Wählt man einen modularen Monolithen, ist der spätere Wechsel auf eine Microservice-Architektur besonders einfach, da die fachlichen Komponenten bereits voneinander getrennt sind [12, S. 183].

2.1.3 Domain-Driven Design

Domain-Driven Design ist ein Software-Designansatz, bei dem der fachliche Verantwortungsbereich als Grundlage für die Komponententrennung dient. Innerhalb der Software werden Geschäftsabläufe identifiziert und unabhängige, nach außen abgegrenzte Bereiche (Bounded Contexts) entworfen. Diese bilden die Basis für die Komponenten der Software [12, S. 111].

Die Idee des Bounded Context entspricht dem Designprinzip von Microservices. Jeder Microservice bildet einen Geschäftsbereich ab [12, S. 329].

2.1.4 Schichtenarchitektur

Innerhalb einer Anwendung oder Komponente einer Anwendung gibt es Teilabläufe, die sich auf unterschiedlichen Abstraktionsstufen befinden. Die Schichtenarchitektur strukturiert den Quellcode nach diesen Schichten, die jeweils einen bestimmten technischen Verantwortungsbereich haben [12, S. 163].

Jede Schicht ist von den anderen isoliert und kann nur die direkt darunterliegende Schicht verwenden. Die Isolierung macht die Schichten austauschbar und erleichtert somit die Wartbarkeit und Testbarkeit [12, S. 164]. Die Anwendung wird durch die Trennung ebenfalls einfacher zu erweitern und leichter zu verstehen.

Ein Nachteil der Schichtenarchitektur ist, dass durch die Aufteilung zusätzliche Klassen und Aufrufe entstehen, was die Anwendung verlangsamen kann. Wenn zu viele Aufrufe die Schichten durchlaufen, ohne dass innerhalb der Schichten eine nennenswerte Verarbeitung stattfindet, spricht man vom Sinkhole-Antipattern [12, S. 165].

2.1.5 Asynchrone Programmierung

Asynchrone Programmierung erlaubt es, auf potenziell langsame Operationen, wie Netzwerk- oder Datenbankzugriffe zu warten, ohne dabei den ausführenden Thread zu blockieren. Statt blockierend auf das Ergebnis zu warten, wird die Operation nur ausgelöst; sobald das Ergebnis vorliegt, wird die Ausführung an der Stelle fortgesetzt [11].
In vielen Programmiersprachen gibt es dafür die Schlüsselwörter `async`/`await`. Eine mit `async` markierte Methode kann an einer `await`-Stelle pausieren, wobei der Thread währenddessen für andere Aufgaben bzw. den Beginn der Bearbeitung einer weiteren Aufgabe freigegeben wird. Dadurch lassen sich mit wenigen Threads viele gleichzeitige Anfragen effizient verarbeiten [11].

2.2 Mobile and Embedded Software Development

2.2.1 Publisher-Subscriber-Modell, MQTT

Das Publisher-Subscriber-Muster ermöglicht es einem Sender (Publisher), Nachrichten an mehrere Empfänger (Subscriber) zu senden, ohne deren konkrete Identität kennen zu müssen. Subscriber können sich für den Erhalt bestimmter Nachrichten beim Publisher registrieren und erhalten diese bei deren Veröffentlichung [2, S. 225].

Ein Vorteil ist die geringe Kopplung zwischen Publisher und Subscriber. Das Hin-

zufügen neuer Subscriber erfordert keine Anpassung des Publishers. Damit eignet sich das Muster gut für Systeme, bei denen Nachrichten an mehrere unabhängige Komponenten verteilt werden müssen [2, S. 226].

Je nach Implementierung kann eine zusätzliche Komponente (Broker) zur Vermittlung der Nachrichten eingesetzt werden, deren Betrieb zusätzliche Infrastruktur erfordert. Ein Beispiel für ein solches broker-basiertes Publisher-Subscriber-System ist MQTT, ein für ressourcenbeschränkte Geräte und instabile Netzwerkverbindungen entwickeltes Protokoll, bei dem Publisher Nachrichten zu benannten Themen (Topics) veröffentlichen und Subscriber sich für die sie interessierenden Topics beim Broker registrieren, ohne den Publisher direkt zu kennen [2, S. 227].

2.3 Cloud and Distributed Systems

2.3.1 gRPC

Ein Remote Procedure Call (RPC) ermöglicht es, Methoden eines anderen Prozesses bzw. einer entfernten Anwendung auszuführen. Der Client behandelt den entfernten Aufruf wie einen lokalen Methodenaufruf, dem Parameter übergeben werden können. Nach Beendigung wird das Ergebnis vom Server an den Client übergeben. Per RPC können zwei getrennte Anwendungen über eine Schnittstelle kommunizieren.

gRPC ist ein Framework von Google für die Verwendung von Remote Procedure Calls in Anwendungen [15]. Die Schnittstellen werden mithilfe des Schemaformats Protocol Buffers (Protobuf) definiert, in dem das Nachrichtenformat und die Schnittstellen selbst festgelegt werden. Mit Hilfe dieses Formats werden Nachrichten serialisiert und deserialisiert. Nachrichten werden dabei binär dargestellt, wodurch sie kompakt übertragen werden können [15].

Neben der effizienten Kommunikation durch das Binärformat bietet gRPC starke Typisierung der Schnittstellen. Da der Quellcode zur Ausführung der Aufrufe für viele Sprachen aus den Proto-Dateien generiert werden kann, sinkt der Entwicklungsaufwand [15]. Die erhöhte Komplexität bei der Implementierung von gRPC-Schnittstellen kann ein Nachteil gegenüber Alternativen darstellen. Die binäre Darstellung der Nachrichten ist nicht menschenlesbar, was Tests erschwert.

2.3.2 REST

REST ist ein Entwurfsstil für Schnittstellen zwischen verteilten Anwendungen [6, Kap. 5]. Informationen werden als Ressourcen betrachtet, die über eindeutige URLs adressiert werden können. Die Verarbeitung erfolgt über HTTP-Methoden wie GET,

POST, PUT und DELETE.

Kommunikation nach REST soll zustandslos sein, d. h. alle zur Bearbeitung benötigten Informationen müssen übergeben werden [6, Kap. 5.1.3]. Typischerweise wird das menschenlesbare JSON als Nachrichtenformat verwendet, was die Fehlersuche vereinfacht. REST verwendet HTTP, was weit verbreitet ist und durch viele verschiedene Clients genutzt werden kann. Daher eignet sich REST besonders gut für öffentliche Schnittstellen.

Ein Nachteil von REST ist ein höherer Übertragungsaufwand aufgrund des textbasierten Nachrichtenformats sowie die schwache Typisierung, durch die Fehler oft erst zur Laufzeit erkannt werden können.

2.3.3 Containerisierung

Als Container bezeichnet man einen isolierten Prozess bzw. eine Prozessgruppe auf einem Linux-System, der sich mit den übrigen Prozessen nur den Kernel teilt [4].

Eine Anwendung kann unter Verwendung eines Container-Image (Bauplan für Container) zusammen mit allen Abhängigkeiten (Laufzeitumgebung, Bibliotheken, Konfiguration) zu einer isolierten, portablen Einheit verpackt werden [4].

Da alle nötigen Abhängigkeiten für den Container definiert sind und er isoliert mit diesen betrieben werden kann, ist die Portabilität von containerisierten Softwarekomponenten besonders gut. Änderungen im Deployment gehen dadurch besonders schnell, viele cloudbasierte Bereitstellungsdienste arbeiten mit Containern und erwarten containerisierte Komponenten [4].

Docker ist die verbreitetste Plattform zur Erstellung und zum Betrieb solcher Container [4].

2.4 Embedded Systems

2.4.1 Echtzeitbetriebssystem

Die Größe der Rechner, die in Telemetrieeinheiten zum Einsatz kommen, sollte dem Umfang der Anwendung entsprechen, um Platz und Stückkosten zu sparen. Kleine Mikrocontroller haben nicht genügend Ressourcen, um vollwertige Betriebssysteme zu verwenden. Benötigt man parallel laufende Aufgaben, muss man sich anders behelfen.

Eine Möglichkeit ist kooperatives Multitasking, wobei Funktionen so entworfen

werden, dass sie die CPU nie dauerhaft blockieren und immer nur für einen kurzen Moment entscheiden, ob etwas getan werden muss und wenn ja, was [13, S. 185]. Dieser Ansatz wird bei steigender Funktionalität schnell sehr kompliziert.

Eine weitere Lösung ist ein Echtzeitbetriebssystem (RTOS). Dabei handelt es sich um ein leichtgewichtiges Betriebssystem, das eine zeitlich vorhersehbare Ausführung einzelner Aufgaben (Tasks) ermöglicht (Echtzeit). Ein RTOS verwaltet die Prozessorressourcen, koordiniert die Ausführung der Aufgaben und sorgt für eine nebenläufige Verarbeitung im Zeitscheibenverfahren [13, S. 34]. Dadurch wird die Erstellung von Funktionen einfacher.

Durch die Verwendung erhöhen sich Rechen- und Speicheraufwand sowie die Komplexität und damit der Entwicklungsaufwand.

2.5 Objektorientierte Modellierung / Rechnerarchitektur

2.5.1 Zustandsautomaten

Ein Zustandsautomat (FSM) ist ein Modell zur Darstellung von Systemverhalten. Das System wird durch eine endliche Menge von benannten Zuständen beschrieben, in denen es sich befinden kann. Durch Ereignisse werden Übergänge in andere Zustände ausgelöst [5, S. 186].

Die Beschreibung eines Systems oder Teilsystems als Zustandsautomat kann helfen, Abläufe übersichtlicher und komplexe Steuerungslogik leichter nachvollziehbar zu machen. Zustandsautomaten sind leicht erweiterbar, die Testbarkeit wird ebenfalls erhöht.

2.6 Datenbanken / Data Engineering

2.6.1 Relationale Datenbanken

Relationale Datenbanken basieren auf der Theorie der Mengenlehre. Sie speichern Daten in Tabellen, die über Beziehungen (Relationen) miteinander verknüpft sind. Jede Tabellenzeile stellt einen Datensatz dar, jede Spalte beschreibt ein Attribut. Beziehungen zwischen Tabellen werden über Primär- und Fremdschlüssel hergestellt, die Abfrage der Daten erfolgt mit der Abfragesprache SQL [8].

Relationale Datenbanken erfüllen das ACID-Paradigma, Transaktionen garantieren Atomarität, Konsistenz, Isolation (Jeder Nutzer kann die Datenbank nutzen, als wäre er der einzige Nutzer) und Beständigkeit der Daten zu jedem Zeitpunkt [8].

Die Darstellung über Primär- und Fremdschlüssel erlaubt die eindeutige Abbildung von Datensätzen und deren Zusammenhängen. Das relationale Datenschema reduziert Redundanz, was zur Erhaltung der Datenintegrität über Verarbeitungen hinweg beiträgt [8].

Relationale Datenbanken sind für stark verzeigerte Daten geeignet und solche, bei denen Konsistenz wichtig ist. Durch die Garantie der ACID-Eigenschaften und die Verarbeitung der Beziehungen (Joins) kann der Aufwand für Schreibvorgänge höher ausfallen als bei nicht relationalen Datenbanken (NoSQL) [8].

2.6.2 Objektrelationales Mapping (ORM)

Ein objektrelationaler Mapper (ORM) bildet die Zeilen und Spalten einer relationalen Datenbank automatisiert auf Objekte und Klassen einer objektorientierten Programmiersprache ab. Anwendungscode arbeitet dadurch mit typisierten Objekten statt mit rohen SQL-Abfragen und manuellem Mapping der Ergebnismengen auf Datenstrukturen, was Boilerplate-Code reduziert und die Typsicherheit zur Kompilierzeit erhöht.
Das vom ORM generierte SQL ist für Entwickler weniger direkt kontrollierbar als handgeschriebenes SQL, was bei komplexen oder performancekritischen Abfragen zu ineffizienteren Ausführungen führen kann als eine manuell optimierte Abfrage [7, S. 98].

2.6.3 Zeitseriendatenbanken

Zeitseriendatenbanken sind auf die Speicherung großer Mengen zeitlich erfasster Daten optimiert [10]. Jeder Datenpunkt wird mit einem Zeitstempel und Metadaten gespeichert.

Relationale Datenbanken halten Daten sortiert in einem baumförmigen Index (B-Tree), um spätere Lesevorgänge zu beschleunigen. Beim Einfügen eines neuen Datensatzes muss zunächst die passende Position innerhalb der Baumstruktur gefunden werden, der Einfügevorgang besitzt eine Zeitkomplexität von $O(\log_b N)$ [3, S. 497]. N bezeichnet die Anzahl bereits gespeicherter Datensätze, b den Verzweigungsfaktor der Baumstruktur.

Zeitseriendatenbanken wie InfluxDB sindfür kontinuierliche Schreiboperationen ausgelegt: Neue Datenpunkte werden in einem Schreibpuffer (Write-Ahead-Log) sequenziell angehängt und erst später persistiert, statt wie bei einem B-Tree eine passende Einfügeposition suchen zu müssen [9]. Dadurch bleibt der Schreibaufwand pro Datenpunkt unabhängig von der bereits gespeicherten Datenmenge näherungsweise konstant.

3 Problemanalyse

3.1 Problem

Für den Betrieb eines Fuhrparks ist die aktuelle Position oder vergangene Positionen und gefahrene Routen von Fahrzeugen von besonderem Interesse. Dadurch lassen sich Fahrtenbücher kontrollieren oder sogar automatisiert ausfüllen und der aktuelle Aufenthaltsort der Fahrzeuge ist nachvollziehbar.

Um Auskunft über das Fahrverhalten der Mitarbeiter zu bekommen, sei es um potentiellen Verschleiß einzelner Fahrzeuge zu ermitteln oder um Sicherheitsaspekte überwachen zu können, müssen zumindest Geschwindigkeit und Beschleunigung erfasst werden.

Fahrzeuge dürfen nur von bestimmten autorisierten Fahrern gefahren werden. Diese müssen zentral gespeichert werden. Fahrer müssen mit einem Fahrzeug verknüpfbar sein. Einem Dienstwagen wird ein fester Fahrer zugeteilt.

Einem Fahrzeug muss eine Telemetrieeinheit eindeutig zugeordnet werden können, damit man nachvollziehen kann, für welches Fahrzeug und damit auch für welchen Fahrer gesammelte Daten erhoben wurden.

3.1.1 Stakeholder

Folgende Stakeholder wurden für die Anwendungsdomäne im betrieblichen Einsatz identifiziert. Dabei handelt es sich um Personen, die Anforderungen an das System stellen, mit diesem interagieren oder von dessen Ergebnissen betroffen sind. Bei einer Entwicklung bzw. Weiterentwicklung im betrieblichen Umfeld sollten die Interessen dieser Personen geachtet werden.

Fahrzeughalter / Betreiber des Fuhrparks: Der Betreiber des Fuhrparks ist an den Daten und deren Anwendungsmöglichkeiten interessiert.

Fahrer: Fahrer nutzen die ihnen zugeordneten Fahrzeuge. Sie sollen durch die Telemetrieeinheit nicht zusätzlich durch Arbeit belastet werden und müssen einem Einsatz zustimmen (DSGVO).

Systemadministrator: Ein Administrator ist für die Verwaltung der im System hinterlegten Fahrzeuge, Fahrer und Telemetrieeinheiten verantwortlich und der eigentliche Nutzer der Anwendung.

Betriebsrat: Da die Telemetrieeinheiten grundsätzlich geeignet sind, Verhalten und Leistung von Fahrern zu überwachen, ist auch der Betriebsrat Stakeholder.

Systementwickler: Entwickelt das System und wartet und erweitert es fortlaufend, hat daher auch ein Interesse.

3.2 Anforderungen

3.2.1 Funktionale Anforderungen

FR 1: Telemetrieeinheit (Hardware)

FR 1.1: Eine Telemetrieeinheit muss Geschwindigkeit, Beschleunigung und GPS-Position erheben können.

FR 1.2: Eine Telemetrieeinheit muss die erhobenen Daten über LTE senden können.

FR 1.3: Eine Telemetrieeinheit muss mit Fehlerzuständen umgehen und nach Störungen wieder Daten erheben und senden können, sobald dies wieder möglich ist (GPS-Fix vorhanden, gültige Internetverbindung).

FR 2: Telemetrieeinheiten (System)

FR 2.1: Das System muss Telemetrieeinheiten eindeutig registrieren und löschen können.

Die Löschung soll nur möglich sein, wenn keine Fahrt für die Telemetrieeinheit gespeichert ist, d. h. die Fahrten der Telemetrieeinheit müssen vorher gelöscht werden. Beim Löschen einer Telemetrieeinheit soll eine Verbindung zu einem Fahrzeug, falls vorhanden, gelöscht werden.

FR 2.2: Das System muss Telemetrieeinheiten mit einem Fahrzeug eindeutig verbinden und diese Verbindung wieder trennen können.

FR 2.3: Das System muss es erlauben, gespeicherte Telemetrieeinheiten unter der Verwendung von Filtern zu suchen und deren aktuellen Zustand in einer Detailansicht auszugeben. Das Filtern soll sowohl additiv (Alle, auf die mindestens ein Kriterium zutrifft) als auch restriktiv (alle Kriterien müssen zutreffen)

möglich sein.

FR 3: Fahrzeuge

FR 3.1: Das System muss die eindeutige Anlage und Löschung von Fahrzeugen mit fahrzeugbezogenen Daten erlauben. Die Löschung von Fahrzeugen soll nur möglich sein, wenn alle Fahrten des Fahrzeugs ebenfalls vorher gelöscht wurden. Eine vorhandene Verbindung zu einer Telemetrieeinheit oder einem Fahrer soll automatisch entfernt werden.

Für Fahrzeuge sollen folgende Daten gespeichert werden:

- **Fahrzeug-Ident.-Nr.:** Soll als ID dienen, da sie bereits einzigartig ist.
- **Kennzeichen und Erstzulassung:** Sind optional, da ein Auto auch speicherbar sein soll, bevor es zugelassen wurde bzw. ein Auto auch wieder ab- und angemeldet werden kann.
- **Wagenfarbe:** Pflichtangabe
- **Marke:** Pflichtangabe
- **Modell:** Pflichtangabe
- **Baujahr:** Pflichtangabe
- **Leistung in PS:** Pflichtangabe
- **Benötigter Führerschein:** Pflichtangabe

Hinzu kommen die optionalen Verbindungen zu Fahrer und Telemetrieeinheit.

FR 3.2: Das System muss eine Überarbeitung der Fahrzeugdaten erlauben. Die Fahrzeug-Ident.-Nr. soll nicht änderbar sein, da sie mit einem Fahrzeug fest verbunden ist. Bei einem Fehler muss das Fahrzeug gelöscht und neu angelegt werden.

FR 3.3: Das System muss die letzte gespeicherte Position eines Fahrzeugs mit Angaben zu Fahrer, Telemetrieeinheit sowie den Telemetriedaten auf einer Karte anzeigen und selbstständig aktualisieren können. Bewegt sich das Fahrzeug gerade, d. h. liegt ein Positionsupdate nicht länger als eine festgelegte Zeitspanne zurück, soll dies graphisch auf der Karte unterscheidbar sein von Fahrzeugpositionen, die sich länger nicht mehr geändert haben und die Fahrzeuge damit parken und nicht unterwegs sind.

FR 3.4: Es soll möglich sein, Fahrzeuge gefiltert zu suchen, in einer Detailansicht zu

betrachten und die Karte mit den aktuellen Standorten / letzten Positionen auf die Filterausgabe zu beschränken. Das Filtern soll sowohl additiv (Alle, auf die mindestens ein Kriterium zutrifft) als auch restriktiv (alle Kriterien müssen zutreffen) möglich sein.

FR 4: Fahrer

FR 4.1: Das System muss Personen (Fahrer) mit personenbezogenen Daten registrieren und löschen (DSGVO) können.

Die Löschung soll nur möglich sein, wenn keine Fahrt für den Fahrer gespeichert ist, d. h. die Fahrten des Fahrers müssen vorher gelöscht werden. Beim Löschen eines Fahrers soll eine Verbindung zu einem Fahrzeug, falls vorhanden, gelöscht werden.

Gespeichert werden sollen:

- **Vorname**
- **Nachname**
- **Geburtsdatum**
- **Führerscheine:** Mindestens einer muss ausgewählt werden.

FR 4.2: Das System muss eine eindeutige Zuordnung von Fahrer und Fahrzeug erlauben und die Änderung bzw. Entfernung der Zuweisungen erlauben.

FR 4.3: Es soll möglich sein, Fahrer gefiltert zu suchen und in einer Detailansicht zu betrachten. Das Filtern soll sowohl additiv (Alle, auf die mindestens ein Kriterium zutrifft) als auch restriktiv (alle Kriterien müssen zutreffen) möglich sein.

FR 5: Fahrten

FR 5.1: Das System muss Fahrtrouten auf einer Karte anzeigen können. Im Abstand des Sendeintervalls sollen die Datenpunkte auf der gezeichneten Route nachvollziehbar sein.

FR 5.2: Es soll möglich sein, die Fahrtdaten einer einzelnen Fahrt auf einem Graphen zu betrachten.

FR 5.3: Das System soll das Filtern von Fahrten sowie deren Betrachtung in einer Detailansicht erlauben. Das Filtern soll sowohl additiv (Alle, auf die mindes-

tens ein Kriterium zutrifft) als auch restriktiv (alle Kriterien müssen zutreffen) möglich sein.

FR 5.4: Das System soll die Löschung der Fahrten erlauben (DSGVO).

3.2.2 Nicht funktionale Anforderungen

NFR 1 Performance: Eine Telemetrieeinheit soll bei bestehender Verbindung mindestens einen Datenpunkt pro Sekunde übertragen können.

NFR 2 Skalierbarkeit: Das System soll die gleichzeitige Verarbeitung von Daten mehrerer Telemetrieeinheiten unterstützen.

NFR 3 Portabilität: Die Benutzeroberfläche soll als browserbasierte Webanwendung bereitgestellt werden und ohne zusätzliche Client-Software nutzbar sein.

NFR 4 Wartbarkeit: Das System soll modular aufgebaut sein, um zukünftige Erweiterungen und Anpassungen zu erleichtern.

Die folgende Priorisierung wurde vorgenommen:

NFR	Eigenschaft	Priorität
NFR 1	Performance	Hoch
NFR 2	Skalierbarkeit	Hoch
NFR 3	Portabilität	Mittel
NFR 4	Wartbarkeit	Mittel

3.2.3 Anwendungsfälle

Im Folgenden werden nicht alle Anwendungsfälle des Systems aufgeführt, sondern nur eine repräsentative Auswahl.

Telemetrieeinheit registrieren

Akteur: Systemadministrator

Vorbedingung: Die Telemetrieeinheit ist per USB mit dem Rechner, auf dem die Anwendung bedient wird, verbunden.

Ablauf: 1. Der Administrator startet in der Benutzeroberfläche die Registrierung einer neuen Telemetrieeinheit.

2. Die Benutzeroberfläche fordert über die USB-Verbindung die Geräte-ID von der Telemetrieeinheit an.
3. Die Telemetrieeinheit übermittelt ihre eindeutige Geräte-ID.
4. Der Administrator legt mithilfe der Geräte-ID die Telemetrieeinheit im System an.

Nachbedingung: Die Telemetrieeinheit ist im System angelegt und kann anschließend mit einem Fahrzeug verbunden werden.

Fehlerfall: Die Geräte-ID bereits vergeben, dann ist die Registrierung nicht erneut möglich.

Fahrzeug anlegen

Akteur: Systemadministrator

Vorbedingung: Anwendung geöffnet.

Ablauf:
1. Der Administrator startet die Fahrzeuganlage.
2. Der Administrator gibt die Fahrzeug-Ident.-Nr. sowie die Pflichtangaben (Wagenfarbe, Marke, Modell, Baujahr, Leistung, benötigter Führerschein) ein; Kennzeichen und Erstzulassung können optional angegeben werden.
3. Der Administrator bestätigt die Eingabe.

Nachbedingung: Das Fahrzeug ist im System angelegt und kann anschließend mit einer Telemetrieeinheit und einem Fahrer verbunden werden.

Fehlerfall: Ist die Fahrzeug-Ident.-Nr. bereits vergeben oder fehlt eine Pflichtangabe, wird die Anlage mit einer Fehlermeldung abgelehnt.

Fahrer anlegen

Akteur: Systemadministrator

Vorbedingung: Anwendung geöffnet.

Ablauf:
1. Der Administrator startet die Personen- bzw. Fahreranlage.
2. Der Administrator gibt Vorname, Nachname und Geburtsdatum des Fahrers ein und wählt mindestens einen Führerschein aus.
3. Der Administrator bestätigt die Eingabe.

Nachbedingung: Der Fahrer ist im System angelegt und kann anschließend einem Fahrzeug zugewiesen werden.

Fehlerfall: Wird kein Führerschein ausgewählt oder fehlt eine Pflichtangabe, wird die Anlage mit einer Fehlermeldung abgelehnt.

Telemetrie-Datenpunkt erheben und senden

Akteur: Telemetrieeinheit

Vorbedingung: Die Telemetrieeinheit ist mit Strom versorgt, mit dem Mobilfunknetz verbunden und beim MQTT-Broker angemeldet. Ein gültiger GNSS-Fix liegt vor.

Ablauf:

1. Die Telemetrieeinheit fragt im Sendeintervall die aktuelle GPS-Position sowie den Zeitstempel über das GNSS-Modul ab.
2. Es werden Geschwindigkeit und Beschleunigung berechnet.
3. Die erhobenen Werte werden zu einem Datenpunkt im JSON-Format zusammengefasst.
4. Der Datenpunkt wird über MQTT an den Broker gesendet.

Nachbedingung: Der Datenpunkt wurde erfolgreich an den Broker übertragen.

Fehlerfall: Liegt kein gültiger GNSS-Fix oder keine Netzwerkverbindung vor, wird kein Datenpunkt erhoben bzw. gesendet. Schlägt das Senden mehrfach hintereinander fehl, wechselt die Telemetrieeinheit in einen Fehlerzustand und versucht nach Wiederherstellung der Verbindung automatisch erneut zu senden.

Telemetrie-Datenpunkt verarbeiten

Akteur: System

Vorbedingung: Ein Datenpunkt einer Telemetrieeinheit wurde über den MQTT-Broker empfangen.

Ablauf:

1. Das System ermittelt anhand der Geräte-ID das zugeordnete Fahrzeug bzw. den zugeordneten Fahrer.
2. Es wird ermittelt, ob eine neue Fahrt beginnt, eine bestehende Fahrt fortgesetzt wird oder eine laufende Fahrt endet.

3. Der Datenpunkt wird zusammen mit dem Fahrtbezug persistent in der Zeitseriendatenbank gespeichert.
4. Der Datenpunkt wird zur Live-Aktualisierung an die Benutzeroberfläche weitergeleitet.

Nachbedingung: Der Datenpunkt ist gespeichert, der Fahrtzustand ist gegebenenfalls aktualisiert und die Kartenansichten der Benutzeroberfläche berücksichtigen die Änderung.

Fehlerfall: Kann die Geräte-ID keinem registrierten Fahrzeug zugeordnet werden, wird der Datenpunkt verworfen.

Fahrer löschen

Akteur: Systemadministrator

Vorbedingung: Der Fahrer ist im System angelegt.

Ablauf:
1. Der Administrator wählt den zu löschenden Fahrer aus.
2. Das System prüft, ob dem Fahrer noch Fahrten zugeordnet sind.
3. Sind Fahrten vorhanden, muss der Administrator diese zunächst löschen, bevor die Löschung des Fahrers fortgesetzt werden kann.
4. Der Administrator bestätigt die Löschung des Fahrers.
5. Das System entfernt den Fahrer inklusive einer eventuell bestehenden Verbindung zu einem Fahrzeug.

Nachbedingung: Der Fahrer sowie seine personenbezogenen Daten sind vollständig aus dem System entfernt (DSGVO).

Fehlerfall: Sind dem Fahrer noch Fahrten zugeordnet, wird die Löschung mit einer Fehlermeldung abgelehnt.

3.2.4 Abnahmekriterien

Die Anwendung gilt als abgabefertig, wenn die in Abschnitt 3.2 aufgeführten Anforderungen erfüllt sind.

4 Lösungskonzept

4.1 Systemarchitektur

Damit Fahrzeuge Fahrtdaten erheben und senden können, muss es eine Telemetrieeinheit im Fahrzeug geben. Somit ergibt sich die Komponente **TelemetryUnit**.

Um die gespeicherten Daten darzustellen und Nutzeraktionen durchführen zu können, braucht die Anwendung eine Benutzeroberfläche (**FleetControlUI**), die den gesamten Anwendungszustand abfragen kann und entsprechende Aktionen umsetzen kann.

Die Anwendung muss außerdem eine Datenbasis aufbauen bzw. verwalten können und die Bearbeitung von Abfragen des Systemzustands durch die Benutzeroberfläche ermöglichen.

Da die Anwendung hohe Skalierbarkeit, Elastizität und Erweiterbarkeit priorisieren soll, wurde sich für eine Microservice-Architektur (siehe Abschnitt 2.1.1) entschieden. Ein (modularer) Monolith (siehe Abschnitt 2.1.2) wurde bewusst nicht gewählt, da bei diesem trotz interner Komponententrennung stets die gesamte Anwendung als Einheit bereitgestellt und skaliert werden müsste, was den priorisierten Zielen unabhängiger Skalierbarkeit und Elastizität entgegensteht. Ein weiterer Grund ist die starke Eignung für den Einsatz im Cloud-Umfeld, in dem es viele Dienste gibt, die speziell auf den Betrieb von Microserviceanwendungen ausgelegt sind.

Um die restlichen Komponenten, d. h. die Dienste (Services), zu bestimmen, wurde das Prinzip des begrenzten Kontexts aus dem DDD (siehe Abschnitt 2.1.3) verwendet. Dazu wurden unabhängige Arbeitsabläufe identifiziert und in Komponenten (Dienste) übersetzt. Diese Aufteilung nach Arbeitsabläufen sowie die Isolierung der Bereiche (Bounded Context) entspricht dem Microservice-Gedanken und eignet sich daher gut für die Abgrenzung der Dienste.

Es ergeben sich drei weitere Arbeitsablaufbereiche und die daraus abgeleiteten Dienste:

Der erste Bereich umfasst den Erhalt und die Verarbeitung der eingehenden Telemetriedaten. Daten werden zur späteren Nutzung aufbereitet und persistiert. Dies ergibt den Dienst **IngestionService**.

Ein weiterer Bereich umfasst die Abfrage und Steuerung des allgemeinen Systemzustands von der Benutzeroberfläche aus. Dazu gehört die Abfrage, Anlage, Änderung und Löschung von Fahrern, Fahrzeugen, Telemetrieeinheiten und Fahrten und deren Verknüpfungen. Der abgeleitete Dienst **FleetControlApi** bildet diesen Bereich ab. Der Systemzustand besteht grundlegend aus Verknüpfungen (verzeigerte Daten). Telemetrieeinheiten können Fahrzeugen zugewiesen werden, denen wiederum ein Fahrer zugeteilt werden kann. Erhobene Fahrtdaten beziehen sich auf diese Verknüpfungen. Es handelt sich um relationale Daten (siehe Abschnitt 2.6.1). Daher wurde eine relationale Datenbank, genauer **PostgreSQL**, zur Speicherung verwendet. PostgreSQL wurde gewählt, da bereits Erfahrungen mit dieser Datenbanktechnologie durch besuchte Vorlesungen bestanden.

Die eigentlichen Fahrtdaten, d. h. die Datenpunkte, befinden sich auf einer anderen Granularitätsstufe als die abstrakten Fahrten selbst. Sie benötigen wenigstens ein eigenes Datenmodell, wenn nicht sogar eine eigene Datenbank. Bei den Fahrtdaten handelt es sich um hochfrequente Datenpunkte, die kontinuierlich über einen Zeitraum hinweg erfasst und mit einem Zeitstempel gespeichert werden. Die Daten werden anschließend zeitraumbasiert ausgewertet. Dabei handelt es sich um Zeitseriendaten (siehe Abschnitt 2.6.3). Die Anwendung *FleetControl* soll auch für große Fahrzeugflotten einsetzbar sein, d. h. Nachrichten von vielen, sich in Bewegung befindenden Fahrzeugen gleichzeitig bearbeiten können. Um dem Skalierungsgedanken gerecht zu werden, soll eine dedizierte Zeitseriendatenbanktechnologie, genauer **InfluxDB**, zum Einsatz kommen. *InfluxDB* bietet den Vorteil, dass die Einfügerate auch bei großen Mengen bereits gespeicherter Fahrtdaten nicht so stark abnimmt, wie es beispielsweise bei relationalen Datenbanktechnologien wie *PostgreSQL* der Fall wäre. Die Abfrage der Fahrtdaten stellt einen weiteren Zuständigkeitsbereich dar, es ergibt sich der reine Abfragedienst **DriveDataService**.

Zusätzlich zu den fachlichen Diensten wird **Caddy** als Reverse Proxy vor *FleetControlUI* und *FleetControlApi* eingesetzt, um beide Dienste ohne eigene TLS-Konfiguration per HTTPS erreichbar zu machen (siehe Abschnitt 5.1.5). Die Notwendigkeit wurde erst im Zuge der Implementierung festgestellt, soll hier aber schon erwähnt werden.

4.2 Kommunikation

Viele Telemetrieeinheiten müssen an den *IngestionService* viele Datenpunkte senden. Um Kopplung zu vermeiden und die Erweiterung der Anwendung zu erleichtern, soll die Kommunikation über ein Publish-Subscribe-Modell (siehe Abschnitt 2.2.1) ablaufen. Da Sender und Empfänger voneinander entkoppelt sind und sich nicht gegenseitig kennen müssen, können die beteiligten Komponenten unabhängig voneinander skaliert werden. Dadurch kann beispielsweise die Anzahl der Instanzen eines

Dienstes entsprechend der anfallenden Last erhöht werden, ohne dass die Sender angepasst werden müssen. Dazu wurde MQTT (Mosquitto) eingesetzt, welches das Publish-Subscribe-Modell unter Verwendung einer zentralen Broker-Komponente implementiert.

Für die Systemanlage einer Telemetrieeinheit soll diese über USB mit dem Rechner verbunden werden. Durch das Senden eines Befehls soll eine einzigartige Geräte-ID zurückgegeben werden, mit der dann die Hardware eindeutig im System registriert werden kann. In der Planungsphase wurde diese Funktionalität fälschlicherweise *FleetControlApi* zugeordnet (siehe Abschnitt 5.1.5). Der USB-Zugriff muss stattdessen in *FleetControlUI* erfolgen. Damit das finale Kommunikationsmodell nachvollziehbar ist, wird hier bereits die finale, funktionierende Version dargestellt.

Beim Eintreffen eines Datenpunkts muss *IngestionService*, falls noch nicht geschehen, eine Fahrtanlage in *FleetControlApi* auslösen. Der Fahrtabschluss muss ebenfalls in *FleetControlApi* angeordnet werden.

Die Abfrage der Fahrtdaten wird ebenfalls nicht direkt von der Benutzeroberfläche durchgeführt. *FleetControlUI* kommuniziert nur mit *FleetControlApi*. Der Dienst führt die eigentliche Abfrage der Daten bei *DriveDataService* durch und gibt die Daten an die Benutzeroberfläche zurück. So muss die Benutzeroberfläche nicht wissen, wie die Backendkomponenten aufgeteilt sind. Die spätere Änderung dieser Komponenten wird dadurch erleichtert, dass die Benutzeroberfläche bei Änderungen am Backend unverändert bleiben kann.

Für die Kommunikation zwischen *FleetControlApi* und der Benutzeroberfläche wurde REST gewählt (siehe Abschnitt 2.3.2): Da die Benutzeroberfläche im Browser läuft und dort öffentlich erreichbar sein muss, ist eine breite Client-Unterstützung wichtiger als maximale Effizienz. Die ressourcenbasierte Sichtweise auf Fahrzeuge, Fahrer und Fahrten passt zudem gut zum REST-Modell. Da die Schnittstelle öffentlich ist bzw. zwischen Benutzeroberfläche und dem übrigen System sitzt, macht es Sinn, wenigstens diese Schnittstelle menschenlesbar zu gestalten.

Für die Kommunikation zwischen den Backend-Diensten untereinander (*IngestionService*, *FleetControlApi*, *DriveDataService*) wurde dagegen gRPC eingesetzt (siehe Abschnitt 2.3.1): Diese Dienste rufen sich sehr häufig und selbstständig auf, sodass die starke Typisierung und das effiziente Binärformat von gRPC hier vorteilhaft sind. Der Nachteil der schlechten Lesbarkeit binärer Nachrichten fällt dabei kaum ins Gewicht, da es sich um interne Aufrufe zwischen selbst entwickeltem Code handelt und keine externen Clients oder Menschen die Nachrichten direkt einsehen müssen, wie es bei der öffentlichen REST-Schnittstelle zur Benutzeroberfläche der Fall ist.

Um automatische Karten-Updates auszulösen, statt eine manuelle Aktualisierung der Benutzeroberfläche durch den Anwender zu erfordern, muss auch *IngestionSer-*

vice Nachrichten an *FleetControlUI* senden (ein Ereignis, das über das Eintreffen eines Datenpunkts informiert). In der Planungsphase des Projekts wurde dies nicht ausreichend bedacht. Die Notwendigkeit weiterer Infrastruktur wurde erst während der Entwicklung festgestellt (siehe Abschnitt 5.0.4). Damit das Kommunikationskonzept der gesamten Anwendung verständlich wird, soll die Ergänzung an dieser Stelle trotzdem schon beschrieben werden.

Da *FleetControlUI* zur Vereinfachung der Wartbarkeit ausschließlich mit *FleetControlApi* kommunizieren soll, kann *IngestionService* die Benutzeroberfläche nicht direkt über neu eingetroffene Datenpunkte informieren. Die Benachrichtigung muss daher über *FleetControlApi* weitergeleitet werden.

Für die Meldung von *IngestionService* an *FleetControlApi* wird erneut ein Publish-Subscribe-Modell (siehe Abschnitt 2.2.1) verwendet, umgesetzt über einen Eventbus (*RabbitMQ*). Auf leistungsfähigere, aber komplexere Alternativen wie *Kafka* kann dabei verzichtet werden, da deren zusätzliche Funktionen, etwa ein sehr hoher Durchsatz oder eine dauerhafte Speicherung des Ereignis-Verlaufs, für den vorliegenden Anwendungsfall nicht benötigt werden.

FleetControlApi abonniert dabei die von *IngestionService* veröffentlichten Ereignisse. Ein Browser könnte dies nicht selbst tun, da er das von *RabbitMQ* verwendete Nachrichtenprotokoll nicht unterstützt.

Die vom Eventbus empfangene Nachricht muss von *FleetControlApi* anschließend an alle im Browser geöffneten Instanzen von *FleetControlUI* weitergereicht werden.

Alternativ hätte für diese Benachrichtigung auch hier gRPC anstelle des Eventbus verwendet werden können, um sich die Komponente zu sparen. Ein direkter Aufruf würde *IngestionService* jedoch dazu zwingen, sich um potentielles Fehlschlagen zu kümmern, etwa wenn *FleetControlApi* gerade nicht erreichbar ist. Fehlschlagen soll aber bewusst folgenlos bleiben (Feuern und vergessen).

Die Weiterleitung soll über *SignalR* umgesetzt werden, womit über eine dauerhaft offene Verbindung serverseitig ausgelöste Ereignisse direkt an die Clients gesendet werden können. Die Clients müssen somit nicht wiederholt nach potentiellen Ereignissen fragen.

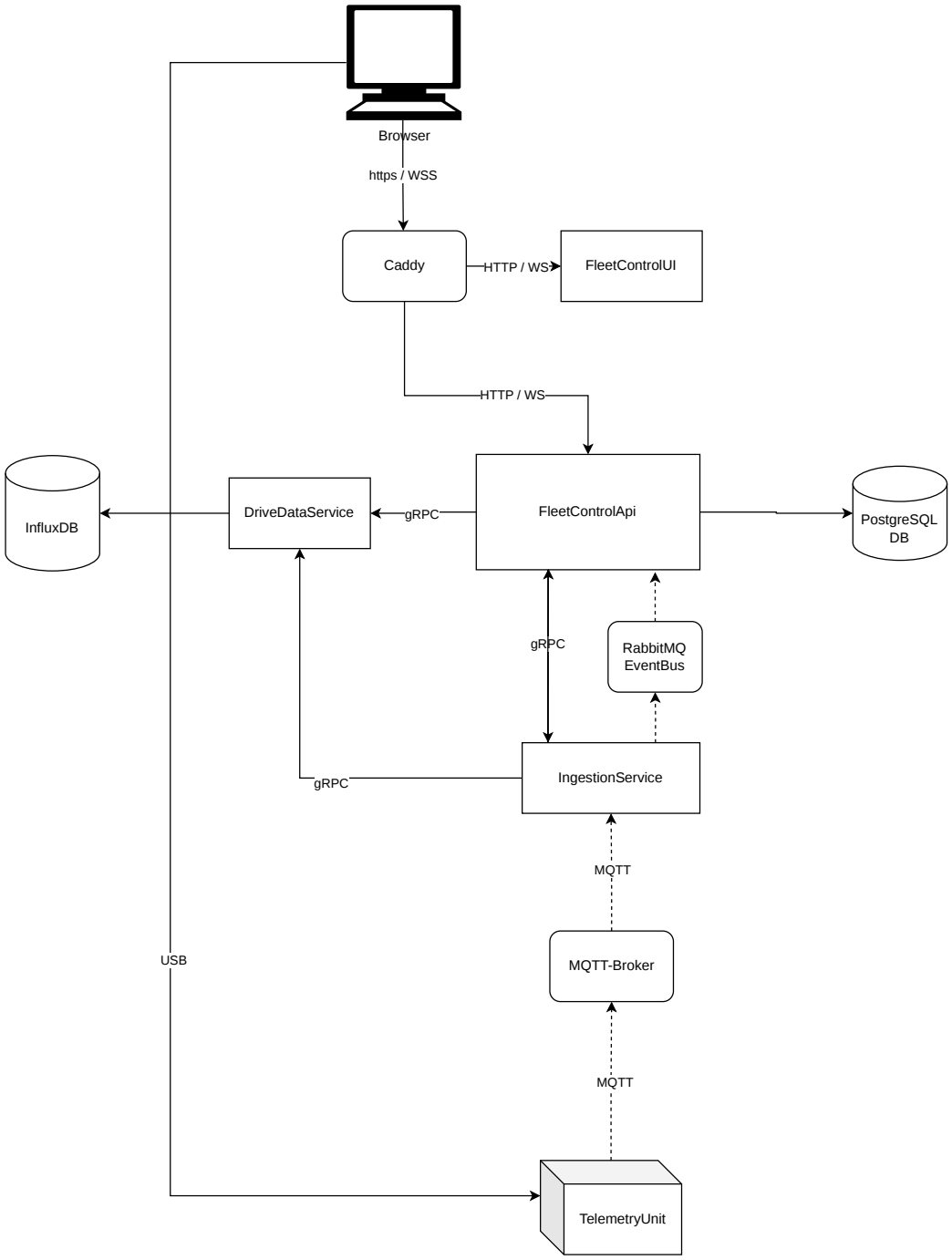


Abbildung 4.1: Architekturdiagramm der endgültigen Komponenten und ihrer Kommunikationswege

4.3 Komponenten

4.3.1 TelemetryUnit

Hardware

Für die Telemetrieeinheit wurde ein **STM32F103RB** mit einem ARM Cortex-M3-Kern gewählt, eine Entwicklungsplatine, die mit allen für die Programmierung benötigten Hardwarekomponenten, GPIO-Pins, einem Taster, einer LED sowie der Unterstützung gängiger Kommunikationsschnittstellen (UART, I2C, CAN) ausgestattet ist. Dieses Entwicklungsboard war aus dem vorherigen Verlauf des Studiums bereits bekannt, sodass ersichtlich war, dass die Speicherressourcen für die Umsetzung der Telemetrieeinheit ausreichen.

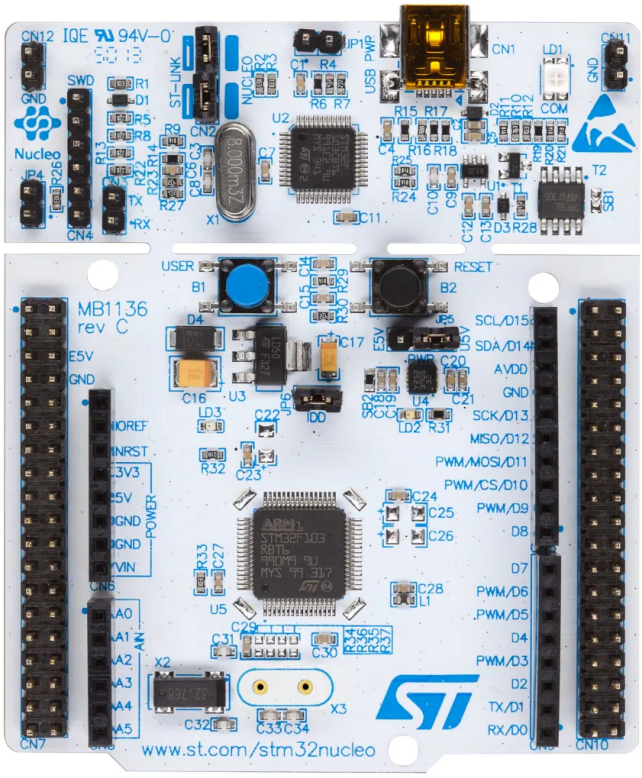


Abbildung 4.2: STM32F103RB Nucleo Entwicklungsplatine

Der Hersteller des Mikrocontrollers, ST-Microelectronics, bietet mit dem Programm *STM32CubeMX* eine Möglichkeit, STM32-Basisprojekte zu generieren und die Hardware graphisch zu konfigurieren mit anschließender Quellcodeerzeugung für die Konfiguration der Peripherie, was den Entwicklungsprozess deutlich verkürzt. Dies

war auch ein Grund, warum sich für diese Hardware entschieden wurde.

Die Telemetrieeinheit soll mindestens Geschwindigkeit, Beschleunigung, Uhrzeit und Koordinaten erheben und diese über das Internet übertragen können. Für die Bestimmung der Beschleunigung bzw. der Geschwindigkeit soll die Standard-IMU-Sensoreinheit **MPU-6050** (Gyroskop/Neigung, Beschleunigung, Temperatur) verwendet werden. Die Einheit kommuniziert über I2C, welches die gewählte Mikrocontrollerplatine unterstützt.

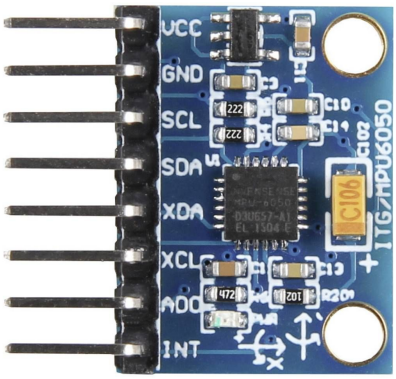


Abbildung 4.3: IMU-Sensoreinheit MPU6050

Für die Erhebnung der Ortsposition, der Uhrzeit und die Datenübertragung wurde ein LTE/GNSS-Modul vom Typ **Waveshare SIM7600E-H** gewählt. Das Modul kann mit einer Simkarte über LTE Nachrichten senden und empfangen, die Ortsposition bestimmen sowie UTC-Zeit ausgeben, außerdem unterstützt das Modul MQTT. Bedient wird es über UART mit AT-Befehlen, sodass der STM32-Mikrocontroller mit dem LTE/GNSS-Modul kommunizieren kann.



Abbildung 4.4: Waveshare SIM7600E-H 4G HAT Entwicklungsplatine

Zum Betrieb werden ebenfalls eine GPS-Antenne und eine LTE-Antenne benötigt. Beide Antennen lassen sich mit der Platine im Komplettpaket erwerben.



Abbildung 4.5: GPS-Antenne



Abbildung 4.6: LTE-Antenne

Sowohl das verwendete STM32-Nucleo-Board als auch das LTE/GNSS-Modul [16] können mit 5 V versorgt werden, die intern auf 3,3 V heruntergeregelt werden. Im Rahmen dieses Projekts soll die Telemetrieeinheit nicht fest mit der Elektronik des Testfahrzeugs (Audi A3 8P von 2006) verbaut werden. Daher wurde die 12-V-Zigarettenanzündebuchse verwendet, um die benötigte Spannung bereitzustellen. Die 12-V-Buchse liefert typischerweise 12,0–12,8 V, wenn der Motor ausgeschaltet ist, und 13,5–14,3 V bei laufendem Motor [1, S. 4]. Um die Telemetrieeinheit in einem beliebigen Fahrzeug zu verwenden, muss eine Spannungswandlung auf die benötigten 5 V erfolgen. Dies geschieht mit einem DC/DC-Spannungswandler, in diesem Fall ein Step-Down-/Buck-Converter. Ein handelsüblicher USB-Adapter für die Zigarettenanzündebuchse enthält genau so einen Spannungswandler. Der verwendete Adapter kann mit 12–24 V Gleichstrom als Eingangsspannung umgehen und liefert konstante 5 V bei maximal 3,1 A auf jedem der zwei USB-Anschlüsse.

Die Mikrocontroller-Platine und die Platine des LTE/GNSS-Moduls müssen jeweils

eine eigene 5-V-Spannungsversorgung haben, d. h. jede Platine muss durch einen separaten Anschluss versorgt werden. Das LTE/GNSS-Modul kann bei der Datenübertragung kurzzeitig einen hohen Stromverbrauch haben, wodurch es zu Spannungseinbrüchen kommen kann. Werden beide Platinen nicht von getrennten Spannungsquellen versorgt sondern eine Platine versorgt die andere, kann die Gesamtspannung unter die benötigte Mindestspannung fallen und es kann zu ungewollten Neustarts des STM32-Mikrocontrollers kommen. Der verwendete USB-Adapter teilt die Bordnetzspannung auf zwei USB-Anschlüsse auf, die mit jeweils konstanten 5 V der Aufgabe gerecht werden.

Der STM32 kommuniziert mit dem LTE/GNSS-Modul über UART. Die beiden Platinen werden über die benötigten Pins verbunden:

- RX (STM32) nach TXD (LTE/GNSS-Platine)
- TX (STM32) nach RXD (LTE/GNSS-Platine)
- Gnd (STM32) nach Gnd (LTE/GNSS-Platine), um Signale gegen ein gemeinsames Nullpotential zu vergleichen.

Der Anschluss der IMU-Einheit MPU-6050 wird nicht dargestellt, da sich am Ende gegen die Verwendung entschieden wurde (siehe Abschnitt 5.1.5) und die folgende Darstellung zum Verständnis der endgültigen Hardware herangezogen werden soll.

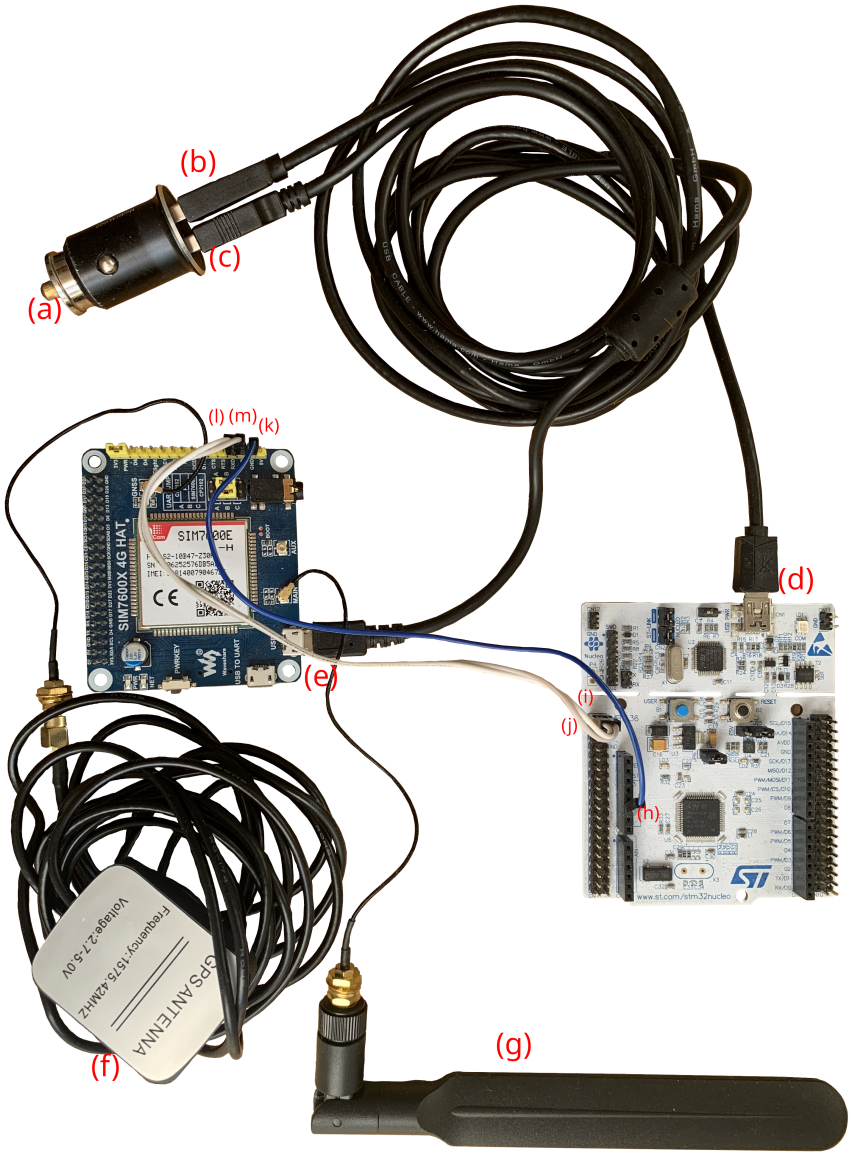


Abbildung 4.7: Verdrahtete Telemetrieeinheit ohne MPU6050

(a) 12 V an 2x 5 V USB-Buchse
(b) 5 V an USB (Mikrocontroller)
(c) 5 V an USB (LTE/GNSS-Modul)
(d) 5 V (Mikrocontroller)
(e) 5 V (LTE/GNSS-Modul)
(f) GPS-Antenne
(g) LTE-Antenne
(h) Bezugspotential (Gnd)
(i) RX (Empfangen)
(j) TX (Senden)
(k) Bezugspotential (Gnd)
(l) RXD (Empfangen)
(m) TXD (Senden)

Technologieauswahl

Kleine Mikrocontroller wie der STM32F103RB (20 KB RAM, 128 KB Flash laut Datenblatt des Herstellers [14, S. 1]) haben nicht genug Ressourcen, um ein herkömmliches Betriebssystem (Linux, MacOS, Windows) zu verwenden.

Will man mehrere Funktionen parallel ausführen, muss jede Funktion so entworfen werden, dass sie die CPU nie dauerhaft blockiert, während andere Funktionen ebenfalls ausgeführt werden müssen. Funktionen müssen speziell dafür entworfen werden, was die Entwicklung deutlich schwieriger macht. Damit man Module so entwickeln kann, als gäbe es dieses Problem nicht, kann ein RTOS (siehe Abschnitt 2.4.1) verwendet werden. Die Erweiterbarkeit des Projekts und die Verständlichkeit des Codes werden dadurch erhöht.

Um die Programmierung von Mikrocontrollern ohne Betriebssystem zu erleichtern, gibt es die auf C basierende HAL-Bibliothek. Sie abstrahiert den direkten Registerzugriff in Funktionen. Da es sich bei der HAL-Bibliothek um C-Code handelt, liegt eine Verwendung der Hochsprache C für die Telemetrieeinheit nahe. C bietet allerdings weniger Möglichkeiten zur Codestrukturierung als C++. Um den Code in Klassen aufteilen zu können, wurde sich für die Verwendung von C++ statt reinem C entschieden. C-Funktionen können, sofern sie mit `extern "C"` deklariert werden, sowohl in C- als auch in C++-Code aufgerufen werden. C++-spezifische Konstrukte wie Klassen sind aus reinem C-Code nur mit einer Vermittlungsschicht nutzbar.

Programmentwurf

Um eine Telemetrieeinheit im System anzulegen, soll die global einzigartige Geräte-Identifikationsnummer des STM32 als Identifikator dienen. Eine Anlage im System soll möglich sein, wenn eine Telemetrieeinheit über USB mit dem Rechner verbunden ist, auf dem die Anwendung geöffnet ist. Dazu soll die Benutzeroberfläche einen Befehl über UART an alle USB-Schnittstellen senden (Broadcast). Eine verbundene Telemetrieeinheit antwortet auf den Befehl mit der Geräte-Identifikationsnummer, die bereits auf der Hardware auf eine 36 Zeichen lange UUID formatiert wird. Das kontinuierliche Warten auf diesen Befehl sowie die Abfrage und Sendung der fertigen UUID bei Erhalt ist somit eine unabhängige Aufgabe, die die Telemetrieeinheit durchführen muss.

Die Telemetrieeinheit soll sich beim Start mit dem Internet verbinden, eine MQTT-Verbindung aufbauen, den Erhalt valider GPS-Daten abwarten und dann jede Sekunde eine Wertabfrage an das LTE/GNSS-Modul sowie die IMU-Sensoreinheit durchführen und die erhaltenen Daten als Datenpunkt über MQTT senden. Ein Datenpunkt soll folgende Daten enthalten:

1. Geräte-Identifikationsnummer
2. Zeitstempel in UTC
3. Geschwindigkeit in km/h
4. Beschleunigung in m/s²
5. Längengrad
6. Breitengrad

Der abstrakte Ablauf wird als Zustandsautomat (siehe Abschnitt 2.5.1) modelliert. Die Telemetrieeinheit startet im Zustand **Idle** und wartet darauf, dass valide GPS-Daten erhalten werden und die Internetverbindung korrekt vorliegt. Ist das der Fall, wechselt der Automat in den Zustand **Publishing**, d. h. ein Datenpunkt soll erhoben und gesendet werden. Tritt dabei ein Fehler auf, geht der Automat in den Übergangszustand **Error**, danach wird wieder bei **Idle** gestartet. Wenn die Sendebereitschaft nicht mehr vorliegt, wird **Error** nicht betreten, da dies nicht zwangsläufig ein Fehler ist. In diesem Fall springt der Automat direkt nach **Idle**, um erneut zu prüfen, ob gesendet werden kann.

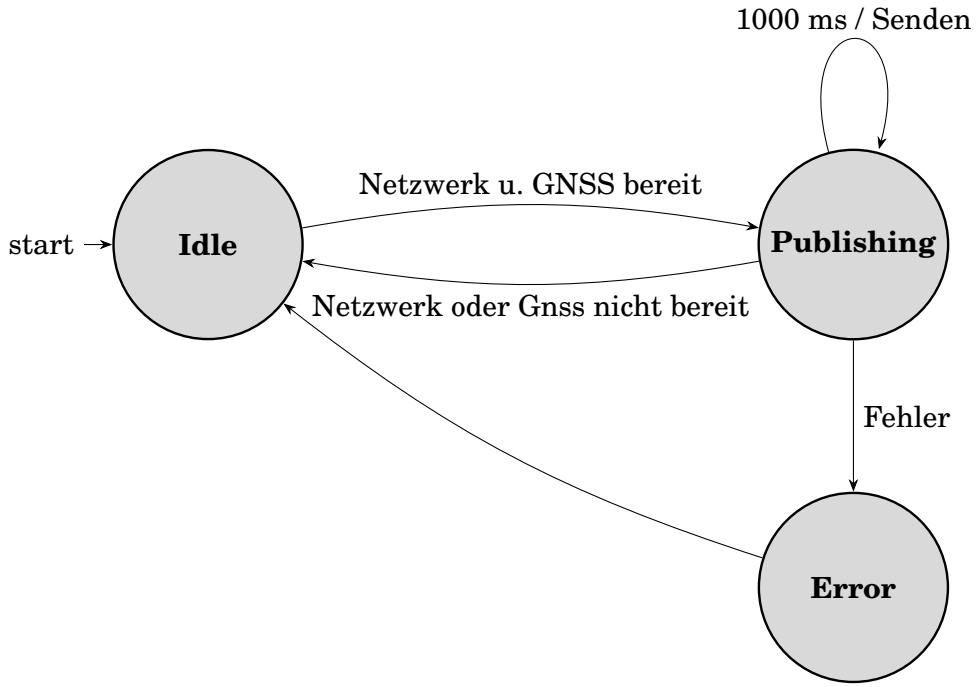


Abbildung 4.8: Zustandsautomat der Telemetry-FSM

Netzwerkverbindung und GNSS-Verbindung wurden als eigene, von einander unabhängige Zustandsautomaten modelliert. Der Zustandsübergang des allgemeinen

Telemetrieautomaten von **Idle** nach **Publishing** wird von Zuständen der beiden darunter liegenden Automaten abhängig gemacht.

Für den Zustandsautomaten der Netzwerkverbindung wurden die folgenden Zustände bestimmt:

1. **NetworkIdle:** Startzustand
2. **NetworkConnecting:** In diesen Zustand wird gewechselt, wenn eine Verbindungsaufbau eingeleitet wurde
3. **NetworkReady:** Falls der Verbindungsaufbau erfolgreich war, d. h. die Telemetrieeinheit mit dem Internet verbunden ist
4. **NetworkMqttConnected:** Beim Erreichen von *NetworkReady* wird die MQTT-Verbindung aufgebaut, bei Erfolg wird in diesen Zustand gewechselt.
5. **NetworkError:** Falls der Verbindungsaufbau nicht erfolgreich war, zählt dies als Fehler und eine Recoverysequenz soll versuchen, die Verbindung erneut sauber aufzubauen. Der Automat wechselt dann in den Zustand *NetworkIdle*.

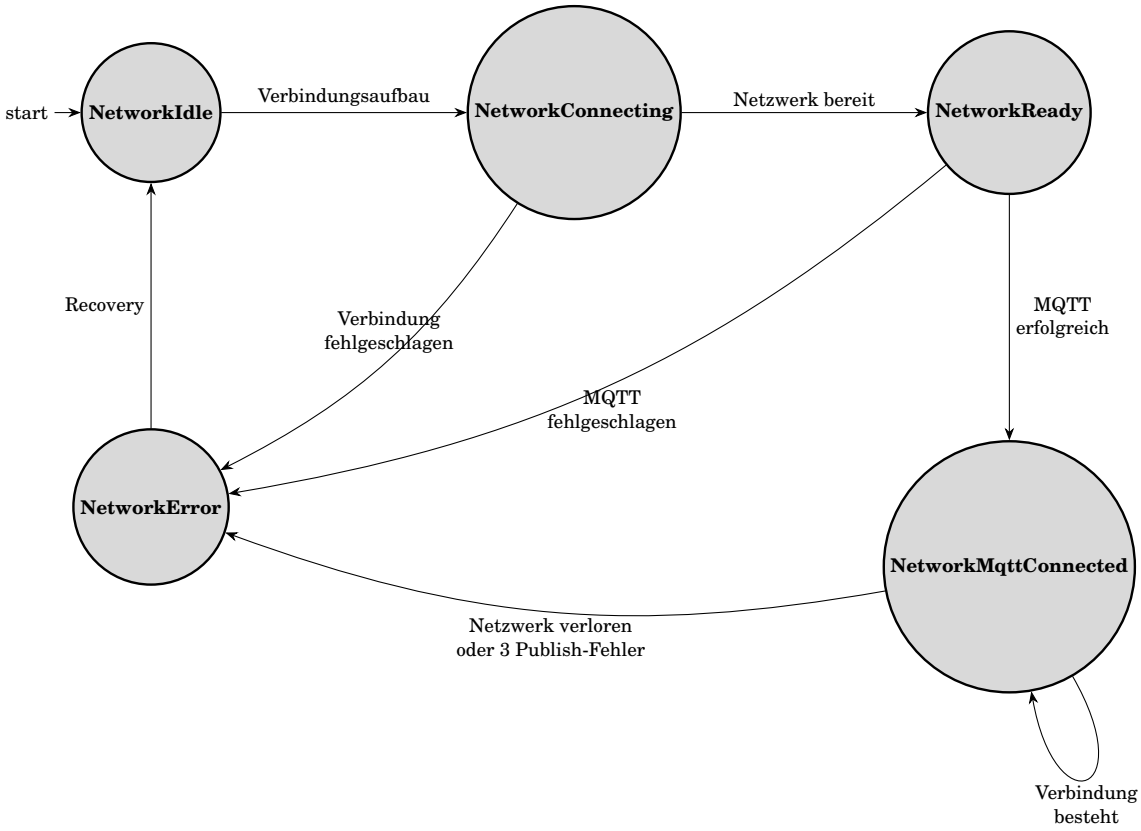


Abbildung 4.9: Zustandsautomat der Network-FSM

Für den GNSS-Automaten wurden ebenfalls die benötigten Zustände bestimmt.

1. **GnssIdle:** Startzustand des Automaten. Die GNSS-Aktivierungssequenz wird eingeleitet.
2. **GnssEnabling:** Nach der Aktivierung des GNSS wird in diesen Zustand gewechselt. Der Empfang der NMEA-Daten wird gestartet und anschließend in den Zustand *GnssReady* gewechselt.
3. **GnssReady:** Das GNSS ist aktiviert und es wird regelmäßig auf einen gültigen GNSS-Fix geprüft. Liegt nach 60 Sekunden weiterhin kein gültiger Fix vor, wird von einem Fehler ausgegangen und in den Zustand *GnssError* gewechselt.
4. **GnssError:** Eine Recovery-Sequenz wird eingeleitet, bei der das GNSS erneut aktiviert und der NMEA-Datenempfang gestartet wird. Anschließend wird wieder in den Zustand *GnssReady* gewechselt.

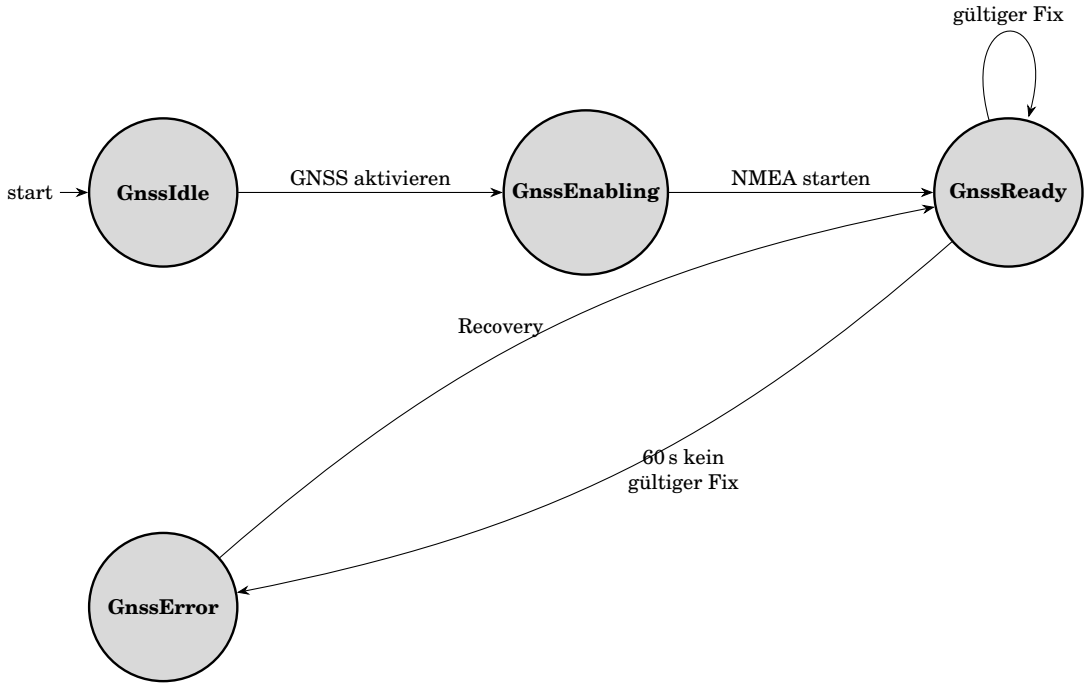


Abbildung 4.10: Zustandsautomat der GNSS-FSM

Neben den Zuständen benötigt jeder Zustandsautomat eine Funktion, die den nächsten Schritt berechnet und eine Möglichkeit, den aktuellen Zustand abzufragen. Der übergeordnete Systemautomat muss für den Wechsel in den Sendezustand *Publishing* die Zustände der untergeordneten Automaten für Netzwerkverbindung und GNSS-Modul abfragen können.

Somit besteht der Programmablauf aus drei RTOS-Tasks, die nebenläufig ausgeführt

werden:

1. Befehlsempfang / Geräte-ID-Ausgabe
2. Automatenbetrieb, Hauptaufgabe
3. Default-Task, kann für taskübergreifende Aufgaben und Koordination der Tasks genutzt werden

Modellierung auf Klassenebene

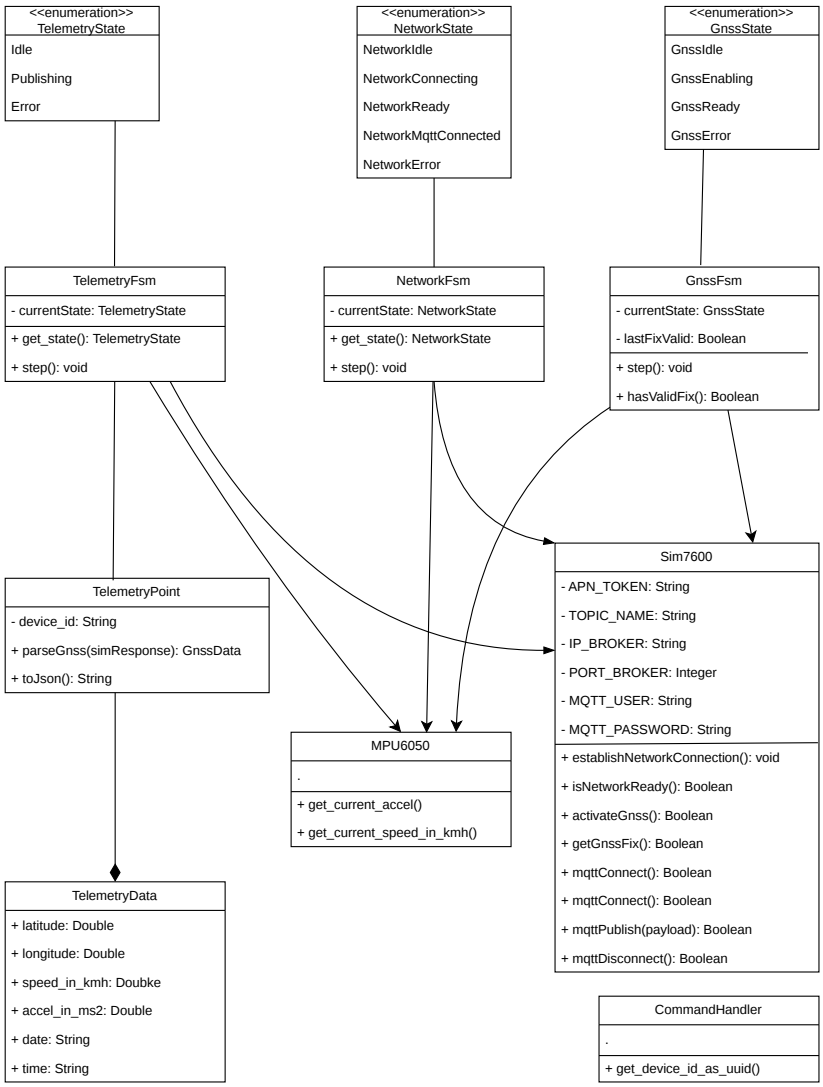


Abbildung 4.11: Klassendiagramm der Entwurfsebene von *TelemetryUnit*

Alle drei Automaten sollen eine Klasse darstellen, wobei die Zustände eigene Dateien bekommen sollen. Jeder Automat nutzt eine Klasse, die den Hardwarezugriff auf

das LTE/GNSS-Modul kapselt (`Sim7600`) und eine Klasse, die den Hardwarezugriff auf die Sensoreinheit kapselt (`MPU6050`). Auf `MPU6050` wurde letztlich verzichtet (siehe Abschnitt 5.1.5), sie soll hier trotzdem erwähnt werden.

`Sim7600` hält alle benötigten Verbindungsdaten und veröffentlicht Funktionen, die die Automaten dann nutzen können, ohne selbst Kenntnis von der Hardware haben zu müssen.

Der übergeordnete Automat `TelemetryFsm` verwendet eine Klasse `TelemetryPoint`, die für den Zusammenbau eines fertigen Datenpunkts zuständig ist. Diese soll wiederum die gerade verwendeten Daten in einer Datenklasse `TelemetryData` halten.

Für die Ausgabe der Geräte-ID soll es ebenfalls eine eigene Klasse geben (`CommandHandler`); auf deren Umsetzung wurde jedoch letztlich verzichtet (siehe Abschnitt 5.0.1).

4.3.2 IngestionService

Dieser Dienst nimmt Telemetriedaten von Fahrzeugen über MQTT entgegen, ordnet sie einem Fahrzeug bzw. einem Fahrer, erkennt Fahrtbeginn und -ende automatisch und persistiert die Daten in einer InfluxDB-Zeitseriendatenbank. Die Kommunikation mit anderen Diensten erfolgt über gRPC.

Technologieauswahl

Als Implementierungsplattform des Dienstes wurde **.NET** eingesetzt. **ASP.NET Core** enthält eine integrierte Unterstützung für gRPC (siehe Abschnitt 2.3.1) und Protocol Buffers. Ein Projekt kann direkt als ASP.NET-gRPC-Projekt angelegt werden, was die Entwicklung beschleunigt.

Programmentwurf

Der Aufgabenbereich von *IngestionService* umfasst sechs Unterbereiche:

1. **MQTT-Eingang:** Der Dienst verbindet sich beim Start mit dem MQTT-Broker, abonniert die konfigurierte MQTT-Topic (Bezeichner für Nachrichtenkanal) und nimmt eingehende Nachrichten der Telemetrie-Einheiten entgegen. Bei Verbindungsverlust wird automatisch versucht, die Verbindung wiederherzustellen und die MQTT-Topic erneut zu abonnieren. Hier beginnt die Bearbeitungskette von *IngestionService*.

2. **Nachrichtenverarbeitung:** Jeder eingehende JSON-Datenpunkt muss eingelesen und einer im System angelegten Telemetrieeinheit zugeordnet werden.

3. **Zustandsautomat für laufende Fahrten:** Ein Zustandsautomat muss für jedes aktive Gerät (Fahrt begonnen, aber noch nicht beendet) den Fahrtzustand festhalten und in Abhängigkeit davon Fahrtanlage und Fahrtende beim Dienst *FleetControlApi* anordnen. Dazu muss es ebenfalls eine Retry-Komponente geben, die bei erfolgloser Anfrage wiederholt versucht, *FleetControlApi* zu erreichen, um die Konsistenz der Datenbasis auch über Verbindungsabbrüche hinweg wahren zu können.

4. **Synchronisation der Zuordnung von Fahrer, Fahrzeug und Telemetrieeinheit mit *FleetControlApi*:** Da Fahrtdaten für ein Fahrzeug und einen Fahrer erhoben werden, muss *IngestionService* jederzeit wissen, welche Verbindungen aktuell zwischen Fahrer, Fahrzeug und Telemetrieeinheit bestehen. Dazu muss der Dienst einen Zwischenspeicher über die Zuordnungen führen, der beim Hochfahren im System hinterlegte Zuordnungen von *FleetControlApi* abfragen muss. Der Zwischenspeicher muss während des Betriebs aktuell gehalten werden.

5. **Persistenz:** Effiziente Speicherung der Telemetriedaten in der Influx-Datenbank.

6. **Benachrichtigungen:** Damit später die Karten der Benutzeroberfläche sich ohne Zutun des Nutzers aktualisieren können, muss *IngestionService* selbst Benachrichtigungen bei eingehenden Positionsänderungen von Fahrzeugen senden. Benachrichtigungen werden an den verwendeten Eventbus *RabbitMQ* übergeben. Die Infrastruktur für die Benachrichtigungen wurde, wie bereits am Anfang des Lösungskapitels erwähnt, ursprünglich nicht vorab eingeplant, der Bereich soll aber trotzdem schon aus Gründen der Vollständigkeit hier erwähnt werden.

Zustandsautomat für laufende Fahrten Für jedes bekannte Gerät, von dem ein Datenpunkt eintrifft, wird ein eigener Zustandsautomat geführt, der den Fahrtzustand des zugehörigen Fahrzeugs anhand eintreffender Geschwindigkeitswerte nachvollzieht. Ein Gerät, für das gerade kein Zustand geführt wird (keine Fahrt am Laufen), gilt implizit als *Idle*. Der Automat unterscheidet vier geführte Zustände:

1. **Verbunden:** Das Gerät sendet Daten, es liegt jedoch keine laufende Fahrt vor.

2. **Fährt:** Es wurde eine laufende Fahrt erkannt.

3. **Steht:** Das Fahrzeug befindet sich innerhalb einer laufenden Fahrt im Stillstand.

4. **Verbindung verloren:** Für das Gerät sind während einer laufenden Fahrt

für eine gewisse Zeit keine neuen Daten eingetroffen.

Erkennung des Fahrtbeginns Ein Wechsel von *Verbunden* nach *Fährt* erfolgt erst, wenn mehrere aufeinanderfolgende Datenpunkte oberhalb eines Geschwindigkeits-Schwellwerts liegen. Damit soll verhindert werden, dass ein einzelner Ausreißerwert fälschlich eine Fahrt auslöst. Die Fahrtanlage wird in *FleetControlApi* ausgelöst.

Erkennung des Fahrtendes Sobald während *Fährt* ein Datenpunkt mit Geschwindigkeit null eintrifft, wechselt der Automat nach *Steht*. Hält der Stillstand über eine gewisse Zeitspanne, gilt die Fahrt als beendet. Das Fahrtende wird dabei an *FleetControlApi* gemeldet, der Automat kehrt in den Zustand *Verbunden* zurück und eine erneute Bewegung würde einen neuen Fahrtstart auslösen.

Umgang mit Verbindungsabbrüchen Bleiben für ein Gerät während einer laufenden Fahrt für kurze Zeit neue Datenpunkte aus, wechselt der Automat in den Zustand *Verbindung verloren*, ohne die Fahrt zu beenden. Bleibt die Verbindung über einen längeren Zeitraum unterbrochen, soll die Fahrt beendet werden.

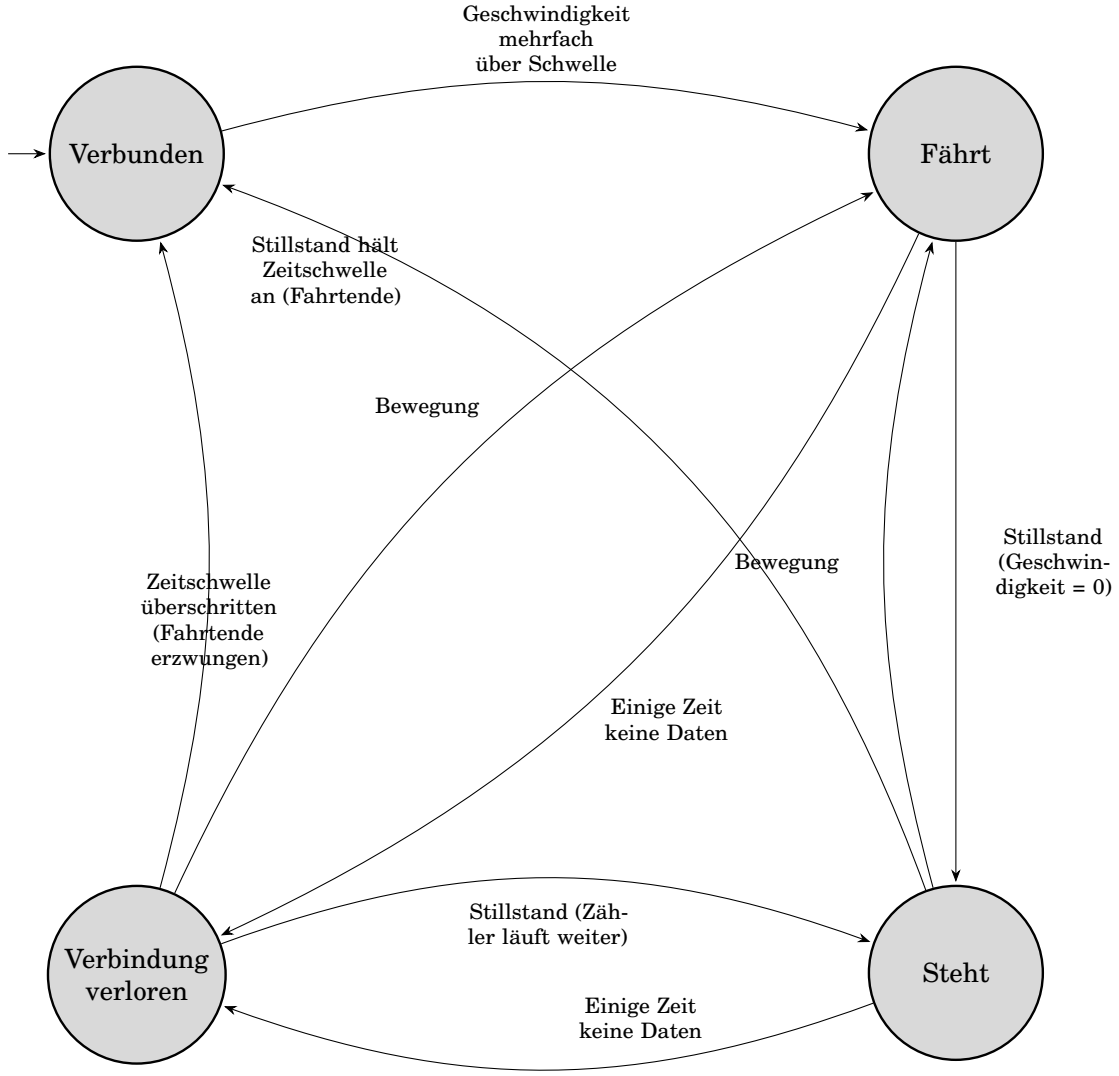


Abbildung 4.12: Zustandsautomat für laufende Fahrten

Synchronisation mit FleetControlApi Die Meldung von Fahrtbeginn und -ende an *FleetControlApi* erfolgt über eine Netzwerkverbindung und kann daher theoretisch fehlschlagen, falls der Dienst nicht erreichbar ist. Anordnungen für Fahrtbeginn und -ende sollen vorgemerkt werden, und in regelmäßigen Abständen soll eine Zustellung erneut probiert werden, bis dies erfolgreich war. Dadurch bleibt die Datenbasis auch über temporäre Verbindungsabbrüche zwischen den Diensten hinweg konsistent.

Persistenz: Eingehende, verarbeitete Telemetriedaten müssen dauerhaft in der Influx-Datenbank gespeichert werden. Damit die Schreiblatenz der Datenbank

die Verarbeitung bei vielen eingehenden MQTT-Nachrichten nicht verzögert, soll das Schreiben nicht direkt im Verarbeitungspfad erfolgen. Datenpunkte werden zunächst zwischengespeichert und unabhängig davon, entkoppelt von der übrigen Verarbeitungskette, gesammelt in die Datenbank geschrieben.

Benachrichtigungen: Damit sich die Karten der Benutzeroberfläche ohne Zutun des Nutzers aktualisieren können, muss *IngestionService* bei jeder gespeicherten Positionsänderung eine Benachrichtigung an den verwendeten Eventbus *RabbitMQ* übergeben, der erst später während der Implementierung wirklich eingeplant wurde (siehe Abschnitt 5.0.4).

gRPC-Schnittstelle: *IngestionService* kommuniziert mit *FleetControlApi* über gRPC in beide Richtungen (siehe Abschnitt 2.3.1). Aus den beschriebenen Anforderungen ergeben sich vier Endpunkte:

1. **Fahrtanlage auslösen** (Client) – meldet die vom Zustandsautomaten erkannte Fahrtanlage. Übertragen werden sollen Startzeitpunkt und Geräte-ID.
2. **Fahrtende auslösen** (Client) – meldet das vom Zustandsautomaten erkannte Fahrtende. Übertragen werden sollen Endzeitpunkt und Geräte-ID.
3. **Aktuelle Zuordnungen abfragen** (Client) – lädt beim Start die im System hinterlegten Zuordnungen von Geräte-ID, Fahrer, Kennzeichen und Fahrzeug
4. **Zuordnungsänderungen entgegennehmen** (Server) – nimmt Änderungen entgegen, sobald sie während des Betriebs auftreten, statt selbst regelmäßig nachzufragen.

Klassenaufteilung

Der Aufbau von *IngestionService* wird durch die folgenden Klassendiagramme konkretisiert.

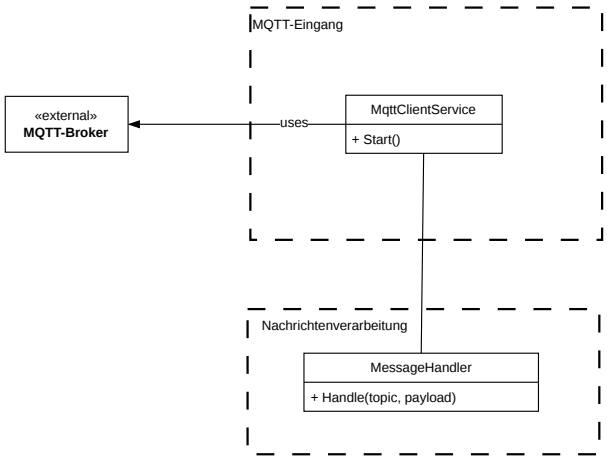


Abbildung 4.13: Klassendiagramm für MQTT-Eingang und Nachrichtenverarbeitung

Abbildung 4.13 zeigt die geplanten Klassen des MQTT-Eingangs und der Nachrichtenverarbeitung. Der `MqttClientService` startet bei Dienststart die MQTT-Verbindung und empfängt eingehende Telemetriedaten vom MQTT-Broker, die dann an den `MessageHandler` weitergegeben werden. Diese Klasse soll für die weitere Verarbeitung der eingehenden Nachrichten zuständig sein.

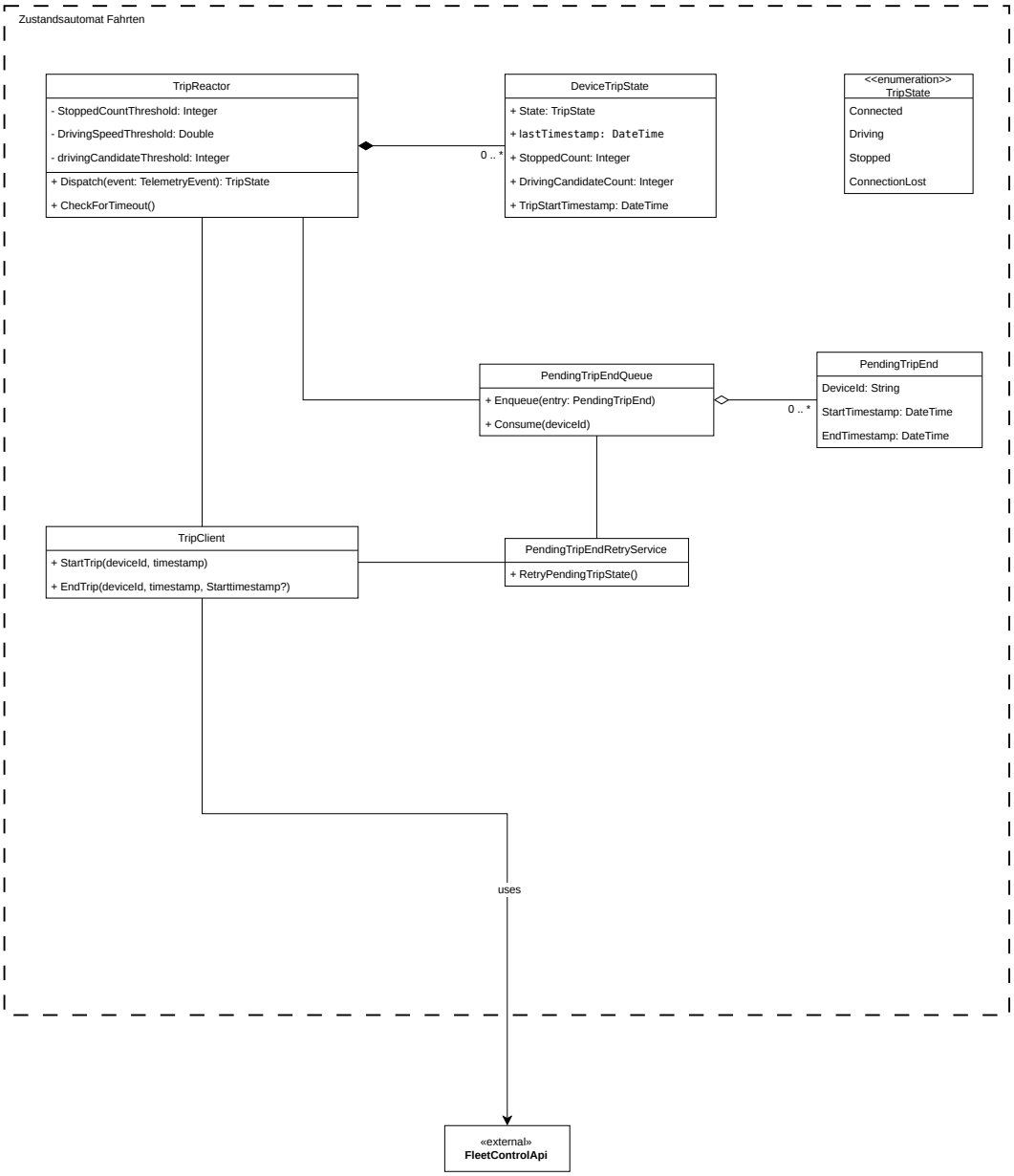


Abbildung 4.14: Klassendiagramm des Zustandsautomatenbereichs

Abbildung 4.14 zeigt die geplanten Klassen für den Fahrtzustandsautomatenbereich. `TripReactor` verwaltet für jedes Gerät mit laufender Fahrt eine eigene `DeviceTripState`-Instanz und meldet erkannte Fahrtanlagen bzw. -beendigungen über `TripClient` per gRPC an *FleetControlApi*. Schlägt eine Fahrtende-Meldung fehl, wird sie in der `PendingTripEndQueue` vorgemerkt und vom

`PendingTripEndRetryService` periodisch erneut versucht, zuzustellen.

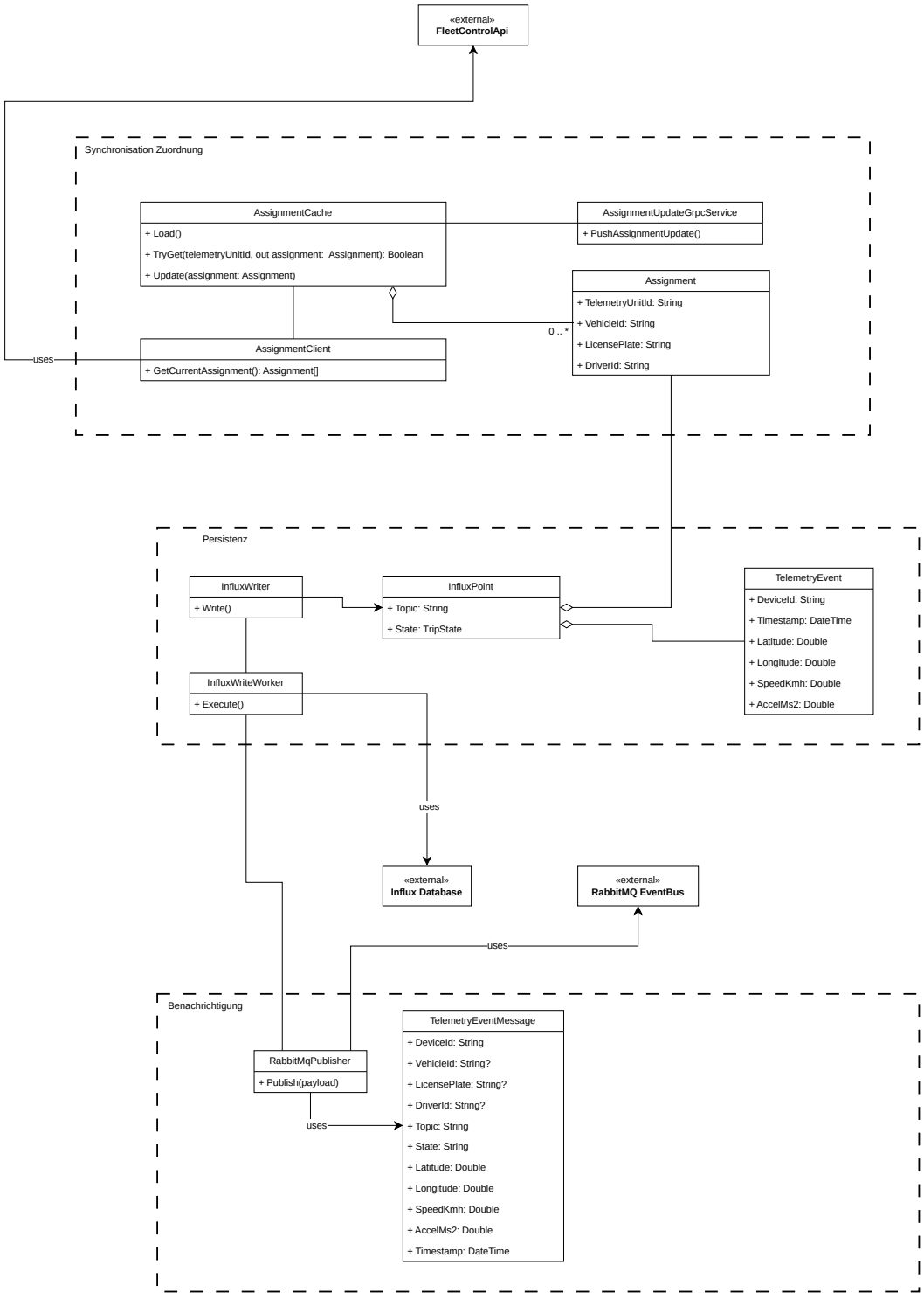


Abbildung 4.15: Klassendiagramm für Zuordnungssynchronisations-, Persistenz- und Benachrichtigungsbereich

Abbildung 4.15 fasst die Klassen der Zuordnungssynchronisation, der Persistenz und der Benachrichtigungen zusammen. Der `AssignmentCache` hält die aktuellen Zuordnungen zwischen Telemetrie-Einheit, Fahrzeug und Fahrer im Speicher, lädt sie beim Start über den `AssignmentClient` von *FleetControlApi* und wird über den `AssignmentUpdateGrpcService` laufend aktualisiert. Verarbeitete Datenpunkte werden vom `InfluxWriter` entkoppelt über eine Warteschlange an den `InfluxWriteWorker` übergeben, der sie in einer Influx-Datenbank speichert und anschließend als `TelemetryEventMessage` über den `RabbitMqPublisher` an den RabbitMQ-Eventbus übergibt.

4.3.3 DriveDataService

Der Dienst ist für die Abfrage von Datenpunkten aus *InfluxDB* zuständig.

Aufbau

DriveDataService muss über gRPC-Schnittstelle folgende Abfragen auf den in InfluxDB gespeicherten Daten ermöglichen:

- Abruf aller Datenpunkte einer Telemetrie-Einheit innerhalb eines angegebenen Zeitraums, entspricht Daten für eine Fahrt.
- Abruf des jeweils letzten bekannten Datenpunkts für eine Menge von Telemetrie-Einheiten oder einer Menge von Fahrzeugen für die Darstellung der aktuellen bzw. letzten Positionen in einer Live-Kartenansicht.
- Löschen aller Datenpunkte einer Telemetrie-Einheit innerhalb eines angegebenen Zeitraums, damit die Datenpunkte einer Fahrt gelöscht werden können.

Die technische Umsetzung ist in Abbildung 4.16 dargestellt.



Abbildung 4.16: Klassendiagramm der Entwurfsebene von *DriveDataService*

Abbildung 4.16 zeigt die einzige Klasse `TelemetryQueryService`, sie kapselt die vier benötigten Abfragen (`GetTelemetryPoints`, `GetLatestPoints`, `GetLatestPointsByVehicle`, `DeleteTelemetryPoints`) und macht sie für *FleetControlApi* verfügbar, zurückgegeben werden Ergebnisse als `TelemetryPoint`. Die Klasse soll zudem eine dauerhafte Verbindung zur Influx-Datenbank halten.

4.3.4 FleetControlApi

FleetControlApi stellt eine zentrale Schnittstelle für die browserbasierte Benutzeroberfläche bereit.

Technologieauswahl

Für die Implementierung der REST-API wird **ASP.NET Core** aus dem .NET-Umfeld eingesetzt. Das Framework bietet einen eigenen Projekttyp für Komponenten mit HTTP-basierten APIs, was den Entwicklungsaufwand reduziert.

Persistenz

Die persistente Speicherung der relationalen Stammdaten (Fahrer, Fahrzeuge, Fahrten, Telemetrieeinheiten) wurde, wie bereits zu Beginn des Lösungskapitels erwähnt, mit *PostgreSQL* umgesetzt.

Für den Datenzugriff soll ein ORM, genauer *Entity Framework Core*, eingesetzt werden(siehe Abschnitt 2.6.2), damit die Implementierung unabhängig vom SQL-Dialekt der Datenbanktechnologie bleibt. So kann das Datenbankschema direkt aus den Entitätsklassen der Domain-Schicht abgeleitet werden und Änderungen am Datenmodell können nachvollzogen werden.

Datenmodell

Aus den Anforderungen ergibt sich das Datenmodell für *FleetControlApi*. Benötigt werden die Entitäten **TelemetryUnits** (Telemetrieeinheiten), **Vehicles** (Fahrzeuge), **VehicleDrivers** (Fahrer), **Trips** (Fahrten) und **DriversLicenses** (Führerscheine). Die Führerscheine benötigen eine eigene Tabelle, da ein Fahrer mehrere besitzen kann.

- **Vehicles:** Als Primärschlüssel wird die global einzigartige, 17 Zeichen lange FIN verwendet, die seit 1981 nach internationaler Norm für die meisten Pkw und Nutzfahrzeuge standardisiert ist. Es wird von modernen Fahrzeugen im Fuhrpark ausgegangen.
  Das Kennzeichenfeld ist auf maximal 9 Zeichen begrenzt, da ein normales deutsches Kennzeichen maximal 8 Zeichen ohne das Trennzeichen haben darf.
  Die Textfelder wurden auf 50 Zeichen begrenzt, um übermäßig lange Eingaben zu verhindern und somit Datenqualität sicherzustellen und Speicherverschwendung zu verhindern.
  Bei der Fahrerlaubnis ist eine Begrenzung nicht nötig, da der Typ später durch einen *Enum*-Typen realisiert wird und nur vordefinierte Werte annehmen kann.
  Der optionale Fremdschlüssel zeigt auf einen Fahrer.

- **VehicleDrivers:** Als Primärschlüssel wird eine 36 (mit Trennzeichen) Zeichen lange UUID verwendet, die mit hoher Sicherheit einzigartig ist. Eine UUIDv4 besitzt 122 zufällige Bits, wodurch etwa $2^{122} \approx 5{,}3 \cdot 10^{36}$ mögliche Werte existieren. Die Wahrscheinlichkeit einer Kollision ist daher für den vorgesehenen Anwendungsfall vernachlässigbar gering.

- **TelemetryUnits:** Der Primärschlüssel ist ebenfalls eine UUID, die allerdings garantiert einzigartig ist, da sie aus der global einzigartigen, vom Hersteller vergebenen Identifikationsnummer auf der Hardware erzeugt und bei Neuan-

lage an das System übergeben wird.
Der Fremdschlüssel ist die 17 Zeichen lange FIN eines optional verbundenen Fahrzeugs.

- **DriversLicenses:** Aus bereits genannten Gründen kommt eine UUID als Primärschlüssel zum Einsatz, der Fremdschlüssel zeigt auf den Fahrer, dem ein Führerschein zugeordnet ist. Das Datum des Führerscheinerwerbs wird ebenfalls gespeichert.
- **Trips:** Eine Fahrt ist immer mit genau einer Telemetrieeinheit verbunden. Die Verknüpfungen mit Fahrzeug und Fahrer dürfen nicht über Fremdschlüssel dargestellt werden, da eine Fahrt eine Momentaufnahme ist und sich diese Zuweisungen ändern können. Beginn und Ende der Fahrt werden als Zeitstempel gespeichert und bieten eine Möglichkeit, die zugehörigen Datenpunkte zuzuordnen.

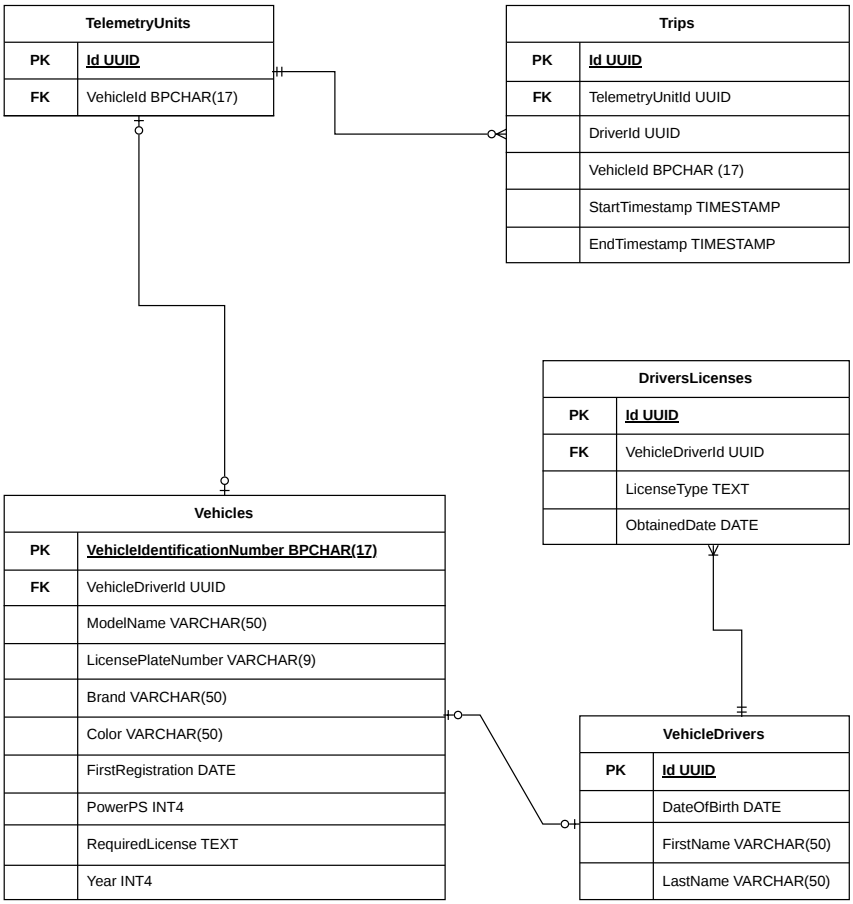


Abbildung 4.17: Entity-Relationship-Diagramm von *FleetControlApi*

Modellierung auf Klassenebene

FleetControlApi lässt sich in vier Schichten auf unterschiedlichen Abstraktionsebenen teilen. Der Dienst wurde als Schichtenarchitektur (siehe Abschnitt 2.1.4) modelliert, um Erweiterbarkeit, Testbarkeit und Lesbarkeit zu verbessern.

1. **Api:** Ist der Einstiegspunkt des Dienstes. Stellt die REST-Schnittstelle für die Benutzeroberfläche bereit, implementiert die benötigten gRPC-Endpunkte, für die Kommunikation mit *IngestionService*, sowie ein SignalR-Hub für Echtzeit-Aktualisierungen der browserbasierten Benutzeroberfläche (ursprünglich nicht mitgeplant; die Notwendigkeit stellte sich bei Implementierung/Tests heraus, siehe Abschnitt 5.0.4).
2. **Service:** Enthält die eigentliche Logik des Diensts. Zu jeder Entität gibt es eine zugehörige Service-Klasse.
3. **Data:** Übernimmt die persistente Speicherung mittels Entity Framework Core in einer SQL-Datenbank.
4. **Domain:** Enthält ausschließlich die reinen Entitätsklassen (`Vehicle`, `VehicleDriver`, `TelemetryUnit`, `Trip`, `DriversLicense`) sowie zugehörige Enums, ohne Abhängigkeit zu einer anderen Schicht.

API-Schicht

Die API-Schicht kapselt die Einstiegspunkte von *FleetControlApi* nach außen. Sie enthält drei Kommunikationswege: REST (Browser/UI), gRPC sowie RabbitMQ/SignalR (Ereignisse für Live-Updates an den Browser senden).

Die REST-Schnittstelle wird durch die Klassen `VehicleController`, `PersonController`, `TripController`, `TelemetryUnitController` in Arbeitsablaufsbereiche geteilt, um Leserlichkeit und Modularität zu gewährleisten. Die Controller-Klassen enthalten nach dem Prinzip der Schichtenarchitektur selbst keine Logik und verwenden Instanzen der Service-Schicht, die die Geschäftsregeln enthält.

Die API-Schicht implementiert drei gRPC-Endpunkte:

1. `StartTrip()` (Server) – Fahrtanlage mit Geräte-ID und Zeitstempel anordnen
2. `EndTrip()` (Server) – Fahrtabschluss mit Geräte-ID und Zeitstempel anordnen. Kann optional auch einen Fahrtstartzeitpunkt enthalten, falls zum Zeitpunkt des Fahrtendes *FleetControlApi* noch nicht erfolgreich zur Fahrtanlage erreicht wurde.

3. `GetCurrentAssignments()` (Server) – Ausgabe der aktuellen Zuordnungen von Telemetrieeinheit, Fahrer, Fahrzeug und Kennzeichen.

Das folgende Klassendiagramm wurde für die Lösung entworfen.

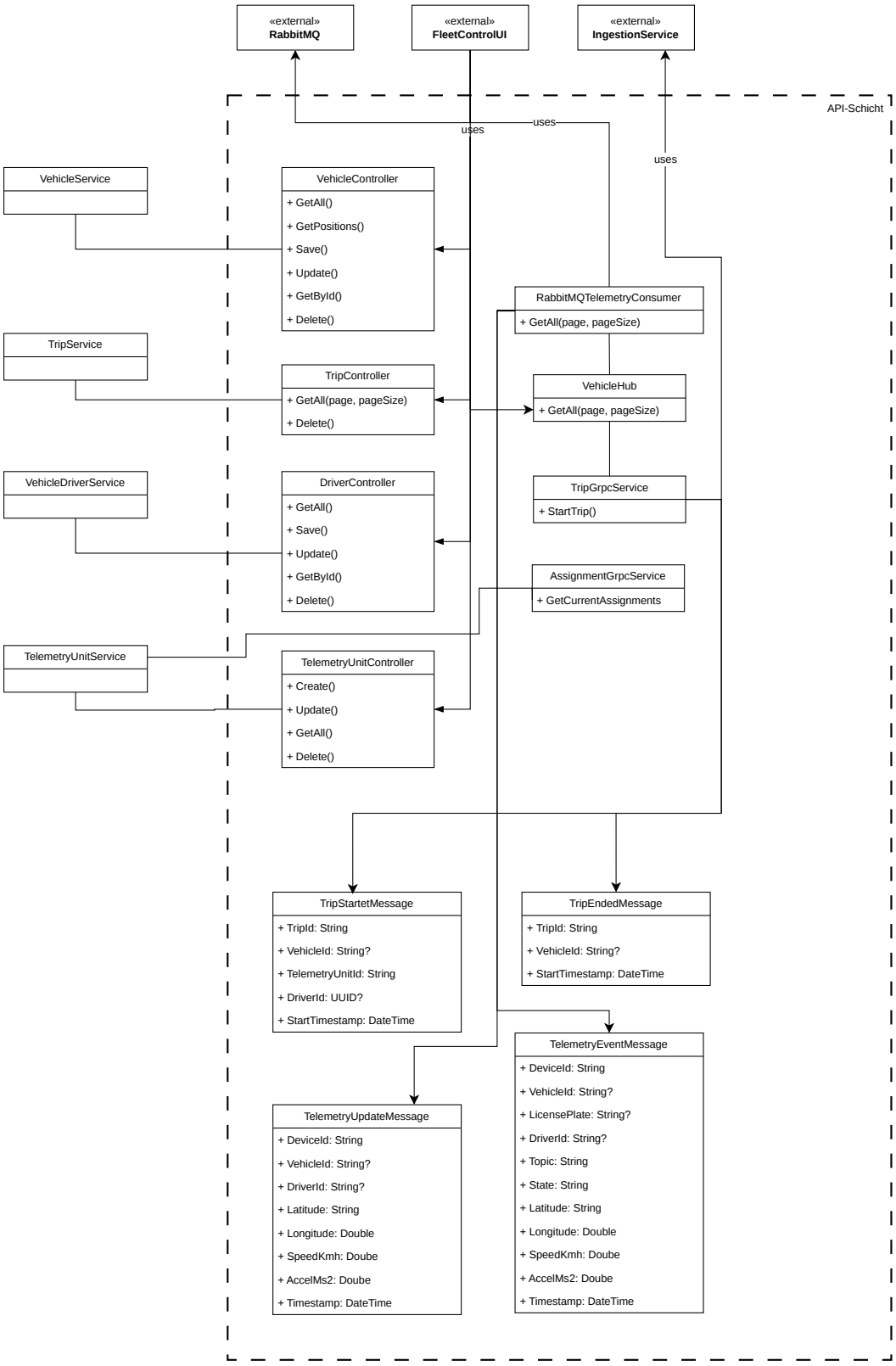


Abbildung 4.18: Klassendiagramm der *API*-Schicht von *FleetControlApi*

Jede Controller-Klasse hält eine Referenz zur zugehörigen Service-Klasse. `TripGrpcService` und `AssignmentGrpcService` sollen die gRPC-Endpunkte bzw. den aus den Protodateien später generierten Code kapseln.

`RabbitMqTelemetryConsumer` ist zuständig für den Empfang der Ereignisse bzw. der Datenpunkte, die über RabbitMQ von *IngestionService* für die automatischen Kartenaktualisierungen gesendet werden.

`VehicleHub` kapselt den SignalR-Broadcast-Kanal um Kartenaktualisierungen an allen im Browser geöffneten *FleetControlUI*-Instanzen anzustoßen.

Service-Schicht

Die *Service*-Schicht enthält die Geschäftslogik von *FleetControlApi*. Die Service-Klassen `VehicleService`, `PersonService`, `TripService`, `TelemetryUnitService` führen mit Hilfe der Repository-Klassen der Data-Schicht die CRUD-Operationen und die Validierung von entitätsübergreifender Konsistenz durch.

Die Klasse `AssignmentPushService` ist für die Mitteilung von Änderungen des Systemzustands bzw. der Verbindungen zwischen den Entitäten an *IngestionService* zuständig. Der ausgehende gRPC-Endpunkt wird in einer Klasse `IngestionServiceClient` gekapselt.

Um die in InfluxDB gespeicherten Telemetriedaten abfragen und an die Benutzeroberfläche weitergeben zu können, kapselt `TelemetryQueryClient` die gRPC-Abfrage der Daten bei *DriveDataService*.

Dazu werden der in der Datenbank gespeicherte Anfangs- bzw. Endpunkt einer Fahrt über `TripService` abgefragt, um die zu einer Fahrt gehörenden Datenpunkte zu erhalten.

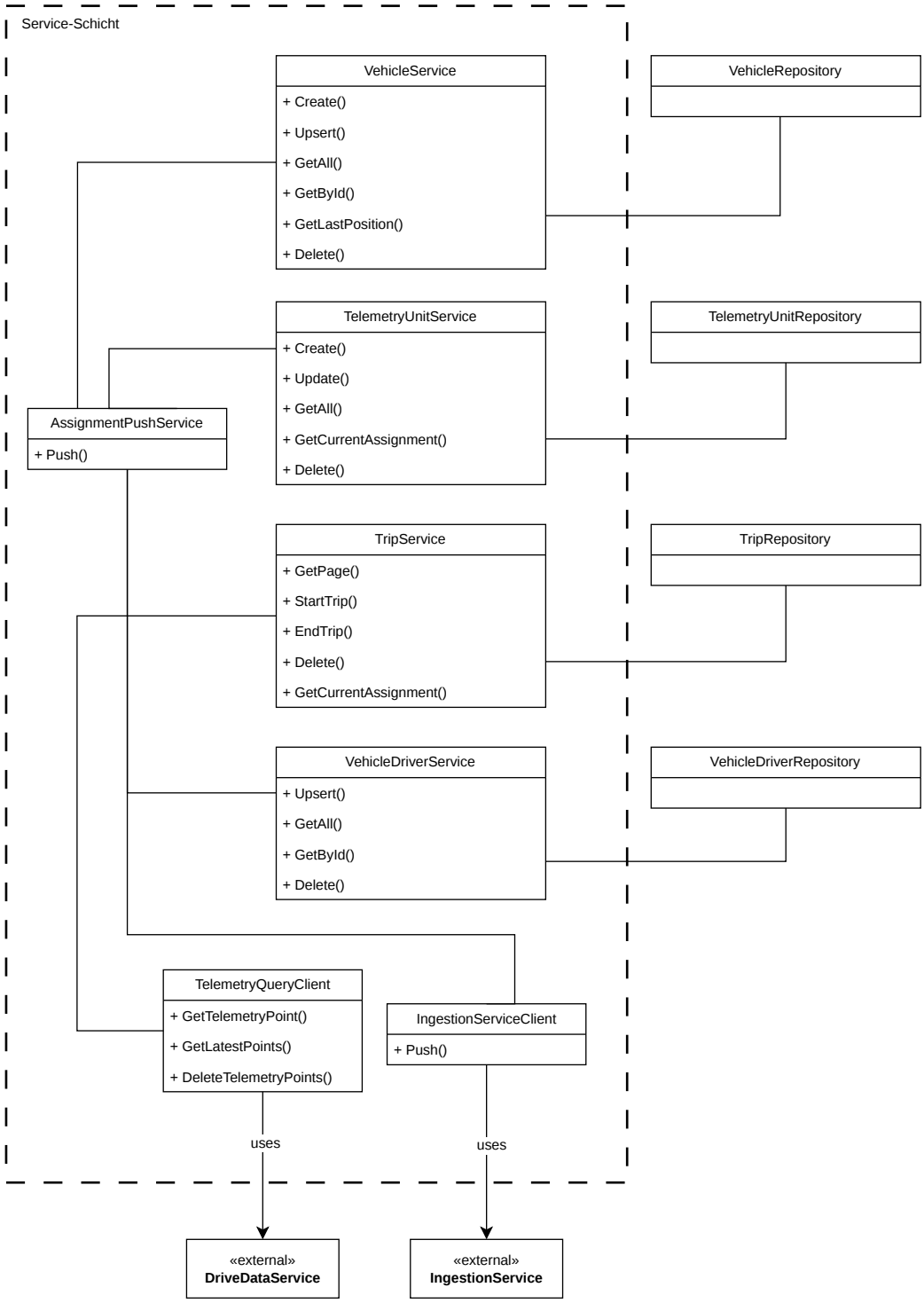


Abbildung 4.19: Klassendiagramm der *Service*-Schicht von *FleetControlApi*

Data-Schicht

Die *Data*-Schicht übernimmt die Speicherung der Stammdaten in der relationalen Datenbank. Für jeden Aufgabenbereich gibt es eine Repository-Klasse, die den direkten Datenbankzugriff kapselt. Die Repository-Klassen verwenden die Entitäten der Domain-Schicht. Um die Abfrage von Entitätszuordnungen zu realisieren gibt es mit der Klasse `TelemetryAssignment` ein entitätsunabhängiges Datenmodell.

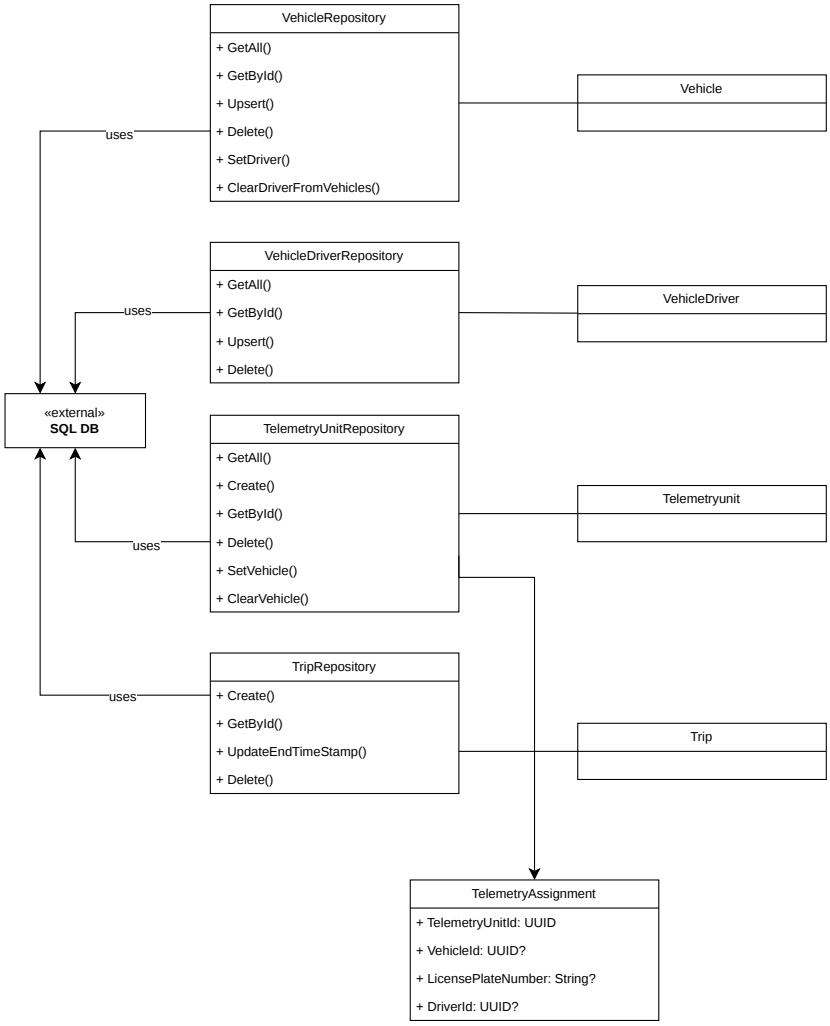


Abbildung 4.20: Klassendiagramm der *Data*-Schicht von *FleetControlApi*

Domain-Schicht

Die *Domain*-Schicht ist die tiefste Schicht der Architektur. Sie enthält die Entitätsklassen (`Vehicle`, `VehicleDriver`, `TelemetryUnit`, `Trip`, `DriversLicense`) sowie den zugehörigen Enum-Typ `DriversLicenseType`. Die Schicht bleibt durch ihre fehlende Abhängigkeit von der Data-Schicht unabhängig von der verwendeten Datenbanktechnologie. Will man diese später austauschen, muss der in der Domain-Schicht befindliche Code nicht verändert werden, was Erweiterbarkeit und Änderbarkeit der Anwendung in Zukunft begünstigt.

Aufbau und Begründung der einzelnen Attribute (Schlüsselwahl, Feldlängen, Beziehungen) wurden bereits im Datenmodell (Abschnitt 4.3.4 bzw. ER-Diagramm) beschrieben und werden hier nicht wiederholt.

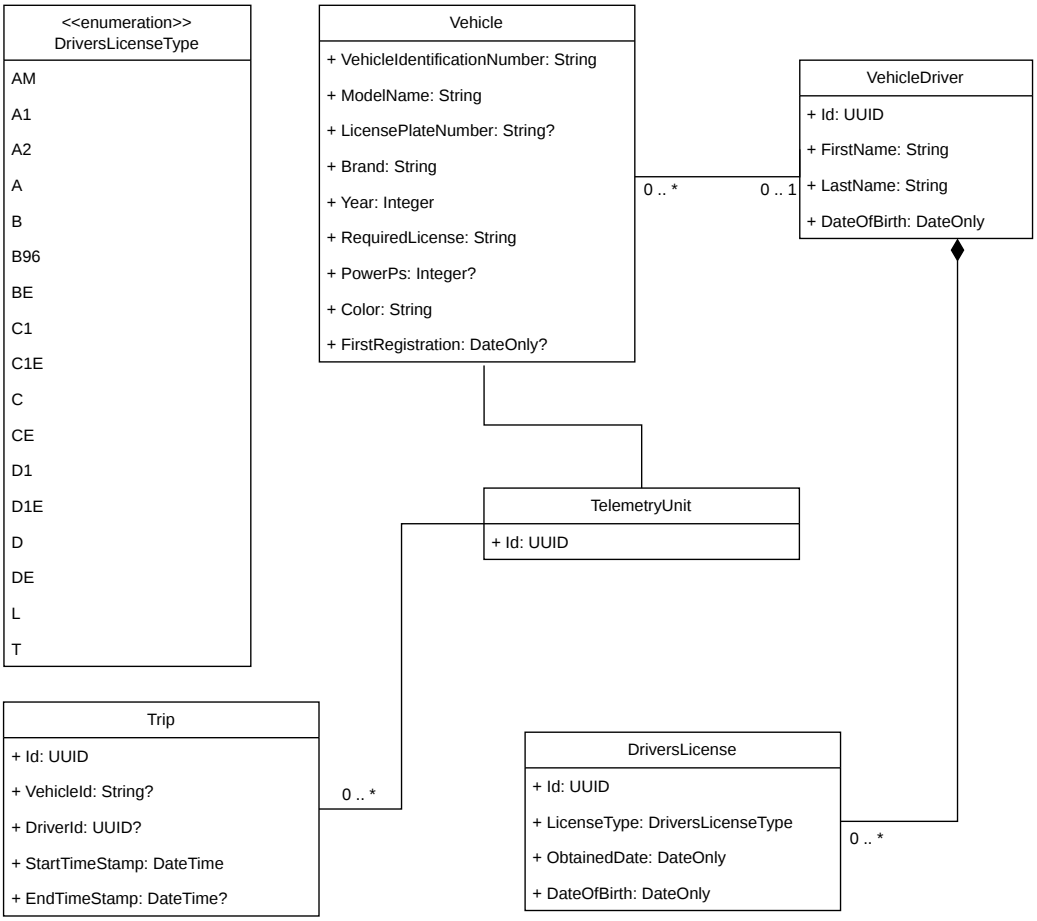


Abbildung 4.21: Klassendiagramm der *Domain*-Schicht von *FleetControlApi*

4.3.5 FleetControlUI

Die Benutzeroberfläche macht das System bedienbar. Sie soll die systemseitige Anlage und Verknüpfung von Telemetrieeinheiten, Fahrzeugen und Fahrern ermöglichen sowie die Darstellung der letzten bzw. aktuellen Fahrzeugpositionen auf einer Karte. Angefangene und abgeschlossene Fahrten sollen ebenfalls auf einer Karte dargestellt und angesehen werden können. Aus den übertragenen Datenpunkten soll eine Fahrtroute gezeichnet werden. Man soll zu jedem Datenpunkt die Fahrtdaten zu dem zugehörigen Zeitpunkt ansehen können. Außerdem soll eine Fahrt auch als Graph dargestellt werden, um beispielsweise Maximal- und Minimalwerte, Brems- und Beschleunigungsphasen schneller erkennen zu können.

Technologieauswahl

Damit die Benutzeroberfläche plattformunabhängig bleibt und nicht installiert werden muss, wurde sich für eine browserbasierte Oberfläche entschieden. Dafür wurde **Angular** gewählt. *Angular* strukturiert Projekte in *components* (Komponenten), welche leicht als wiederverwendbare Bausteine in andere eingesetzt werden können und so die Entwicklung der Benutzeroberfläche vereinfacht. *Angular* verwendet die Websprache *TypeScript*, die im Gegensatz zu *JavaScript* Typsicherheiten bietet, sodass viele Fehler schon zur Compile-Zeit entdeckt werden.

Die Benutzeroberfläche ist als Single-Page-Application (SPA) konzipiert: Nach dem initialen Laden werden alle Ansichten client-seitig gerendert, ohne dass bei jeder Interaktion eine neue Seite vom Web-Server geladen werden muss. Dadurch wird die Benutzeroberfläche flüssiger, was die User Experience verbessert.

Aufbau

Die Benutzeroberfläche wurde in Zuständigkeitsbereiche eingeteilt, die jeweils eine eigene Ansicht bekommen. Die identifizierten Ansichten sind:

1. **Fahrzeuge**
2. **Fahrten**
3. **Personen(potentielle Fahrer)**
4. **Telemetrieeinheiten**

Eine Kopfleiste soll für jede der vier Seiten einen Reiter enthalten. Ein Klick auf den Reiter wechselt in die entsprechende Ansicht.

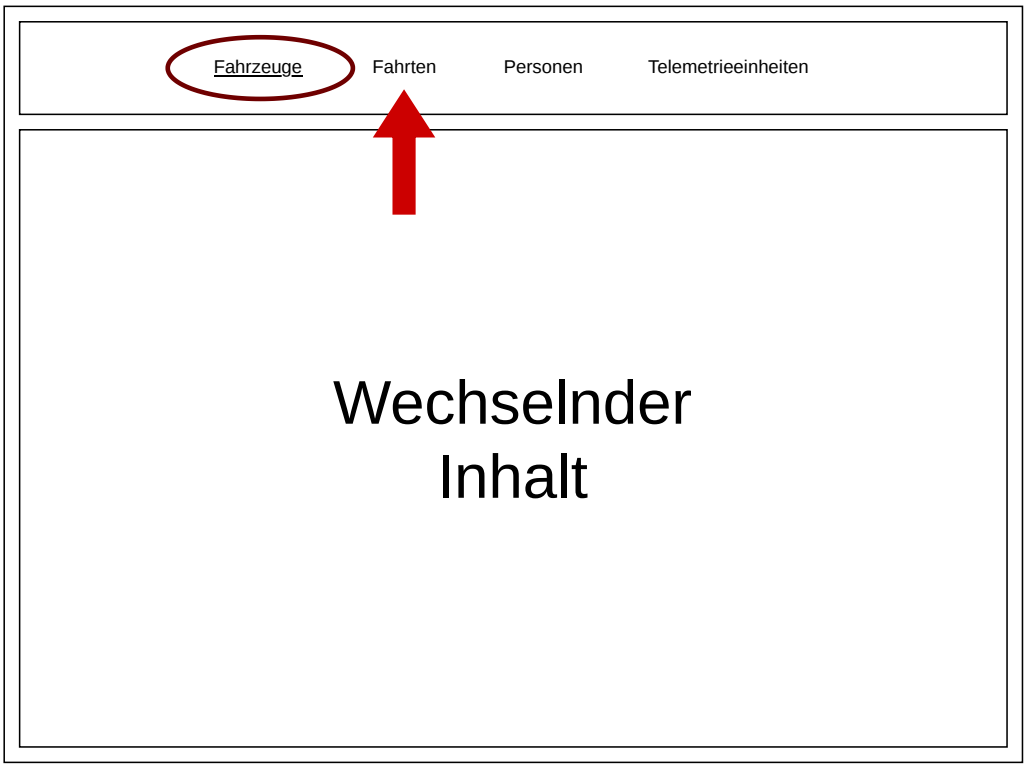


Abbildung 4.22: Darstellung des Navigationskonzept

Ansicht Fahrzeuge

Die Fahrzeugeseite soll eine Karte zeigen, die die letzte Position aller Fahrzeuge darstellt und gerade aktive Fahrzeuge (Vergangene Zeit nach letztem Positionsupdate ist kleiner als bestimmter Wert) durch Pulsieren des Positionspunkts markiert.

Darunter soll sich eine Liste der bereits angelegten Fahrzeugen befinden. Zwischen Karte und Liste soll eine Schaltfläche die Neuanlage eines Fahrzeugs ermöglichen. Bei Betätigung der Schaltfläche soll ein Anlageformular erscheinen, das die Liste nach unten verdrängt. Wenn die Anlage abgebrochen oder abgeschlossen wird, soll das Formular wieder verschwinden.

Die Felder der Fahrzeuganlage ergeben sich aus dem Datenmodell von *FleetControlApi* (siehe Abschnitt 4.3.4). Die Datenkonsistenz soll durch Einschränkung der Anlagemaske gesichert werden. Pflichtfelder werden markiert, um dem Anwender zu signalisieren, dass sie eingegeben werden müssen. Der Abschluss der Anlage ist sonst nicht möglich. Die Einschränkungen sehen wie folgt aus:

- **Kennzeichen:** Max. 9 Zeichen (8 und das Trennzeichen), optional (Fahrzeug könnte noch nicht zugelassen oder gerade abgemeldet sein)
- **Marke:** Max. 50 Zeichen, Pflichtfeld
- **Modell:** Max. 50 Zeichen, Pflichtfeld
- **Baujahr:** Zahl, Pflichtfeld
- **Fahrzeug-Identnr.:** Pflichtfeld, es müssen genau 17 Zeichen eingegeben werden, da eine moderne internationale FIN genau 17 Zeichen beträgt. Ein Texthinweis unter dem Eingabefeld soll darauf hinweisen.
- **Benötigter Führerschein:** Dropdownliste mit den Optionen, Pflichtfeld.
- **Leistung (PS):** Zahl, min. 0, Pflichtfeld.
- **Farbe:** Max. 50 Zeichen, Pflichtfeld.
- **Erstzulassung:** Datum, optional, da das Auto auch als Neuwagen eine unzugelassene Neuanlieferung sein könnte.

Da beim Anwählen eines Listenelements die Karte während der Auswahl nur die Position dieses Fahrzeugs exklusiv anzeigen soll, muss die Karte dabei noch sichtbar bleiben, damit die Nutzung nicht unpraktisch wird.

Ein angelegtes Fahrzeug erscheint in der Liste, die deshalb auf immer nur zehn Fahrzeuge gleichzeitig begrenzt sein soll. Bei mehr als zehn Fahrzeugen soll es eine Seitenauswahl unter der Liste geben.

Die Auswahl eines angelegten Fahrzeugs filtert die Positionskarte und expandiert die sonst minimierten Fahrzeugdetails. Sie können betrachtet oder geändert werden, wobei nur die variablen Daten änderbar sind. Die unveränderlichen Fahrzeugdaten sollen auch als unveränderlich im System abgebildet werden. Sollte ein Fehler bei der Anlage passieren, muss das Fahrzeug gelöscht und nochmal neu angelegt werden.

Die veränderlichen Felder sind:

- **Kennzeichen:** Kann bei Zulassung dazukommen bzw. sich ändern, bei Abmeldung wegfallen.
- **Erstzulassung:** Kann dazukommen, wenn das Auto ein angelieferter Neuwagen ist.
- **Telemetrieeinheit:** Ein Fahrzeug muss mit einer Telemetrieeinheit eindeutig verknüpft werden können. Eine Auswahl soll durch eine Dropdown-Liste erfolgen, die die IDs aller angelegten Telemetrieeinheiten auflistet (bereits

verbundene unwählbar, graphisch durch hellere Schrift markiert). Die Verbindung kann auch wieder getrennt werden, sprich es gibt die Option keine Telemetrieeinheit.

- **Fahrer:** Ein Fahrzeug muss mit einem Fahrer eindeutig verknüpft werden können. Eine Auswahl soll durch eine Dropdown-Liste erfolgen, die die IDs und Namen aller angelegten Fahrer auflistet (bereits verbundene unwählbar, graphisch durch hellere Schrift markiert). Die Verbindung kann wieder getrennt werden, sprich es gibt die Option kein Fahrer.

Da große Mengen von Fahrzeugen im System existieren können, soll es rechts neben Karte und Liste einen Filterbereich geben, der die angezeigten Fahrzeuge einschränkt. Die Filteroptionen sollen additiv (eine Eingabe muss zutreffen) und restriktiv (alle Eingaben müssen zutreffen) anwendbar sein.

Fahrzeuge können wieder gelöscht werden, dazu soll es eine Schaltfläche in den Fahrzeugdetails geben. Um ein Fahrzeug löschen zu können, darf es keine Fahrten des Fahrzeugs mehr geben. Sie müssen vorher gelöscht werden. Andernfalls würden sich die Fahrten auf nicht mehr vorhandene Fahrzeuge beziehen und die Qualität der Datenbasis wäre beschädigt. Bestehende Verknüpfungen werden im Falle einer erfolgreichen Löschung aufgehoben. Eine Fehlernachricht soll darauf hinweisen, dass zugehörige Fahrten erst gelöscht werden müssen und die Löschung bis dahin unmöglich ist.



Abbildung 4.23: Grobe Darstellung der Fahrzeugeansicht

Ansicht Fahrten

Die Fahrtenansicht soll ebenfalls eine Karte besitzen, die abgeschlossene Fahrten darstellt. Dazu werden die Datenpunkte eingezeichnet und zu farbigen Routen verbunden. Start und Ziel sollen gekennzeichnet werden, Bewegung des Mauszeigers auf einen Datenpunkt soll die zugehörigen Daten zeigen. Unter der Karte soll eine Schaltfläche die Karte gegen eine Graphdarstellung einer Fahrt austauschen können. Vorhandene Fahrten werden wie schon die Fahrzeuge in einer Liste mit maximal zehn Elementen gezeigt. Die zehn Fahrten der aktuellen Seite werden auf der Karte angezeigt. Auswahl einer Fahrt expandiert die Fahrtdetails und beschränkt die Karte auf nur diese Fahrt, solange die Auswahl nicht durch eine Schaltfläche abgebrochen wird. Es soll eine Schaltfläche zur Löschung in den Fahrtdetails geben. Folgende Fahrtdetails werden festgehalten:

1. **Fahrzeug:** ID und Kennzeichen.
2. **Telemetrie-Einheit:** Geräte-ID.

3. **Fahrer:** ID und Name.
4. **Start:** Zeitstempel.
5. **Ende:** optionaler Zeitstempel.
6. **Anzahl Datenpunkte**

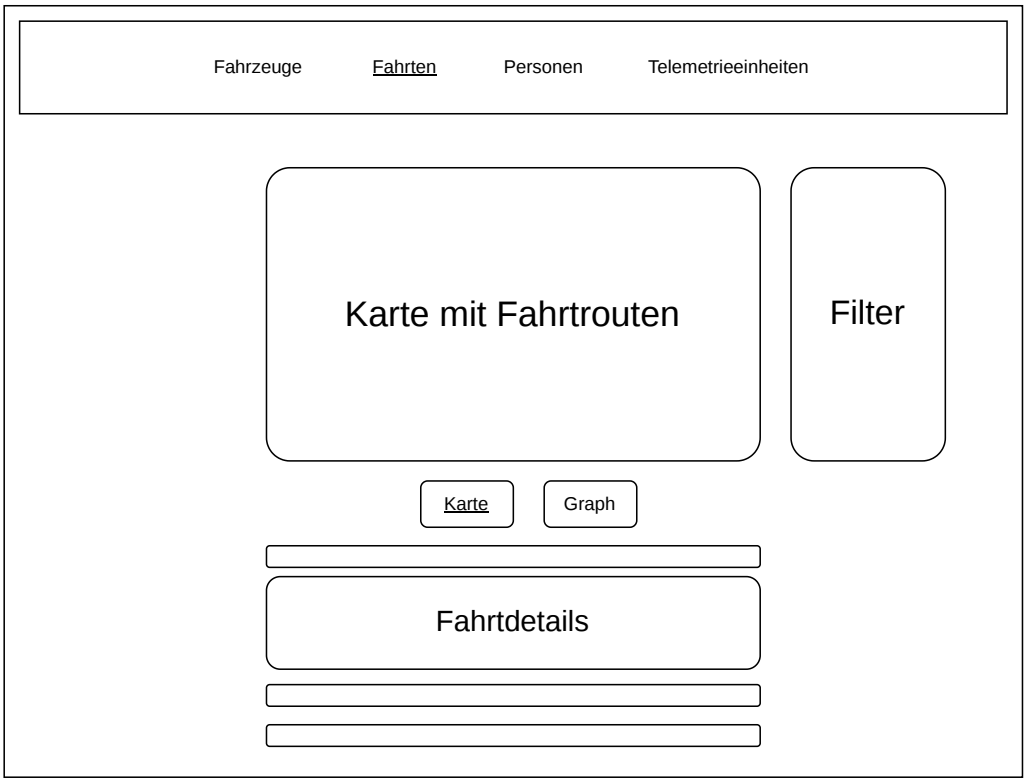


Abbildung 4.24: Grobe Darstellung der Fahrtenansicht

Ansicht Personen

Die Personenansicht soll nur eine Liste, wie es sie bereits in den anderen Ansichten gibt, und eine Schaltfläche zur Neuanlage darüber bieten. Die Betätigung soll wieder ein Anlageformular expandieren. Pflichtfelder sollen als solche markiert werden, die fehlerhafte Anlage soll zur Wahrung der Datenqualität unmöglich sein. Folgende Felder soll die Eingabemaske bekommen:

- **Vorname** Text, max. 50 Zeichen, Pflichtfeld

- **Nachname** Text, max. 50 Zeichen, Pflichtfeld
- **Geburtsdatum** Datum, Pflichtfeld
- **Führerscheine:** Klasse und Erwerbsdatum, mehrere Einträge sollen möglich sein, optional

Auswahl einer Person in der Liste soll die Details expandieren und bearbeitbar machen, die Zuweisung als Fahrer zu einem Fahrzeug soll hier möglich sein. Personen sollen ebenfalls gelöscht werden können, aber nur, wenn die zugehörigen Fahrten vorher gelöscht wurden (Verweise auf nichtidentifizierbare Fahrer unmöglich).

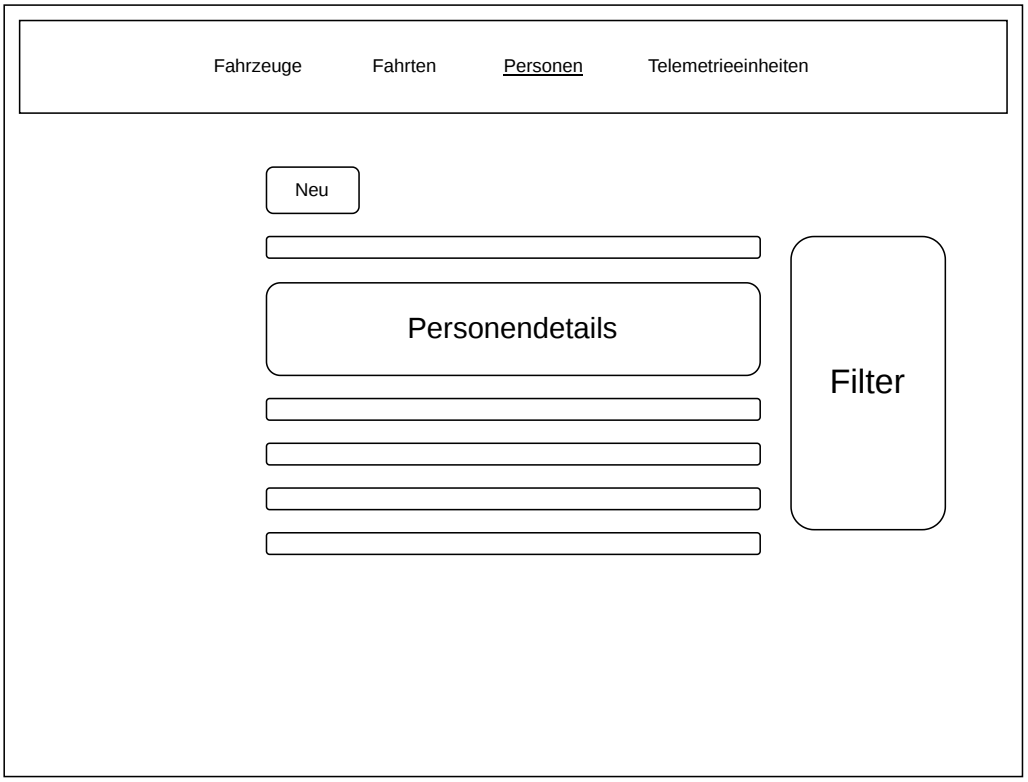


Abbildung 4.25: Grobe Darstellung der Personenansicht

Ansicht Telemetrieeinheiten

Die Ansicht für Telemetrieeinheiten soll ebenfalls die seitenbasierte Liste für bereits angelegte Einheiten bekommen. Darüber soll eine Schaltfläche für die Neuanlage liegen, allerdings soll diese unanwählbar sein, solange keine Telemetrieeinheit über

USB verbunden ist.

Eine systemseitige Anlage benötigt die global einzigartige Geräte-ID einer Telemetrieeinheit in Form einer UUID. Dazu soll eine Schaltfläche neben der Geräteanlage existieren, die den Befehl (Zeichenkette) zur Herausgabe dieser ID an alle USB-Anschlüsse des Rechners schickt. Wird von einem Anschluss mit einer UUID geantwortet, wird diese in einem Feld neben der Broadcast-Schaltfläche angezeigt und die Anlageschaltfläche wird anwählbar bzw. eine Anlage wird möglich. Die Geräte-ID wird der Identifikator im System, so wird die Konsistenz von Hardwareeinheitenbestand und der Repräsentation im System garantiert. Die Auswahl einer Einheit in der Liste soll die Details expandieren, Löschung und Verknüpfung mit einem Fahrzeug sollen darin möglich sein.

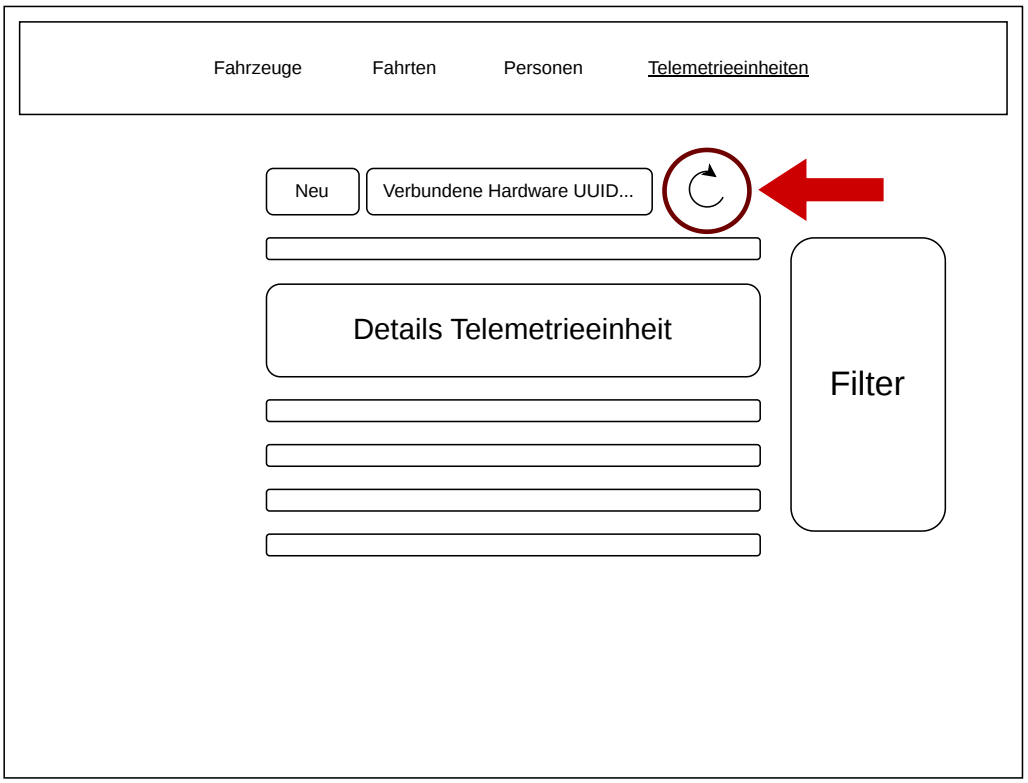


Abbildung 4.26: Grobe Darstellung der Telemetrieeinheitenansicht, in rot das Auslösen des Broadcast-Befehls zum Verbindungsaufbau mit einer angeschlossenen Hardware-Einheit

Modellierung der Angular-Komponenten

Die Benutzeroberfläche soll als Single Page Application (SPA) umgesetzt werden, daher gibt es eine zentrale Klasse `MainPageComponent`, die die ausgewählte Seite hält und die Navigation steuert. Sie hält zusätzlich eine Funktion, um die Färbung der Oberfläche zu wechseln (Dunkel / Hell).

Für jede der vier Seiten gibt es eine Komponente:

1. `VehiclesPageComponent`
2. `TripPageComponent`
3. `PersonPageComponent`
4. `TelemetryUnitPageComponent`

Sie verwenden die Unterkomponenten `FilterComponent` (Generischer Filterbereich) und `AppliedFilters` (Seitenspezifische Filteroptionen). Jeder der vier Seiten verwendet die gleiche Filterkomponente, die für jede Seite einzigartigen Filterfelder werden bei der Seitenerstellung als Konfiguration mitgegeben.

`VehicleMapComponent` und `CardWidgetComponent` stellen die Karte der Fahrzeugseite bzw. die der Fahrtenseite dar und werden von den zugehörigen Komponenten instanziiert. Für die Darstellung der Daten gibt es eigene Modellklassen.

Die Kommunikation mit *FleetControlApi* erfolgt über eigene Service-Klassen. Die Verbindungsabfrage über die *Web Serial API* (Broadcast des Geräte-ID-Abfragebefehls) mit einer verbundenen Hardware-Telemetrieeinheit erfolgt über `UsbTelemetryService`. Ursprünglich war für die Klasse geplant, dass sie eine HTTP-Anfrage an *FleetControlApi* stellt und *FleetControlApi* auf die USB-Hardware zugreift. Dies stellte sich jedoch bei der Inbetriebnahme als Fehlentwurf heraus und wurde geändert (siehe Abschnitt 5.1.5).

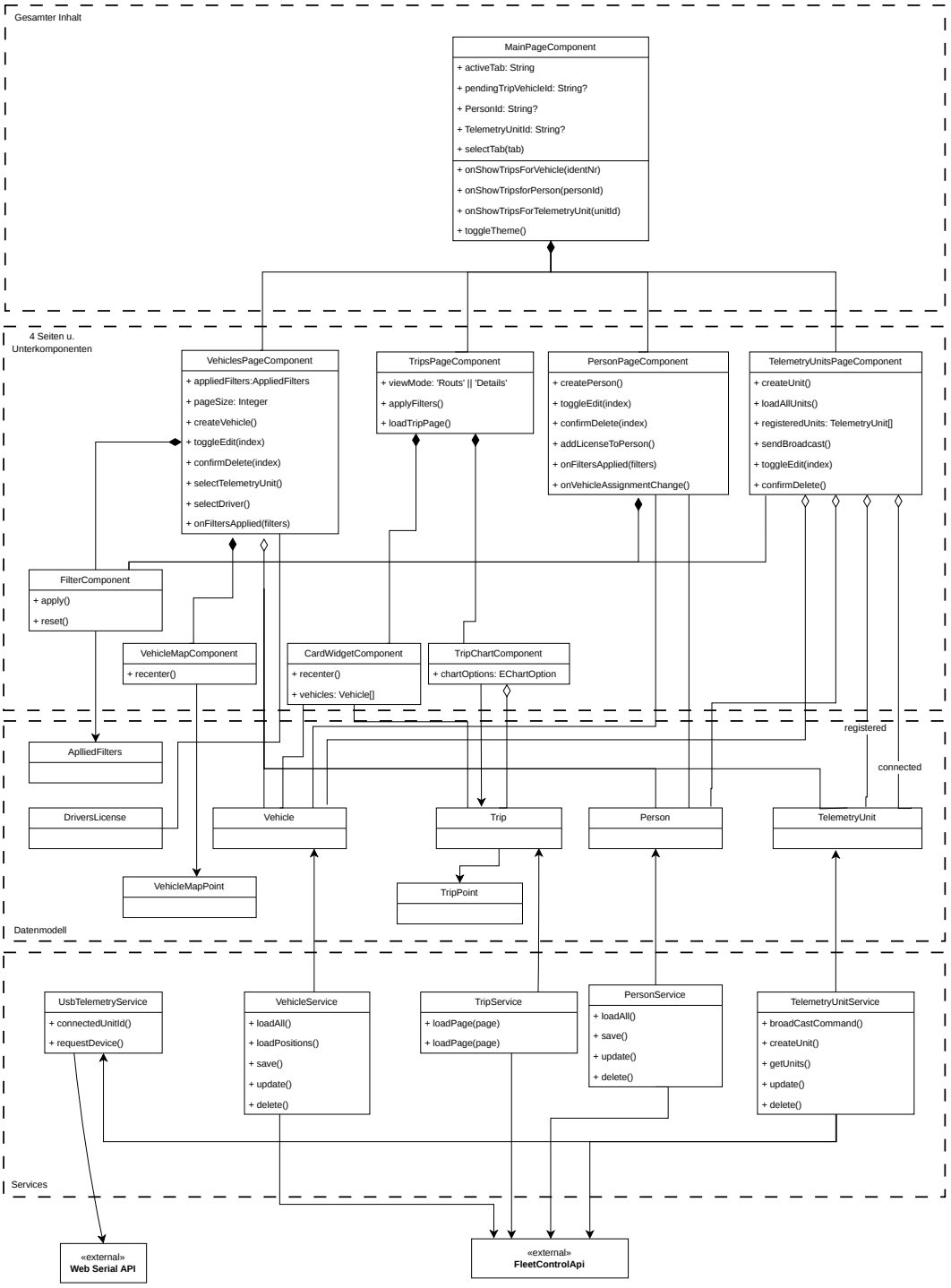


Abbildung 4.27: Unvollständiges Klassendiagramm der Entwurfsebene von *FleetControlUI*

5 Implementierung

5.0.1 TelemetryUnit

Projekterstellung

Das Mikrocontrollerprojekt benötigt zusätzliche Komponenten, die ein Software-Projekt, das auf einem Rechner mit Betriebssystem läuft, nicht braucht:

- Ein architekturspezifisches Linker-Skript.
- Eine Abstraktion um direkte, adressbasierte Registerzugriffe wie die *HAL*-Bibliothek. So bleibt der Code lesbar und ist schneller zu entwickeln. Außerdem bleibt der Anwendungscode selbst unabhängig von der Hardware.
- RTOS-Treiber
- Zusätzlich zum Anwendungscode müssen alle verwendeten Hardwarekomponenten für die gewünschte Nutzung konfiguriert werden, beispielsweise um sie überhaupt zu aktivieren oder eine von mehreren Funktionen auszuwählen. Man benötigt dafür zusätzlichen Konfigurationscode.

Der Hersteller des Mikrocontrollers stellt das Programmgeneratorwerkzeug *STM32CubeMX* zur Verfügung, mit dem ein leeres Projekt für die Zielhardware mit allen benötigten Abhängigkeiten graphisch konfiguriert und dann generiert werden kann, inklusive dem Hardwarekonfigurationscode.

Folgende Konfiguration wurde über STM32CubeMX vorgenommen:

- **LED und Button:** Nucleo-Standardpins – `LD2` (grüne LED) an GPIO-Pin PA5, `UserButton` an PC13-Pin.
- **Zwei UARTs:** `USART2` (PA2/PA3, 115200 Baud) als Verbindung zum Rechner über den ST-Link Virtual-COM-Port, `USART3` (PC10/PC11) zum SIM7600-Modul
- **RTOS:** FreeRTOS mit einem Gesamt-Heap von 5120 Byte und einer Mindest-Stackgröße von 128 Worten pro Task (Standardeinstellung)
- **Drei Tasks:** `defaultTask` (taskübergreifende Aktionen), `CommandHandler`

(Ausgabe der Geräte-ID an angeschlossenen Rechner) und `TelemetryFsm` (Zustandsautomat für Datenerfassung und -sendung)

- **Zwei Mutexe:** `huart_sim_mutex` und `huart_debug_mutex` zum Schutz der beiden UART-Schnittstellen vor nebenläufigem Zugriff mehrerer Tasks.
- **I2C:** wurde ursprünglich für die Anbindung der IMU-Sensoreinheit MPU-6050 konfiguriert, wurde aber letztendlich nicht genutzt (siehe Abschnitt 5.1.5) und im Zuge der Implementierung wieder entfernt.

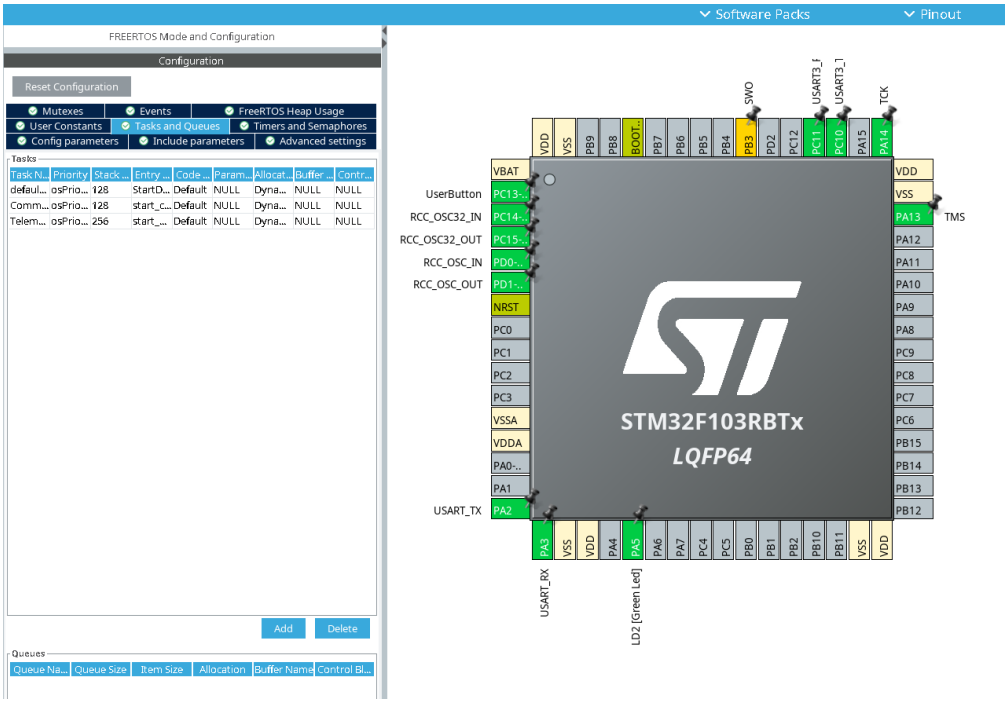


Abbildung 5.1: Hardwarekonfiguration des Projekts in STM32CubeMX

Der Codegenerator erzeugt ein Standardprojekt, auf dessen Basis eine Erweiterung erfolgte. Die Idee der erweiterten Projektstruktur ist, dass der gesamte Anwendungscode für den Programmablauf in einem eigenen Verzeichnis **App/** vom hardwareabhängigen Code in einem Verzeichnis **Platform** isoliert wird, damit man dessen Funktionalität mit Unittests in einem Verzeichnis **Tests/** absichern kann (z. B. Google Test). Außerdem hat das den Vorteil, dass die angeschlossene Hardware für Datenerhebung und Internetkommunikation ausgetauscht werden kann, ohne den Anwendungscode verändern zu müssen. Der Anwendungscode kann für Unittests unabhängig kompiliert werden, für die Abhängigkeiten können dann Mock-Objekte verwendet werden. Dadurch wird sowohl die Testbarkeit als auch Änderbarkeit und Modularität verbessert.

Folgende Aufteilung wurde vorgenommen:

- `App/` – Anwendungslogik: 3 Zustandsautomaten (`Telemetry`, `Network`, `Gnss`)
  - `Telemetry/` – Hauptautomat
  - `Network/` – Automat für Netzwerk-/MQTT-Verbindung
  - `Gnss/` – Automat für GNSS-Verbindung
- `Platform/` – hardwareabhängiger Code (SIM7600-Modul, UART, LED, Geräte-ID-Ermittlung)
  - `SIM7600EH/` – Direkte Befehlskommunikation mit dem LTE/GNSS-Modul
  - `GetDeviceId/` – Geräte-ID-Ermittlung und UUID-Formatierung
  - `MPU6050/` – Treiber für IMU-Sensoreinheit zur Bestimmung von Beschleunigung und Geschwindigkeit (ungenutzt)
  - `PrintUtils/` – Hilfsfunktionen für Debug-Ausgaben (Nicht im Zuge dieser Arbeit verfasst, wurden nur teilweise im Projekt verwendet)
- `Core/` – von STM32CubeMX generierter Code (HAL-Konfiguration, RTOS-Einbindung) – generierte Quelldateien (u. a. `freertos.c`)
- `Tests/` – vorgesehen für UnitTests des Anwendungscodes, im Rahmen dieses Projekts leer.
- Weitere generierte Standardprojektdateien

Uart-Kapselung

Die Uart-Schnittstellen werden vom gesamten Code im *TelemetryUnit* benötigt. Da auf die Schnittstellen von mehreren nebenläufigen RTOS-Tasks gleichzeitig zugegriffen werden soll, muss der Zugriff mit Mutexen geschützt werden. Andernfalls kann es vorkommen, dass Teile von Befehlsantworten von außenstehenden Tasks ausgelesen werden und somit in der zuständigen Task die Antwort nicht mehr identifiziert werden kann. FreeRTOS-kompatible Mutexe können in STM32CubeMx konfiguriert werden, für beide UART-Schnittstellen wurde ein Mutex eingebaut. Um die direkte Uart-Kommunikation sowie die Mutex-Nutzung zu kapseln und vom restlichen Code zu trennen, wurde die Klasse `Uart` erstellt.

```
class Uart
{
  public:
    Uart(
      UART_HandleTypeDef* _huart,
      osMutexId_t& huart_sim_mutexHandle
    );

    HAL_StatusTypeDef read(
      // ... Lesefunktion für AT-Befehle
    );

    HAL_StatusTypeDef readRaw(
      // ... generische Lesefunktion
    );

    HAL_StatusTypeDef write(
      // ... generische Schreibfunktion
    );

  private:
    UART_HandleTypeDef* huart;
    osMutexId_t& mutex;

    void acquire();
    void release();
};
```

Listing 5.1: mutexgesicherte UART-Nutzung in Uart-Klasse (Uart.hpp)

```
    HAL_StatusTypeDef Uart::write(uint8_t* data, uint16_t len,
       uint32_t timeout_ms)
    {
      acquire();
        HAL_StatusTypeDef status =
          HAL_UART_Transmit(huart, data, len, timeout_ms);
      release();
      return status;
    }
```

Listing 5.2: Mutex-Kapselung innerhalb von `Uart::write` (Uart.cpp)

Befehlshandler und Geräte-Identifikation

Der Befehlshandler für die Geräte-ID-Ausgabe für die systemseitige Anlage der Telemetrieeinheit wurde vor den Automaten implementiert. Die Umsetzung erfolgte in einer Prüffunktion, die in einer Schleife wiederholt ausgeführt werden muss, um die UART-Leitung dauerhaft zu überwachen (später in eigener RTOS-Task).

Auf die Erstellung als Klasse wie im Lösungskonzept beschrieben wurde verzichtet, da auf oberster Ebene nur eine Funktion ohne Klassenattribute gebraucht wird und die Klassenabstraktion damit unsinnig wäre.

In `get_device_id.cpp` übernimmt die Funktion `check_for_device_id_request_command` diese Prüfung: Sie liest blockierend byteweise von der Debug-UART und sammelt die Zeichen bis zu einem Zeilenumbruch in einem Puffer. Danach wird der Puffer mit dem einzig unterstützten Befehl `get_device_id` verglichen.

```
1   extern "C" void check_for_device_id_request_command(void)
2   {
3     uint8_t byte_read;
4     uint8_t bytes_read_count = 0;
5
6     while (true)
7     {
8       if (huart_debug_wrapper.readRaw(&byte_read, 1,
            HAL_MAX_DELAY) == HAL_OK)
9       {
10        if (byte_read == '\n' || byte_read == '\r')
11        {
12          cmd_buffer[bytes_read_count] = '\0';
13
14          if (is_get_id_command(cmd_buffer, bytes_read_count))
15          {
16            stm32_get_uid_as_uuid(uuid_device);
17            huart_debug_wrapper.write((uint8_t*)uuid_device,
                36);
18            huart_debug_wrapper.write((uint8_t*)"\r\n", 2);
19          }
20
21          bytes_read_count = 0;
22        }
23        else
24        {
25          cmd_buffer[bytes_read_count++] = byte_read;
26        }
27      }
28    }
29  }
```

Listing 5.3: Byteweises Einlesen und Befehlserkennung (check_for_device_id_request_command)

Für das Lesen und Senden verwendet die Funktion die `Uart`-Klasse. Wird der Befehl erkannt, übernimmt die Funktion `stm32_get_uid_as_uuid`, die die 96-Bit-Geräte-ID extrahiert und im UUID-Format zurückgibt.

Da `check_for_device_id_request_command` aus `freertos.c` (RTOS-Tasks) heraus aufgerufen wird – einer reinen C-Datei –, deklariert der Header `get_device_id.h` die Funktion innerhalb eines `extern "C"`-Blocks. Somit kann die C++-Datei auch im C-Code eingebunden werden.

SIM7600-Modul

Nach dem Handler erfolgte die Implementierung der Hardwarekommunikation mit dem LTE/GNSS-Modul. Die Klasse `Sim7600` kapselt die gesamte Kommunikation mit dem Modul, die über AT-Befehle funktioniert.
Eine Trennung der Funktionalität fand bewusst nicht im hardwareseitigen Code im Verzeichnis **Platform/** statt, da die Funktionalität auf dieser untersten Ebene zusammengehört. Eine Trennung erfolgte im hardwareunabhängigen Anwendungscode in **App/**.
Die Klasse hält zwei `Uart`-Instanzen (LTE/GNSS-Modul und PC-Kommunikation). Eine Lesefunktion `Uart::read` liest byteweise (10 ms Timeout je Byte) von der Leitung und bricht ab, sobald der Puffer ein Endzeichen enthält (`"\r\nOK\r\n"`, `"\r\nERROR\r\n"`). Methoden sind nach Aufgabenbereich gruppiert: Netzwerkverbindung, GNSS und MQTT.

```
1   class Sim7600
2   {
3     public:
4       // Netzwerkverbindung
5       bool check_connection();
6       bool check_sim();
7       bool check_network();
8       bool check_signal();
9       bool check_attach();
10      bool set_apn();
11      bool activate_pdp();
12      bool get_ip(char* out_ip, uint16_t max_len);
13      void establish_network_connection();
14      bool is_network_ready();
15
16      // GNSS
17      bool activate_gnss();
18      bool gnss_nmea_start();
19      void get_gnss(uint8_t* buffer, uint16_t buffer_length);
20      bool has_fix();
21      bool get_gnss_fix(uint8_t* buffer, uint16_t buffer_length);
22
23      // MQTT
24      bool mqtt_start();
25      bool mqtt_create_client();
26      bool mqtt_connect();
27      bool mqtt_disconnect();
28      bool mqtt_publish(const char* payload);
29      bool is_mqtt_connected();
30      bool mqtt_stop();
31
32      Uart huart_sim;
33      Uart huart_debug;
34  };
```

Listing 5.4: Öffentliche Schnittstelle der Klasse Sim7600 (sim7600eh.hpp)

```
1  bool Sim7600::check_network()
2  {
3    uint8_t cmd[] = "AT+CREG?\r\n";
4    uint8_t response[128] = {0};
5
6    huart_sim.write(cmd, sizeof(cmd) - 1);
7
8    bool ok = (huart_sim.read(response, sizeof(response) - 1,
         2000) == HAL_OK);
9
10   if (!ok) {
11     huart_debug.write((uint8_t*)"[NET] No response\r\n", 20);
12     return false;
13   }
14
15   bool registered =
16     strstr((char*)response, ",1") || strstr((char*)response,
           ",5");
17
18   return registered;
19 }
```

Listing 5.5: AT-Befehl senden und Antwort auswerten (Sim7600::check_network)

Telemetriedatenverarbeitung

Nach Implementierung des Hardwarezugriffs wurde der hardwareunabhängige Anwendungscode in **App/** implementiert. Begonnen wurde beim Aufbau eines Datenpunkts.

Die Klasse `TelemetryPoint` übernimmt die Abfrage der Telemetriedaten sowie die Aufbereitung zu einem vollständigen Datenpunkt.

Die Funktion `parse_gnss` führt die Abfragen durch, empfängt und zerlegt die Antwort und liest die benötigten Werte aus (Breiten-/Längengrad, Himmelsrichtung, Datum, UTC-Zeit, Geschwindigkeit in Knoten).

Die Abfrage selbst wurde versehentlich in `TelemetryPoint` implementiert. Das widerspricht dem Ansatz, dass der hardwareabhängige Code ausschließlich in **Platform/** liegt; dies wurde in die bekannten Probleme mit aufgenommen (siehe Abschnitt 7.1.3).

Jedes Koordinatenfeld wird vor der Umrechnung strukturell geprüft (`is_valid_nmea_coordinate`). Koordinaten liegen im NMEA-Format

Grad+Minuten vor (`ddmm.mmmmmm` bzw. `dddmm.mmmmmm`) und werden in `nmea_to_decimal` in Dezimalgrad umgerechnet. Die Geschwindigkeit wird vom GNSS-Modul in Knoten geliefert und mit dem Faktor 1,852 in km/h umgerechnet.

Datum (`ddmmyy`) und UTC-Zeit (`hhmmss.sss`) werden getrennt geliefert und in `format_timestamp` zu einem ISO-8601-Zeitstempel zusammengesetzt. Da die Jahreszahl im NMEA-Format zweistellig vorliegt, wird sie um "20" ergänzt.

`to_json` serialisiert den fertigen Datenpunkt in ein kompaktes JSON-Objekt inklusive Beschleunigung `accel_in_ms2`, deren Berechnung in Abschnitt 5.0.1 beschrieben wird. Die Geräte-ID wird dabei nur einmalig über `stm32_get_uid_as_uuid` abgefragt und danach zwischengespeichert, da sie sich zur Laufzeit nicht ändert.

```
1   const char* TelemetryPoint::to_json()
2   {
3     static char json[192];
4     char timestamp[32];
5
6     if (device_id[0] == '\0')
7     stm32_get_uid_as_uuid(device_id);
8
9     format_timestamp(timestamp, sizeof(timestamp));
10
11    snprintf(
12    json, sizeof(json),
13    "{"
14      "\"device_id\":\"%s\","
15      "\"timestamp_iso_8601\":\"%s\","
16      "\"lat\":%.6f,"
17      "\"lon\":%.6f,"
18      "\"speed_in_kmh\":%.2f,"
19      "\"accel_in_ms2\":%.3f"
20      "}",
21    device_id, timestamp,
22    gnss_data.latitude, gnss_data.longitude,
23    gnss_data.speed_in_kmh, gnss_data.accel_in_ms2
24    );
25
26    return json;
27  }
```

Listing 5.6: JSON-Serialisierung eines Datenpunkts (TelemetryPoint::to_json)

Geschwindigkeit und Beschleunigungsberechnung

Ursprünglich war geplant, Geschwindigkeit und Beschleunigung über dedizierte Sensoren zu erfassen. Dafür wurde die MPU6050-Sensoreinheit im Lösungskonzept eingeplant. Mehrere Funktionen zur Abfrage wurden implementiert. Aufgrund des in Abschnitt 5.1.5 beschriebenen Problems wurde am Ende stattdessen der Geschwindigkeitswert genommen, der vom LTE/GNSS-Modul abgefragt werden kann, beziehungsweise die Beschleunigung aus dieser berechnet.

Sie wird aus der Differenz zweier aufeinanderfolgender GPS-Geschwindigkeitswerte abgeleitet: `calculate_accel_in_ms2` berechnet die Geschwindigkeitsänderung zwischen dem aktuellen und dem vorherigen GNSS-Fix und teilt sie durch die verstrichene Zeit in Sekunden (`HAL_GetTick()`).

```
1  float TelemetryPoint::calculate_accel_in_ms2(float
       new_speed_kmh)
2  {
3    uint32_t now = HAL_GetTick();
4    float accel = 0.0f;
5
6    if (prev_tick_ms != 0)
7    {
8      float dt = (now - prev_tick_ms) / 1000.0f;
9      float dv = (new_speed_kmh - prev_speed_kmh) / 3.6f;
10     accel = dv / dt;
11   }
12
13   prev_speed_kmh = new_speed_kmh;
14   prev_tick_ms = now;
15
16   return accel;
17 }
```

Listing 5.7: Beschleunigung aus GPS-Geschwindigkeitsdifferenz (TelemetryPoint::calculate_accel_in_ms2)

Zustandsautomaten

Die drei im Lösungskonzept beschriebenen Automaten (`TelemetryFsm`, `NetworkFSM`, `GnssFSM`) wurden mit den dort benannten Zuständen als C++-Klassen umgesetzt, jeweils mit einem `Enum` pro Automat und einer `step()`-Methode, die bei jedem Aufruf einen Zustandsübergang ausführt.

Statt in drei getrennten Tasks zu laufen, werden alle drei `step()`-Methoden gemeinsam im Task `TelemetryFsm` einmal pro Sekunde ausgeführt in der Reihenfolge GNSS, Netzwerk, Telemetrie.

```
1     case TelemetryState::Idle:
2     {
3       if (is_network_ready() && is_gnss_ready())
4       {
5         last_publish = 0;
6         current_state = TelemetryState::Publishing;
7       }
8       break;
9     }
```

Listing 5.8: Zustandsübergang Idle -> Publishing (TelemetryFsm::step)

Im Zustand `Publishing` führt `publish_timestamp` den eigentlichen Sendevorgang aus. Ohne gültigen GNSS-Fix wird der Versuch übersprungen, andernfalls wird die von `GnssFsm` zwischengespeicherte Rohantwort weiterverwendet.

```
1     bool TelemetryFsm::publish_timestamp()
2     {
3       if (!gnss_fsm_has_valid_fix())
4       return false;
5
6       telemetry_point.parse_gnss(gnss_fsm_get_last_response());
7
8       const char* json = telemetry_point.to_json();
9       bool ok = telemetry_module.mqtt_publish(json);
10
11      network_fsm_report_publish_result(ok);
12
13      return ok;
14    }
```

Listing 5.9: Vollständiger Sendevorgang (TelemetryFsm::publish_timestamp)

Die im Lösungskonzept in den Diagrammen nur benannten Zeitwerte sind konkret:

- Publish-Intervall: 1000 ms
- Health-Check-Intervall (Netzwerk, im Zustand `NetworkMqttConnected`): 5000 ms

- Wechsel nach `NetworkError` nach 3 aufeinanderfolgenden fehlgeschlagenen Publishes
- Wechsel nach `GnssError` nach 60 s ohne gültigen GNSS-Fix

C/C++-Vermittlung

Wird ein RTOS in STM32CubeMX konfiguriert, generiert es die Datei `freertos.c` in reinem C, die für jede konfigurierte Task einen leeren Funktionskörper enthält. Dieser muss um die eigentliche Task-Logik ergänzt werden. Da die drei Automaten-Klassen in C++ implementiert wurden, können ihre `step()`-Methoden nicht direkt aus dieser C-Datei heraus aufgerufen werden – daher braucht es eine Vermittlungsschicht.

Jede der drei Automaten-Klassen ist deshalb in einer eigenen `interface.cpp`/`interface.h` gekapselt: Eine statische Instanz der jeweiligen C++-Klasse wird über `extern "C"`-Funktionen für den C-Code aufrufbar gemacht. Nur die benötigten Funktionen werden exportiert.

```
1   #include "interface.h"
2   #include "network_fsm.hpp"
3   #include "../../Platform/SIM7600EH/sim7600eh.hpp"
4
5   static Sim7600 sim;
6   static NetworkFSM network_fsm(sim);
7
8   extern "C" void network_fsm_step()
9   {
10    network_fsm.step();
11  }
12
13  extern "C" void network_fsm_report_publish_result(bool ok)
14  {
15    network_fsm.report_publish_result(ok);
16  }
17
18  NetworkState network_fsm_get_state()
19  {
20    return network_fsm.get_state();
21  }
```

Listing 5.10: C/C++-Vermittlung am Beispiel von NetworkFSM (Network/interface.cpp)

Betrieb und Programmablaufsteuerung

Der Durchlauf des Automaten *TelemetryFsm* stellt den Normalbetrieb der Telemetrieeinheit dar. Wird sie eingeschaltet (Stromverbindung besteht), sendet die Telemetrieeinheit Daten. Im Zuge der Entwicklung wurde festgestellt, dass es bei dem Betrieb der Tasks dazu kommen kann, dass bei eingehendem Befehlen wegen der RTOS-Kontextwechsel vom PC Zeichen verloren gehen können und die Befehle nicht immer erkannt werden (siehe Abschnitt 5.1.5).

Damit im Falle einer Verbindung mit einem Rechner der Befehl für die ID-Ausgabe korrekt abgehandelt werden kann, muss sichergestellt werden, dass auf der UART-Leitung, die für die PC-Kommunikation gedacht ist, der Ausgabebefehl vollständig eingelesen werden kann, ohne dass es dazwischen einen RTOS-Kontextswitch gibt, der zu Fehlern führen kann.
Da man keine Telemetriedaten senden muss, wenn die Telemetrieeinheit gerade zur Systemanlage an einen Rechner angeschlossen ist, wurde sich dafür entschieden, dass die Befehlshandler-Task im Normalbetrieb ausgeschaltet bleibt und mithilfe des Tasters auf der Mikrocontrollerplatine *UserButton* aktiviert und deaktiviert werden kann.

Der User-Button dient als Umschalter zwischen zwei sich gegenseitig ausschließenden Betriebsmodi, gesteuert über die globale Variable `led_state`: `defaultTask` entprellt den Button und liest ihn aus, schaltet `led_state` bei jedem Loslassen um, sichtbar durch die grüne leuchtende LED auf der Platine. `CommandHandler` wartet nur bei gesetztem `led_state` auf den Registrierungsbefehl, die drei Zustandsautomaten in der Task `TelemetryFsm` laufen nur, wenn `led_state` nicht gesetzt ist. Für die Umschaltung wird die Default-Task genutzt.

```
1   void StartDefaultTask(void *argument)
2   {
3     const uint32_t DEBOUNCE_MS = 30;
4     bool stable_pin = HAL_GPIO_ReadPin(UserButton_GPIO_Port,
          UserButton_Pin);
5     bool last_reading = stable_pin;
6     uint32_t last_change = HAL_GetTick();
7
8     for (;;)
9     {
10      bool pin = HAL_GPIO_ReadPin(UserButton_GPIO_Port,
            UserButton_Pin);
11
12      if (pin != last_reading)
13      {
14        last_reading = pin;
15        last_change = HAL_GetTick();
16      }
17      else if ((HAL_GetTick() - last_change) >= DEBOUNCE_MS &&
            pin != stable_pin)
18      {
19        if (stable_pin == PIN_STATE_LOW && pin ==
              PIN_STATE_HIGH)
20        {
21          HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
22          led_state = !led_state;
23        }
24        stable_pin = pin;
25      }
26      osDelay(1);
27    }
28  }
29
30  void start_command_handler(void *argument)
31  {
32    for(;;)
33    {
34      if (led_state)
35      {
36        check_for_device_id_request_command();
37      }
38      osDelay(1);
39    }
40  }
41
42  void start_network_fsm(void *argument)
43  {
44    for(;;)
45    {
46      if (!led_state)
47      {
48        gnss_fsm_step();
49        network_fsm_step();
50        telemetry_fsm_step();
51      }
52      osDelay(1000);
53    }
54  }
```

Listing 5.11: RTOS-Tasks mit tastergesteuertem Moduswechsel in freertos.c

5.0.2 IngestionService

IngestionService wurde direkt als .NET-*gRPC*-Projekt angelegt.

Methoden, die E/A-Operationen durchführen, wurden als `async`/`await` implementiert (siehe Abschnitt 2.1.5), um den Dienst performant zu halten.

Der Dienst wurde in folgender Struktur implementiert:

- `MQTT/` – MQTT-Client (Verbindungsaufbau, Reconnect)
- `Handlers/` – Verarbeitung eingehender Nachrichten
- `Assignments/` – Zuordnungs-Cache, gRPC-Client/-Server für Zuordnungen
- `Plausibility/` – GPS-Plausibilitätsprüfung (nicht im Lösungskonzept beschrieben, wird weiter unten beschrieben)
- `Trip/` – Zustandsautomat für laufende Fahrten, Retry-Queue für fehlgeschlagene Fahrtende-Meldungen
- `Influx/` – Schreiben der Telemetriedaten in InfluxDB
- `RabbitMq/` – Benachrichtigung für Live-Kartenaktualisierung
- `Models/` – Datenmodell Telemetrie-Event
- `Protos/` – Proto-Dateien
- `Program.cs` – Einstiegspunkt, registriert alle Hintergrunddienste und den gRPC-Endpunkt
- `Worker.cs` – Hintergrunddienst, der beim Start zunächst den Zuordnungs-Cache lädt und erst danach den MQTT-Client verbindet, damit eingehende Datenpunkte von Anfang an einer Zuordnung zugeordnet werden können

gRPC-Schnittstelle

Mit der gRPC-Schnittstelle, sprich den Proto-Dateien, wurde begonnen. Definiert wurden folgende Dateien:

- `trip.proto` – Fahrtanlage und -ende an *FleetControlApi*, jeweils einen RPC.

Übermittelt werden jeweils Geräte-ID und ein Zeitstempel, da in *FleetControlApi* eine Fahrt nur mit Anfang, Ende und Geräte-ID angelegt wird.

- `assignments.proto` – Abfrage aller aktuellen Zuordnungen beim Start von *FleetControlApi*, Zuordnungen von Fahrzeug, Kennzeichen und Fahrer sind optional, da die Telemetrieeinheit auch Daten senden kann, ohne dass Verbindungen im System angelegt wurden.
- `assignment_updates.proto` – Empfang einzelner Zuordnungsänderungen von *FleetControlApi* zur Laufzeit

```
1   service AssignmentService {
2     rpc GetCurrentAssignments (GetCurrentAssignmentsRequest)
          returns (GetCurrentAssignmentsResponse);
3   }
4
5   message Assignment {
6     string telemetry_unit_id = 1;
7     optional string vehicle_id = 2;
8     optional string license_plate = 3;
9     optional string driver_id = 4;
10  }
```

Listing 5.12: RPC-Definition und Nachrichtenformat in `assignments.proto`

Implementiert wurden die vier RPCs aus Lösungskonzept:

1. `TripService.StartTrip` – Meldung Fahrtbeginn
2. `TripService.EndTrip` – Meldung Fahrtende
3. `AssignmentService.GetCurrentAssignments` – Abfrage aller Zuordnungen im System
4. `AssignmentUpdateService.PushAssignmentUpdate` – empfängt Zuordnungsänderungen zur Laufzeit

Für jeden RPC gibt es eine Klasse, die den generierten gRPC-Client-Code kapselt (`TripClient`, `AssignmentClient`) oder die generierte Server-Basisklasse implementiert (`AssignmentUpdateGrpcService`) und die empfangenen Zuordnungsänderungen an den `AssignmentCache` weiterreicht.

Nach der gRPC-Schnittstelle als Anbindung zu *FleetControlApi* wurden die Bearbeitungsbereiche implementiert.

MQTT-Eingang

Für den MQTT-Client kommt die Bibliothek *MQTTnet* zum Einsatz.

In `MqttClientService` werden drei Methoden implementiert:

1. `StartAsync()`: Registriert die Callbacks für eingehende Nachrichten und Verbindungsabbrüche und baut danach die Erstverbindung auf.
2. `ConnectWithRetryAsync()`: Baut beim Start über eine Retry-Schleife die Verbindung zum konfigurierten Broker auf: Schlägt ein Verbindungsversuch fehl, wird er nach 5 s erneut versucht, bis er gelingt. Zugangsdaten werden optional aus der Konfiguration übernommen, je nachdem, ob sie dort hinterlegt sind.

   Bricht die Verbindung während des Betriebs ab, stößt der `DisconnectedAsync`-Handler dieselbe Retry-Schleife erneut an und abonniert anschließend die Topic neu. Eine `SemaphoreSlim` verhindert dabei, dass mehrere gleichzeitig eintreffende Disconnect-Events parallel jeweils eigene Reconnect-Versuche starten.
3. `HandleMessage()`: Jede eingehende Nachricht wird über das Ereignis `ApplicationMessageReceivedAsync` entgegengenommen und direkt an `MessageHandler.HandleAsync` weitergereicht, der die eigentliche Verarbeitung übernimmt.

```
1     private async Task ConnectWithRetryAsync(
2     MqttClientOptions options, CancellationToken token
3     ) {
4       while (true)
5       {
6         try
7         {
8           await _client!.ConnectAsync(options, token);
9           return;
10        }
11        catch (Exception ex) when (!token.IsCancellationRequested)
12        {
13          await Task.Delay(ReconnectDelay, token);
14        }
15      }
16    }
```

Listing 5.13: Retry-Schleife für den MQTT-Verbindungsaufbau in MqttClientService.cs

Nachrichtenverarbeitung

Eine Methode `MessageHandler.HandleAsync` verarbeitet eingehende Nachrichten in folgenden Schritten:

1. JSON-Payload wird in ein `TelemetryEvent` deserialisiert, fehlt dabei die Geräte-ID oder lässt sich die Nachricht nicht einlesen, wird sie verworfen.
2. Geräte-ID wird gegen den Zuordnungs-Cache geprüft. Ist sie dort nicht bekannt, wird die Nachricht ebenfalls verworfen. Somit können keine Daten für ein nicht registriertes Gerät gespeichert werden, die Datenqualität wird so bewahrt.
3. Weiterleitung nach Plausibilitätsprüfung: Falls kein Fehler festgestellt wird, wird ein Datenpunkt an den Zustandsautomat (`TripReactor`) übergeben und zusammen mit dem erkannten Fahrtzustand und der aktuellen Zuordnung (Fahrer, Fahrzeug, Telemetrieeinheit) an `InfluxWriter` zur Persistierung weitergereicht.

Plausibilitätsprüfung `GpsPlausibilityFilter` berechnet aus der zuletzt akzeptierten Position und der verstrichenen Zeit über die Haversine-Formel die implizite Geschwindigkeit zum neuen Punkt. Übersteigt sie 300 km/h, gilt der Punkt als unplausibel und wird verworfen. Um zu verhindern, dass ein einzelner fehlerhafter Referenzpunkt dauerhaft alle folgenden Punkte blockiert, wird nach drei aufeinanderfolgend verworfenen Punkten für dasselbe Gerät der nächste Punkt unabhängig vom Ergebnis als neue Referenz übernommen.

```
1       var distanceKm = HaversineDistanceKm(last.Lat, last.Lon, lat,
            lon);
2       var impliedSpeedKmh = distanceKm / elapsed.TotalHours;
3
4       if (impliedSpeedKmh > MaxPlausibleSpeedKmh)
5       {
6         var rejections = _consecutiveRejections.AddOrUpdate(
7         deviceId, 1, (_, count) => count + 1);
8
9         if (rejections >= MaxConsecutiveRejections)
10        {
11          Accept(deviceId, lat, lon, timestamp);
12          return true;
13        }
14
15        return false;
16      }
17
18      Accept(deviceId, lat, lon, timestamp);
19      return true;
```

Listing 5.14: Kernprüfung in GpsPlausibilityFilter.IsPlausible

Im ursprünglichen Konzept war dieser Filter nicht vorgesehen und wurde nachträglich eingeführt, um sporadisch auftretende, ungültige GPS-Daten von der Hardware zu kompensieren (siehe Abschnitt 5.1.5).

Zustandsautomat

`TripReactor` hält für jedes aktive Gerät eine `DeviceTripState`-Instanz in einem `ConcurrentDictionary` und verarbeitet sowohl eingehende Datenpunkte (`DispatchAsync`) als auch einen periodischen Timeout-Check (alle 1 s) unter demselben Lock, da beide denselben Zustand verändern können.

Die im Lösungskonzept nur allgemein beschriebenen Schwellwerte sind konkret:

- Fahrtbeginn: 3 aufeinanderfolgende Datenpunkte über 5 km/h. Dies ist notwendig, da auch im Stand eine kleine Abweichung von ca. ± 3 km/h im Stillstand vorlag (siehe Abschnitt 5.1.5).
- Fahrtende: 300 aufeinanderfolgende Stillstands-Punkte, was bei einem Datenpunkt pro Sekunde den im Lösungskonzept genannten 5 Minuten entspricht
- Verbindung verloren: 2 s ohne neue Daten während einer laufenden Fahrt; bleibt die Verbindung 5 Minuten unterbrochen, wird die Fahrt mit dem letzten

bekannten Zeitstempel zwangsweise beendet

```
1     case TripState.Connected:
2     if (data.SpeedKmh > DrivingSpeedThreshold)
3     {
4       state.DrivingCandidateCount++;
5
6       if (state.DrivingCandidateCount >=
            DrivingCandidateThreshold)
7       {
8         state.State = TripState.Driving;
9         state.TripStartTimestamp = data.Timestamp;
10        state.DrivingCandidateCount = 0;
11        beginTrip = true;
12      }
13    }
14    else
15    {
16      state.DrivingCandidateCount = 0;
17    }
18    break;
```

Listing 5.15: Erkennung des Fahrtbeginns in TripReactor.DispatchAsync

Schlägt eine `EndTrip`-Meldung an *FleetControlApi* fehl, wird sie in der `PendingTripEndQueue` vorgemerkt. `PendingTripEndRetryService` versucht sie alle 30 s erneut zuzustellen, bis die Meldung erfolgreich war.

Persistenz und Benachrichtigungen

Verarbeitete Datenpunkte werden nicht synchron in `MessageHandler` gespeichert, sondern an `InfluxWriter` zur Persistierung übergeben, der sie in einer Warteschlange zwischenspeichert. Der als `BackgroundService` laufende `InfluxWriteWorker` liest die gespeicherten Datenpunkte aus der Schlange unabhängig mit einer Funktion `ReadAllAsync` aus und speichert sie in InfluxDB. So blockiert die Schreiblatenz von InfluxDB nie den MQTT-Verarbeitungspfad.

Jeder Datenpunkt wird als Messung namens `sensor` mit den Tags `deviceId`, `topic` und `state` (dem vom Zustandsautomaten erkannten Fahrzustand) sowie den Feldern `lat`, `lon`, `speedKmh` und `accelMs2` gespeichert. Die optionalen Tags `vehicleId`, `licensePlate` und `driverId` werden nur gesetzt, wenn entsprechende Verknüpfungen in der Anwendung angelegt wurden (bekannt durch

`AssignmentCache`, der vor einer Fahranlage über Abfrage bei *FleetControlApi* aktualisiert wird). Falls eine Zuordnung fehlt, fällt der zugehörige Tag weg.

```
1     var point = PointData
2     .Measurement("sensor")
3     .Tag("deviceId", item.Data.DeviceId)
4     .Tag("topic", item.Topic)
5     .Tag("state", item.State.ToString());
6
7     if (item.Assignment.HasVehicleId)
8     point = point.Tag("vehicleId", item.Assignment.VehicleId);
9
10    if (item.Assignment.HasLicensePlate)
11    point = point.Tag("licensePlate",
          item.Assignment.LicensePlate);
12
13    if (item.Assignment.HasDriverId)
14    point = point.Tag("driverId", item.Assignment.DriverId);
15
16    point = point
17    .Field("lat", item.Data.Latitude)
18    .Field("lon", item.Data.Longitude)
19    .Field("speedKmh", item.Data.SpeedKmh)
20    .Field("accelMs2", item.Data.AccelerationMs2)
21    .Timestamp(item.Data.Timestamp, WritePrecision.Ns);
```

Listing 5.16: Anreichern der Datenpunkte mit Informationen zur Zuordnung in InfluxWriteWorker.cs vor der Persistierung

Nach erfolgreichem Schreiben veröffentlicht `InfluxWriteWorker` denselben Datenpunkt zusätzlich als `TelemetryEventMessage` über `RabbitMqPublisher` unter dem Routing-Key `telemetry.{deviceId}`. `RabbitMqPublisher` baut die Verbindung erst beim ersten Aufruf auf, schlägt eine Veröffentlichung fehl, wird die Verbindung verworfen und beim nächsten Datenpunkt neu aufgebaut.

Zuordnungssynchronisation

`AssignmentCache` hält die aktuellen Telemetrieeinheit-Fahrzeug-Fahrer-Zuordnungen in einem `ConcurrentDictionary` im Speicher.

Beim Start lädt `AssignmentCache.LoadAsync` die Zuordnungen einmalig über `AssignmentClient` von *FleetControlApi*. Schlägt der Aufruf fehl, wird er alle 5 s wiederholt, bis er gelingt.

Änderungen während des Betriebs treffen über `AssignmentUpdateGrpcService` ein: *FleetControlApi* teilt jede geänderte Zuordnung sofort per gRPC mit, statt dass *IngestionService* periodisch nachfragen muss.

5.0.3 DriveDataService

Der Abfragedienst *DriveDataService* wurde als *gRPC*-.NET-Projekt angelegt, sodass alle Abhängigkeiten für *gRPC* zur Codegenerierung und Kompilierung bereits vorhanden waren. Es ergibt sich folgende Projektstruktur:

- `Protos/` – Proto-Dateien mit den Service- und Nachrichtendefinitionen
- `Services/` – Implementierung der aus den Protos generierten Service-Basisklassen
- `Program.cs` – Einstiegspunkt, registriert die gRPC-Services

Zuerst wurden die vier RPCs und Nachrichtenformat in der Proto-Datei `telemetry.proto` definiert.

1. `GetTelemetryPoints` (Abfrage aller Messpunkte eines Geräts in einem Zeitfenster)
2. `GetLatestPoints` (Abfrage des jeweils aktuellsten Punkts je Gerät)
3. `GetLatestPointsByVehicle` (Abfrage des jeweils aktuellsten Punkts je Fahrzeug)
4. `DeleteTelemetryPoints` (Löschen der Punkte einer gelöschten Fahrt.

Als gemeinsames Nachrichtenformat dient `TelemetryPoint` mit den Feldern `timestamp`, `lat`, `lon`, `speed_kmh`, `accel_ms2`, `driver_id` und `device_id`.

```
1     service TelemetryQuery {
2       rpc GetTelemetryPoints (GetTelemetryPointsRequest) returns
            (GetTelemetryPointsResponse);
3
4       rpc GetLatestPoints (GetLatestPointsRequest) returns
            (GetLatestPointsResponse);
5
6       rpc GetLatestPointsByVehicle
            (GetLatestPointsByVehicleRequest) returns
            (GetLatestPointsByVehicleResponse);
7
8       rpc DeleteTelemetryPoints (DeleteTelemetryPointsRequest)
           returns (DeleteTelemetryPointsResponse);
9     }
```

Listing 5.17: Service-Definition der RPCs in `telemetry.proto`

Auf Basis dieser Proto-Definition erzeugt der *gRPC*-Compiler beim Kompilieren automatisch die passenden C#-Basisklassen sowie die Datentypen für die Anfragen und Antworten.

`TelemetryQueryService` leitet von der generierten Basisklasse `TelemetryQueryBase` ab und überschreibt die im Proto deklarierten RPC-Methoden. Innerhalb dieser Methoden werden die Anfragen in entsprechende *Flux*-Queries für *InfluxDB* übersetzt und die zurückgelieferten Datenpunkte nach der Ausführung in die generierten Response-Objekte überführt.

```
1     public class TelemetryQueryService :
          TelemetryQuery.TelemetryQueryBase
2     {
3       public override async Task<GetTelemetryPointsResponse>
            GetTelemetryPoints(
4       GetTelemetryPointsRequest request,
5       ServerCallContext context
6       ) {
7         var response = new GetTelemetryPointsResponse();
8         var start = request.Start.ToDateTime();
9         var end = request.End.ToDateTime();
10
11        var flux =
12        /* Flux-Query als Zeichenkette */
13
14        var tables = await _client.GetQueryApi().QueryAsync(flux,
              _org);
15        // ... Mapping der Ergebnisse auf TelemetryPoint-Objekte
16        return response;
17      }
18
19      // weitere RPCs ...
20    }
```

Listing 5.18: Implementierung des RPCs `GetTelemetryPoints` in `TelemetryQueryService.cs`

Abschließend wurde der Service in `Program.cs` eingebunden und ist damit für Clients wie *FleetControlApi* erreichbar.

5.0.4 FleetControlApi

Die vier Schichten wurden als eigenständige .NET-Projekte innerhalb einer gemeinsamen *Solution* (.NET-Oberprojekt) umgesetzt, wodurch die Abhängigkeitsrichtung bereits durch die Projektreferenzen des Build-Systems erzwungen wird und nicht nur durch Konvention eingehalten werden muss:

- `FleetControlServer.Api` – REST-Controller, gRPC-Services, SignalR-Hub, Program.cs (Einstiegspunkt)
- `FleetControlServer.Service` – Geschäftslogik, DTOs, Clients für IngestionService/DriveDataService
- `FleetControlServer.Data` – EF-Core-Kontext, Repositories, Migrationen

- `FleetControlServer.Domain` – Entitätsklassen

Die Api-Schicht referenziert Service, diese wiederum Data und Domain – ein Referenzieren in umgekehrter Richtung ist durch das Projektsystem gar nicht erst möglich. Die Methoden, die E/A-Operationen durchführen (Datenbankzugriffe, gRPC-Aufrufe, HTTP-Anfragen), wurden als `async`/`await` implementiert (siehe Abschnitt 2.1.5), um Threads während des Wartens auf externe Antworten nicht zu blockieren und den Dienst performant zu halten.

REST-Schnittstelle

Für jede Entität existiert ein ASP.NET-Rest-Controller, der als solcher mit `[ApiController]` / `[Route("api/[...]")]` annotiert wird. Die Annotation markiert die Klasse als HTTP-Endpunkt im ASP.NET-Projekt.

Die Controller-Klassen enthalten keine Logik, sie reichen die Anfrage nur an die zugehörige Service-Klasse weiter und wandeln deren Rückgabewert in einen passenden HTTP-Statuscode um. Da einige Fehlerfälle (z. B. Validierungsfehler beim Anlegen eines Fahrzeugs) erwartbar und kein Ausnahmefall sind, geben die Service-Methoden statt Ausnahmen ein Ergebnisobjekt mit `Success`/`Error`-Feldern zurück, das der Controller auswertet.

```
[ApiController]
[Route("api/[controller]")]
public class VehicleController : ControllerBase
{
  private readonly VehicleService _service;

  public VehicleController(VehicleService service)
  {
    _service = service;
  }

  [HttpPost]
  public async Task<IActionResult> Save([FromBody] VehicleDto dto)
  {
    var result = await _service.CreateAsync(dto);
    if (!result.Success)
    {
      return BadRequest(result.Error);
    }
    return Ok(result.Vehicle);
  }
}
```

Listing 5.19: Auszug aus VehicleController.cs: Controller-Muster und Routendefinition

gRPC-Schnittstellen

Analog zu *DriveDataService* in diesem Kapitel wurden zunächst die Proto-Dateien `assignments.proto` und `trip.proto` mit den Service- und Nachrichtendefinitionen angelegt, aus denen der gRPC-Compiler die C#-Basisklassen `AssignmentServiceBase` und `TripServiceBase` generiert. Die Klassen `AssignmentGrpcService` und `TripGrpcService` leiten davon ab und überschreiben die RPC-Methoden.

In `assignments.proto` wird folgender RPC definiert:

- `GetCurrentAssignments` – Abfrage der aktuellen Fahrer-Telemetrieeinheit-Fahrzeug-Zuordnungen

```
service AssignmentService {
  rpc GetCurrentAssignments (GetCurrentAssignmentsRequest)
      returns (GetCurrentAssignmentsResponse);
}
```

Listing 5.20: Service-Definition in `assignments.proto`

In `trip.proto` werden folgende RPCs definiert:

- `StartTrip` – meldet den Beginn einer Fahrt für eine Telemetrieeinheit mit Startzeitpunkt
- `EndTrip` – meldet das Ende einer Fahrt; enthält optional den ursprünglichen Startzeitpunkt, falls der vorherige `StartTrip`-Aufruf fehlgeschlagen ist und die Fahrt nachträglich mit Start- und Endzeitpunkt angelegt werden muss

```
service TripService {
  rpc StartTrip (StartTripRequest) returns (StartTripResponse);
  rpc EndTrip (EndTripRequest) returns (EndTripResponse);
}
```

Listing 5.21: Service-Definition in `trip.proto`

`TripGrpcService` implementiert `StartTrip`/`EndTrip`, über die *IngestionService* Fahrtbeginn bzw. -ende meldet. Zusätzlich zur eigentlichen RPC-Logik löst die Methode direkt eine SignalR-Benachrichtigung (`TripStarted`/`TripEnded`) an den `VehicleHub` aus, damit die Fahrten-Ansicht der Benutzeroberfläche sofort aktualisiert wird, statt auf den nächsten Telemetrie-Datenpunkt warten zu müssen (siehe Abschnitt 5.0.4, Echtzeit-Kommunikation).

Für die ausgehenden gRPC-Verbindungen wurden vorab die Protodateien geschrieben, der generierte Code wird in der Service-Schicht gekapselt und verwendet.

```
    service AssignmentUpdateService {
      rpc PushAssignmentUpdate (AssignmentUpdate) returns
          (PushAssignmentUpdateResponse);
    }

    // Definitionen Nachrichtenformat ...
```

Listing 5.22: Service-Definition in `asssignments_updates.proto`

```
1   service TelemetryQuery {
2     rpc GetTelemetryPoints (GetTelemetryPointsRequest) returns
          (GetTelemetryPointsResponse);
3
4     rpc GetLatestPoints (GetLatestPointsRequest) returns
          (GetLatestPointsResponse);
5
6     rpc GetLatestPointsByVehicle
          (GetLatestPointsByVehicleRequest) returns
          (GetLatestPointsByVehicleResponse);
7
8     rpc DeleteTelemetryPoints (DeleteTelemetryPointsRequest)
          returns (DeleteTelemetryPointsResponse);
9   }
10
11    // Definitionen Nachrichtenformat ...
```

Listing 5.23: Service-Definition für Abfrage von *DriveDataService* in `telemetry.proto`

Service-Klassen

Die Service-Schicht enthält die Geschäftslogik von *FleetControlApi*. Für jede Entität existiert eine Service-Klasse (`VehicleService`, `PersonService`, `TripService`, `TelemetryUnitService`), die auf die Repository-Schnittstellen der Data-Schicht zugreift, entitätsübergreifende Validierung durchführt und wie schon die Controller ein Ergebnisobjekt (`Success`, `Error`) statt einer Exception zurückgibt.

Zwei weitere Klassen kapseln die gRPC-Clients zu anderen Diensten:

- `IngestionServiceClient` – pusht Änderungen an Zuordnungen an *IngestionService*
- `TelemetryQueryClient` – fragt Telemetriedaten bei *DriveDataService* ab

Am Beispiel von `VehicleService` wird der typische Aufbau der Service-Klassen veranschaulicht: Jede Methode delegiert nach ihrer Validierung an genau eine Repository- bzw. Client-Methode.

```
1  public class VehicleService
2  {
3      // Repo Referenzen ...
4
5      public VehicleService(/* ... */)
6      {
7          // ...
8      }
9
10     public async Task<(bool Success, Vehicle? Vehicle, string?
           Error)> CreateAsync(VehicleDto dto)
11     {
12         // ...
13         var existing = await _repo.GetByIdAsync(id);
14         // ...
15         return await UpsertAsync(id, dto);
16     }
17
18     // ..
19 }
```

Listing 5.24: Unvollständige Service-Klasse `VehicleService`

Persistenz

Für die Persistenz der relationalen Daten wird, wie im Entwurfskapitel beschrieben, EF-Core verwendet. Konfigurationen des Datenbankschemas finden hauptsächlich in einer eigenen Konfigurationsklasse (`AppDbContext`) statt, die ihre Fähigkeiten von der EF-Core-Klasse `DbContext` erbt.

Der Datenbank-Connection-String wird nicht im Code hinterlegt, sondern über die Konfiguration injiziert und beim Erstellen des `AppDbContext` per Dependency Injection übergeben: Er liegt unter dem Schlüssel `ConnectionStrings:DefaultConnection` in `appsettings.Development.json` (Standardkonfigurationsdatei eines .NET-Projekts).

In `AppDbContext` werden die Datenbanktabellen definiert bzw. über die Fluent API Details des Schemas konfiguriert, die sich nicht allein durch Attribute an den Entitätsklassen der Domain-Schicht ausdrücken lassen.

Folgende Tabellen wurden definiert:

- `TelemetryUnits`
- `Vehicles`

- `VehicleDrivers`
- `DriversLicenses`
- `Trips`

Um die Qualität der Datenbasis zu schützen, wurden folgende Beschränkungen in `AppDbContext` konfiguriert:

- Die FIN (`VehicleIdentificationNumber`) sowie das im `Trip` gespeicherte Fahrzeug-Snapshot-Feld sind als feste Zeichenkette (`character(17)`) statt variabler Länge (`varchar`) hinterlegt, da eine FIN immer exakt 17 Zeichen lang ist.
- Kennzeichen (`LicensePlateNumber`) und Erstzulassungsdatum (`FirstRegistration`) eines Fahrzeugs sind jeweils über einen eindeutigen Index als unique markiert.
- Das Löschen eines Fahrzeugs entkoppelt lediglich die zugehörige Telemetrieeinheit bzw. den zugewiesenen Fahrer (`OnDelete(DeleteBehavior.SetNull)`), statt sie mitzulöschen.
- Eine Telemetrieeinheit, die einer Fahrt zugeordnet ist, kann nicht gelöscht werden (`OnDelete(DeleteBehavior.Restrict)`), damit bestehende Fahrten nicht referenzlos werden.
- Führerscheine werden beim Löschen des zugehörigen Fahrers automatisch mitgelöscht (`OnDelete(DeleteBehavior.Cascade)`).
- Der Führerscheintyp (`LicenseType`) wird als lesbarer String statt als numerischer Enum-Index gespeichert, damit die Spalte unabhängig von der Deklarationsreihenfolge des Enums bleibt.

```
1  public class AppDbContext : DbContext
2  {
3    // Connection string über Dependency Injection
4    public AppDbContext(DbContextOptions<AppDbContext> options)
5    : base(options)
6    {
7    }
8
9    // Datenbanktabellen
10   public DbSet<TelemetryUnit> TelemetryUnits { get; set; }
11   public DbSet<VehicleDriver> VehicleDrivers { get; set; }
12   public DbSet<DriversLicense> DriversLicenses { get; set; }
13   public DbSet<Vehicle> Vehicles { get; set; }
14   public DbSet<Trip> Trips { get; set; }
15
16
17   protected override void OnModelCreating(ModelBuilder
         modelBuilder)
18   {
19     base.OnModelCreating(modelBuilder);
20
21     modelBuilder.Entity<Vehicle>()
22     .Property(v => v.VehicleIdentificationNumber)
23     .HasColumnType("character(17)");
24
25     // weitere Beschränkungen ...
26   }
27 }
```

Listing 5.25: Schemadefinition und Konfiguration in `AppDbContext`

Ergänzend sind grundlegende Feldbeschränkungen (Pflichtfelder, maximale Zeichenlängen, Wahl der Primärschlüssel) bereits als Datenannotationen direkt an den Domain-Entitätsklassen hinterlegt, wie im Lösungskonzept-Kapitel im Abschnitt 4.3.4 bereits begründet.

Für jeden Aufgabenbereich existiert ein Interface-/Implementierungs-Paar nach dem *Repository*-Muster, das den direkten Zugriff auf `AppDbContext` kapselt:

- `IVehicleRepository` / `VehicleRepository`
- `IVehicleDriverRepository` / `VehicleDriverRepository`
- `ITelemetryUnitRepository` / `TelemetryUnitRepository`
- `ITripRepository` / `TripRepository`

Die Service-Schicht kennt nur das jeweilige Interface, wodurch die konkrete EF-Core-Abhängigkeit dort nicht sichtbar ist.

Domain-Klassen

Die Domain-Klassen (`Vehicle`, `VehicleDriver`, `TelemetryUnit`, `Trip`, `DriversLicense`) wurden gemäß dem im Lösungskonzept beschriebenen Entwurf umgesetzt.

Live-Aktualisierungen mit RabbitMQ / SignalR

Die Klasse `VehicleHub` kapselt lediglich die SignalR-Bibliotheksklasse `Hub`, die den Sendekanal darstellt, und braucht keine zusätzlichen Funktionen.

`RabbitMqTelemetryConsumer` wird als .NET-`BackgroundService` implementiert und läuft damit im Hintergrund. Die Klasse verbindet sich mit dem von *IngestionService* verwendeten RabbitMQ-Dienst und leitet Bewegungs-Updates als `VehicleUpdate`-Nachricht an alle Clients weiter. Die Klasse nutzt dafür eine Instanz von `VehicleHub`.

Die Clients sind dabei nicht der Klasse selbst bekannt, sondern werden zentral von SignalR automatisch verwaltet. Über die injizierte Klasse `IHubContext<VehicleHub>` erhält `RabbitMqTelemetryConsumer` Zugriff auf eine Verbindungsliste, um Nachrichten an alle aktuell verbundenen Clients zu senden.

```
1  public class RabbitMqTelemetryConsumer : BackgroundService
2  {
3    // ...
4    private readonly IHubContext<VehicleHub> _hub;
5    // ...
6
7    protected override async Task ExecuteAsync (
8      CancellationToken stoppingToken
9    ) {
10     // ...
11   }
12
13   private async Task
         ConsumeUntilDisconnectedAsync(CancellationToken
         stoppingToken)
14   {
15     // ... Verbindung zu RabbitMQ
16     // ... Konsumieren von Nachrichten
17   }
18 }
```

Listing 5.26: Implementierung `RabbitMqTelemetryConsumer` als Background Service

5.0.5 FleetControlUI

Das auf Angular basierende FleetControlUI wurde in folgender grundlegender Struktur in `src/app` umgesetzt:

- **models/** – Datenmodelle für Fahrzeug, Person, Telemetrieeinheit und Fahrt
- **pages/** – Hauptseiten, hier nur `main-page` als zentrale Navigationsseite
- **services/** – Klassen für HTTP-basierte Backend-Aufrufe und Verwaltungsoperationen.
- **widgets/** – Ansichts- und Unterkomponenten
- **styles.scss** – Datei für die Definition von SCSS-Variablen für ein dunkles und helles Oberflächendesign

Allgemeines

Die vier Ansichten aus dem Lösungskonzept sind als eigenständige (*standalone*) Angular-Komponenten umgesetzt: `VehiclesPageComponent`, `TripsPageComponent`, `PersonPageComponent` und `TelemetryUnitPageComponent`, jeweils im gleichnamigen Unterordner von `widgets/`. Jede Angular *standalone*-Komponente besteht aus TypeScript-Datei (Verhalten), SCSS-Datei (Styling) und HTML-Datei (Seitenstruktur). Die zentrale Hauptseite `MainPageComponent` bindet alle vier Ansichten ein und schaltet über eine Variable `activeTab` zwischen ihnen um.

Wie im Lösungskonzept beschrieben, wurde die Navigation reiterbasiert umgesetzt. Ein Klick auf einen Reiter wechselt die Ansicht, ohne dass die Seite neu geladen wird (SPA).

```
1    <nav class="tabs" *ngIf="!tabsCollapsed">
2      <button
3      *ngFor="let tab of tabs"
4        (click)="selectTab(tab)"
5        [class.active]="activeTab === tab">
6        {{ tab }}
7      </button>
8    </nav>
```

Listing 5.27: Reiter-Auswahl im HTML-Template von MainPageComponent

```
1    tabs = [
2      'Fahrzeuge', 'Fahrten', 'Personen',
         'Datenerfassungseinheiten'
3    ];
4    activeTab = 'Fahrzeuge';
5
6    selectTab(tab: string) {
7      this.activeTab = tab;
8    }
```

Listing 5.28: Definition der vier Reiter in der Klasse MainPageComponent

Aus Gründen der *UserExperience* wurde sowohl eine dunkle als auch eine helle Farbversion der Oberfläche implementiert, gesteuert durch eine Schaltfläche in der Kopfleiste. Standard ist das dunkle Design aufgrund persönlicher Präferenz.
Dazu wurden SCSS-Variablen für zwei unterschiedliche Farbschemata in einer Datei

styles.scss angelegt.

```
:root {
  --color-bg-page: #101014;
  ... weitere Variablendefinitionen
}

:root.theme-light {
  --color-bg-page: #c9c7c4;
  ... weitere Variablendefinitionen
}
```

Listing 5.29: Theme-Variablen in styles.scss (Auszug)

```
.tabs button {
  color: var(--color-text-soft);
  transition: color 0.2s;
}

.tabs button.active {
  color: var(--color-text-primary);
}
```

Listing 5.30: Verwendung der Variablen

HTTP-Kommunikation

Für jeden Aufgabenbereich existiert ein eigener injizierbarer Service (Angabe im Konstruktor, bereitgestellt durch Angulars Dependency Injection). Umgesetzt wurden:

1. `VehicleService`
2. `PersonService`
3. `TelemetryUnitService`
4. `TripService`

Diese Komponenten kapseln die Kommunikation mit *FleetControlApi*. Jede Service-Komponente übersetzt zwischen dem vom Backend gelieferten Antwort-Objekt und

einem eigenen, auf die Verwendung in der Benutzeroberfläche zugeschnittenen Modellformat (reine Datenklasse). Fehler werden zentral über einen `catchError`-Operator abgefangen, um sie weiterverarbeiten zu können.

```
loadAll(): Observable<Vehicle[]> {
  return
      this.http.get<VehicleResponseDto[]>(`${this.baseUrl}/Vehicle`)
    .pipe(
      map(list => list.map(dto => this.toVehicle(dto))),
      catchError(err => this.rethrow(err))
  );
}
```

Listing 5.31: Alle Fahrzeuge laden in VehicleService.ts

Für asynchrone HTTP-Aufrufe wurde anstelle von `async/await` das reaktive Observable-Muster aus RxJS verwendet, damit der Aufruf die Anwendung nicht blockiert und graphische Elemente beim Eintreffen der Antwortdaten selbstständig neu aufgebaut werden.

Live-Updates

`VehicleLiveService` verbindet sich beim Anwendungsstart per SignalR-Client mit dem `VehicleHub` von *FleetControlApi* (Endpunkt `/hubs/vehicles`). Im Gegensatz zum backend-seitigen `RabbitMqTelemetryConsumer`, der die Reconnect-Logik manuell implementieren musste, übernimmt hier die SignalR-Clientbibliothek automatisches Wiederverbinden über `withAutomaticReconnect()`.

```
this.connection = new signalR.HubConnectionBuilder()
  .withUrl(hubUrl)
  .withAutomaticReconnect()
  .build();

this.connection.on('VehicleUpdate', (update:
    VehiclePositionUpdate) =>
{
  this.vehicleUpdateSubject.next(update);
});

this.connection.start().catch(() => {});
```

Listing 5.32: Verbindungsaufbau in VehicleLiveService

USB-Anbindung

Für die Verbindung mit einer per USB angeschlossenen Telemetrieeinheit kommt die *Web Serial API* (`navigator.serial`) zum Einsatz.

Für ein neues, dem Browser noch nicht bekanntes Gerät fordert die Funktion `requestDevice()` über `navigator.serial.requestPort()` die Nutzer-Berechtigung an. Bereits erlaubte Geräte werden beim Start über `getPorts()` automatisch wieder erkannt.

Zur Identifikation eines Geräts wird der Befehl `get_device_id` über den Port geschrieben und auf eine Antwortzeile im UUID-Format gewartet.

Filterbereiche

Anders als im Lösungskonzept als eine generische, über Feldkonfiguration steuerbare Komponente geplant, wurde die Filterung mit fest eincodierten, entitätsspezifischen Kriterien implementiert.

`FilterSidebarComponent` setzt die Filterlogik für Fahrzeuge, Telemetrieeinheiten und Personen in einer einzigen Komponente um und wird in den drei Komponenten für Fahrzeuge, Personen und Telemetrieeinheiten eingebunden. Sie erhält die verfügbaren Fahrzeuge, Telemetrieeinheiten und Personen jeweils über `@Input`s. Die aktuell aktiven Filter werden über `@Output filtersApplied` als `AppliedFilters`-Objekt an die einbettende Seite zurückgemeldet.

Die Fahrten-Seite verwendet `FilterSidebarComponent` nicht, sondern enthält

eine eigene, unabhängig implementierte Filterlogik direkt im Template (Datumsbereich sowie ein zum Fahrzeug-Filter der `FilterSidebarComponent` strukturell ähnlicher, aber separat implementierter Fahrzeug-Filter). Eine gemeinsame, wiederverwendbare Filterkomponente für alle vier Ansichten wurde damit nicht implementiert, so wie es ursprünglich geplant war.

Kartendarstellung

Beide Kartenansichten (`VehicleMapComponent` für Fahrzeugpositionen, `CardWidgetComponent` für Fahrtrouten) nutzen die Kartenbibliothek *Leaflet* mit dunklen Kartenkacheln von *Stadia Maps*.
Bewegte Fahrzeuge bzw. noch laufende Fahrten werden durch einen pulsierenden Marker hervorgehoben. Fahrtrouten erhalten je eine eigene Farbe sowie einen markierten Start- und Zielpunkt, damit man sie unterscheiden kann. Beide Karten wurden gemäß dem Lösungskonzept konfiguriert.

```
1    if (isOngoing && isLastPoint) {
2      const icon = L.divIcon({
3        className: 'trip-marker-icon',
4        html: `<div class="trip-marker">`
5        + `<span class="trip-marker-pulse"
             style="background-color:${color}"></span>`
6        + `<span class="trip-marker-dot"
             style="background-color:${color}"></span>`
7        + `</div>`,
8        iconSize: [16, 16],
9        iconAnchor: [8, 8]
10     });
11     L.marker(coordinates[index], { icon
         }).addTo(this.routeLayer);
12     return;
13   }
14
15   L.circleMarker(coordinates[0], {
16     radius: 12, color, fillColor: color, fillOpacity: 1
17   }).addTo(this.routeLayer); // Start
18
19   if (!isOngoing) {
20     L.circleMarker(coordinates[coordinates.length - 1], {
21       radius: 14, color, fillOpacity: 0, weight: 3
22     }).addTo(this.routeLayer); // Ziel
23   }
```

Listing 5.33: Start-, Ziel- und Pulsier-Marker (CardWidgetComponent)

Graphdarstellung

Die Graphdarstellung einer Fahrt ist als `TripChartComponent` mit der Bibliothek *ECharts* umgesetzt. Geschwindigkeit und Beschleunigung werden als zwei Liniendiagramme über einer gemeinsamen Zeitachse mit jeweils eigener Y-Achse (km/h bzw. m/s²) dargestellt, sodass beide Größen trotz unterschiedlicher Wertebereiche gemeinsam ablesbar sind. Ein Zoom-Schieberegler (`dataZoom`) erlaubt es, einzelne Abschnitte einer Fahrt vergrößert zu betrachten, um Brems- und Beschleunigungsphasen genauer zu untersuchen.

```
1     yAxis: [
2       { type: 'value', name: 'km/h', position: 'left' },
3       { type: 'value', name: 'm/²s', position: 'right' }
4     ],
5     series: [
6       { name: 'Geschwindigkeit (km/h)', type: 'line', data:
          speed, yAxisIndex: 0 },
7       { name: 'Beschleunigung (m/²s)', type: 'line', data: accel,
          yAxisIndex: 1 }
8     ]
```

Listing 5.34: Achsenkonfiguration für Geschwindigkeit und Beschleunigung in TripChartComponent

Im Gegensatz zur Kartendarstellung, die im TypeScript-Code aufgebaut wird, wird das Liniendiagramm rein deklarativ im HTML eingebunden – die gesamte Datei besteht aus dieser einen Zeile, sämtliche Konfiguration steckt in der gebundenen `chartOptions`-Variablen.

```
1     <div echarts [options]="chartOptions" class="chart"></div>
```

Listing 5.35: Einbindung des Liniendiagramms in trip-chart.component.html

5.0.6 RabbitMQ Eventbus, InfluxDB, PostgreSQL, Mosquitto(MQTT)

Der Eventbus, der MQTT-Broker sowie die beiden Datenbanken wurden als *Docker-Image* verwendet und mit *Docker-Compose*-Dateien konfiguriert.

```
services:
  rabbitmq:
    image: rabbitmq:4-management
    container_name: rabbitmq
    hostname: rabbitmq
    restart: unless-stopped

    ports:
    - "5672:5672"
    - "15672:15672"

    environment:
    RABBITMQ_DEFAULT_USER: ...
    RABBITMQ_DEFAULT_PASS: ...

    volumes:
    - rabbitmq_data:/var/lib/rabbitmq

volumes:
  rabbitmq_data:
```

Listing 5.36: Docker-Compose-Konfiguration des RabbitMQ-Eventbus

5.1 Tests

Da für die Erstellung der Arbeit nur eine begrenzte Zeit zur Verfügung stand, wurden keine Unittests erstellt.
Beim Projektaufbau der einzelnen Dienste wurde Testbarkeit bereits mitbedacht (siehe z. B. die Trennung von Anwendungs- und Hardwarecode bei der *TelemetryUnit*, Abschnitt 5.0.1). Ursprünglich bestand auch die Bestrebung, den Code dadurch mit Unittests abzusichern. Im Verlauf der Arbeit wurde jedoch deutlich, dass diese Vorgehensweise nicht in der gegebenen Zeit durchführbar gewesen wäre.

5.1.1 TelemetryUnit

Um während der Entwicklung erstellten Programmcode der Telemetrieeinheit zu testen, wurden einzelne Funktionen und Komponenten mit Debug-Druckausgaben versehen. Da der Mikrocontroller selbst kein Display hat, wurden die Druckausgaben über eine Uart-Schnittstelle an einen angeschlossenen PC gesendet.

```
1     case GnssState::GnssReady:
2     {
3       // ...
4
5       if (fix)
6       {
7         const char ok[] = "[GNSS FSM] FIX OK\r\n";
8         gnss_module.huart_debug.write((uint8_t*)ok, sizeof(ok) -
              1);
9       }
10      else
11      {
12        const char wait[] = "[GNSS FSM] WAITING FOR FIX\r\n";
13        gnss_module.huart_debug.write((uint8_t*)wait,
              sizeof(wait) - 1);
14      }
15
16      break;
17    }
```

Listing 5.37: Debug-Ausgaben in GnssFSM der Telemetrieeinheit

Somit konnte der gesamte Programmablauf nachvollzogen werden und Fehler im Code entdeckt werden, die nach und nach behoben wurden.

5.1.2 IngestionService

Um den Ablauf dieses Diensts im Verlauf der Entwicklung zu testen, wurden ebenfalls Druckausgaben verwendet.
Dafür kam das .NET-eigene Interface `ILogger<T>` zum Einsatz, welches per .NET Dependency Injection den Klassen zur Verfügung gestellt wurde. Je nach Bereich wurden unterschiedliche Log-Level verwendet: `LogInformation` für den normalen Programmablauf (z. B. erfolgreiche Verbindungsaufbauten, geladene Zuordnungen), `LogWarning` für erwartbare, sich selbst erholende Fehler (fehlgeschlagene Publish-Versuche, verworfene Datenpunkte) und `LogError` für unerwartete Ausnahmen (z. B. beim JSON-Parsing).

```
1       _logger.LogHighlighted(
2         "Device {deviceId}: State={state}",
3         data.DeviceId,
4         resultState
5       );
```

Listing 5.38: Verwendung der Logger-Instanz im Fahrtzustandsautomaten (TripReactor.DispatchAsync)

Durch die Ausgaben konnte das Verhalten von *IngestionService* nachvollzogen, Fehlverhalten identifiziert und an der richtigen Stelle im Code beseitigt werden.

5.1.3 FleetControlApi

Da es sich bei *FleetControlApi* um einen HTTP-Dienst handelt, konnte zur Überprüfung *Swagger* verwendet werden. *Swagger* erzeugt eine im Browser ausführbare API-Dokumentation aus den annotierten REST-Controller-Klassen, die mit Start des Programms im Browser geöffnet werden kann. Die Weboberfläche *SwaggerUI* erlaubt eine Ausführung der Endpunkte mit beliebiger Parametereingabe und druckt die Rückgaben, sodass das Verhalten überprüft werden kann und Fehler gefunden bzw. behoben werden können. Die Konfiguration wurde zu Beginn der Entwicklung in der Projektwurzeldatei `Program.cs` vorgenommen.

5.1.4 FleetControlUI, DriveDataService, Gesamtanwendung

Die Weboberfläche wurde während der Entwicklung durch Sichtprüfungen getestet. Dabei wurden zu Beginn Dummydaten verwendet. Diese wurden nach und nach entfernt, wenn entsprechende Funktionen fertig implementiert waren.

DriveDataService wurde nicht einzeln getestet, InfluxDB-Abfragen wurden manuell im browserbasierten InfluxDB-Client getestet, die Funktion des Diensts wurde dann durch Kontrolle der ankommenden Daten im *FleetControlUI* validiert.

Das Zusammenspiel der Komponenten wurde durch fortlaufende Testnutzung überprüft, identifizierte Fehler wurden entsprechend behoben.

5.1.5 Probleme und Lösungen im Verlauf der Projektumsetzung

Im Folgenden werden einige Probleme, die im Laufe der Entwicklung entdeckt wurden, sowie deren Lösung beschrieben.

Fehlerhafte GPS-Information im Datenpunkt Beim Testen wurde festgestellt, dass während der Autofahrt sporadisch die Koordinaten eines zufällig erscheinenden, weit entfernten Ortes übermittelt wurden.
Nach Verfolgung der Fehlerkette durch die Log- und Druckausgaben wurde die Fehlerursache auf die Telemetrieeinheit eingegrenzt.
Es wurde festgestellt, dass gelegentlich einige vordere Bytes der Koordinatenwerte überschrieben bzw. auf 0 gesetzt wurden. Als Grund werden Timingprobleme beim Einlesen der GNSS-Antwort vermutet. Da die genaue Fehlerursache nur durch relativ großen (zeitaufwändigen) Testaufwand gefunden werden konnte, wurde sich für eine alternative Notlösung beim Einlesen der Datenpunkte in *IngestionService* entschieden.
Die Koordinaten werden auf Plausibilität geprüft, indem errechnet wird, welche Geschwindigkeit notwendig wäre, um von einem Punkt zum nächsten zu kommen. Übersteigt der berechnete Wert eine realistische Grenze (300 km/h), wird der Punkt als unplausibel verworfen, statt in die Datenbasis übernommen zu werden. Dies funktioniert nur, solange nicht der erste Datenpunkt fehlerhaft ist. Es handelt sich um eine Notlösung, da eine echte Fehlerbehebung bei der Telemetrieeinheit zeitlich nicht möglich war.

Zugriff auf die Karten-API der Benutzeroberfläche Die beiden Karten im *FleetControlUI* verwenden die Bibliothek *Leaflet*, die ihre Kachelinformation von der externen Karten-API *Stadia Maps* bekommt.
Für die lokale Verwendung funktioniert dies problemlos, auf der entfernten, virtuellen Maschine ist dafür eine Authentifizierung nötig. Dazu wurde eine Registrierung des DNS-Namens der Anwendung bei *Stadia Maps* vorgenommen, nach der die API-Abfrage ohne Codeänderung möglich war.

Geschwindigkeits- und Beschleunigungsberechnung mit MPU-6050 Bei der Planung sollte zuerst eine IMU-Sensoreinheit vom Typ MPU-6050 (Gyroskop/Neigung, Beschleunigung, Temperatur) zum Einsatz kommen, damit Geschwindigkeits- und Beschleunigungsdaten auch bei kurzzeitig ausfallendem GPS-Fix oder fehlender Internetverbindung erhoben, zwischengespeichert und danach ohne Datenverlust bei erneutem Verbindungserfolg nachgesendet werden können.

Die Sensoreinheit misst Werte in allen drei Dimensionen, wobei die Erdbeschleunigung in Abhängigkeit von der absoluten Ausrichtung der Sensoreinheit zur Erde anteilig in den drei Richtungskomponenten enthalten ist. Für die Anwendung wird aber nur ein Wert in Fahrtrichtung gebraucht. Die Berechnungen sowie die Bereinigung der Werte wären rechnerisch extrem aufwändig und fehleranfällig gewesen.

Das verwendete LTE/GNSS-Modul gibt eine Geschwindigkeitsangabe in Fahrtrichtung aus, die aus der Bestimmung der Koordinaten erfolgt. Die Angabe unterliegt

einer gemessenen Ungenauigkeit von unter ± 5 km/h und wurde als tolerierbar für die Anwendung erachtet. Fahrtdaten liegen somit nur vor, wenn auch ein GPS-Fix vorhanden ist. Der Geschwindigkeitsfehler wurde entsprechend bei der Fahrtstarterkennung im Automaten von *IngestionService* berücksichtigt.

USB-Zugriff zur Abfrage der Geräte-ID einer Telemetrieeinheit Ursprünglich wurde die Abfrage der Geräte-ID einer an einen USB-Port angeschlossenen Telemetrieeinheit durch eine HTTP-Anfrage gelöst, die auf die USB-Ports des Rechners zugreifen konnte und eine ID in der Antwort an die Benutzeroberfläche zurückgab.

Bei der Planung wurde zunächst nicht bedacht, dass *FleetControlApi* später auf einer virtuellen Maschine läuft, die keinen Zugriff auf die USB-Ports hat. Somit wurde die HTTP-Methode inklusive der ganzen Logik und dem Hardwarezugriff zuerst für *FleetControlApi* geplant und auch implementiert.
Als die Anwendung dann nach dem lokalen Test in der Cloud-Umgebung hochgefahren wurde, kam der Planungsfehler zum Vorschein. Da nur das FleetControlUI bzw. der Browser des Benutzers theoretisch Zugriff zur Hardware des Nutzerrechners haben kann, wurde die Funktionalität nach *FleetControlUI* verlagert.
Der Browser kann aus Sicherheitsgründen nur in einem sogenannten *secure context* auf die Hardware des Zielrechners zugreifen. Dafür musste *FleetControlUI* und der einzige von diesem direkt erreichbare Dienst *FleetControlApi* von HTTP auf HTTPS umkonfiguriert werden.

Um HTTPS bereitzustellen, ohne jeden einzelnen der beiden Dienste selbst mit einem eigenen TLS-Zertifikat ausstatten zu müssen, wurde ein Reverse Proxy vorgeschaltet: **Caddy** übernimmt die Verschlüsselung und leitet Anfragen intern unverschlüsselt an die jeweiligen Dienste weiter, die von HTTPS selbst nichts mitbekommen. Ein Vorteil gegenüber z. B. nginx ist, dass Caddy das benötigte Zertifikat selbst besorgt und erneuert über Let's Encrypt, ohne dass dafür eine manuelle Einrichtung nötig ist.

IngestionService und *DriveDataService* konnten unverändert bleiben, da sie nie direkt vom Browser aus aufgerufen werden, sondern nur intern per gRPC über `localhost` auf der virtuellen Maschine miteinander kommunizieren.

Die Konfiguration des Routing findet in einer sogenannten Caddyfile statt. Caddy kann als Dienst unter Verwendung der Datei über ein Standard-Docker-Image gestartet werden. Der Caddy-Service wurde über eine eigene Docker-Compose-Datei zum Projekt hinzugefügt.

```
1     fleetcontrol.polandcentral.cloudapp.azure.com {
2       handle /api/* {
3         reverse_proxy localhost:5000
4       }
5
6       handle /hubs/* {
7         reverse_proxy localhost:5000
8       }
9
10      handle {
11        reverse_proxy localhost:4200
12      }
13    }
```

Listing 5.39: Reverse-Proxy-Konfiguration (Caddyfile)

Undeterministisches Verhalten bei der Geräte-ID-Ausgabe der Telemetrieeinheit Nach Umstellung des USB-Zugriffs auf Zugriff durch den Browser zeigte sich beim Testen das Problem, dass nach Betätigung des Tasters zum Ermöglichen eines Einleseversuchs der Geräte-ID einer angeschlossenen Telemetrieeinheit mehrere Versuche in unterschiedlicher Anzahl notwendig waren, um die ID in der Benutzeroberfläche zu erhalten.
Der Fehler wurde dadurch verursacht, dass noch Reste der Debug-Ausgaben gelesen werden, die nach Tasterdruck gesendet wurden, bevor der RTOS-Scheduler die Task für den Automatenbetrieb stoppt. Der Fehler ließ sich nicht ohne weiteres beheben: Bei einer Neuanlage einer Telemetrieeinheit im System muss die Einleseaktion potentiell mehrfach ausgelöst werden, um die Geräte-ID korrekt einzulesen.

6 Inbetriebnahme

FleetControl wurde als verteilte Microservice-Anwendung entworfen, die einzelnen Komponenten eine unabhängige Skalierung erlauben soll.

Um einen einfachen Einsatz sowie automatisierte Bereitstellungsmethoden zu ermöglichen, wurden die Dienste unter Verwendung von **Docker** für den Betrieb als Container (siehe Abschnitt 2.3.3) erstellt.

Jeder Backend-Dienst wird als eigener Container mit einer Dockerfile definiert, in dem alle Abhängigkeiten enthalten sind. Die Container-Images können in automatisierten Cloud-Hostingangeboten verschiedener Anbieter verwendet werden, sodass eine Überführung in ein reales betriebliches Umfeld einfach wäre.

Um die Anwendungen bequemer starten zu können, wurde zusätzlich Docker-Compose verwendet. Für jeden Dienst wurde dazu zusätzlich eine Docker-Compose-Datei verfasst. Caddy, RabbitMQ, InfluxDB und PostgreSQL wurden bereits während der Entwicklung als Docker-Compose-Dateien angelegt, da sich diese direkt aus Standard-Container-Images starten lassen.

Um die Anwendung im Rahmen dieses Projekts bereitzustellen, wurde sich aus Kostengründen für die einfachste cloudbasierte Bereitstellungsmethode entschieden. Die gesamte Anwendung, inklusive beider Datenbanksysteme, des Eventbus sowie des MQTT-Brokers, wurde auf einer auf Microsoft Azure basierenden VM in Betrieb genommen.

In einem betrieblichen Einsatz macht dies natürlich wenig Sinn, da so eine unabhängige Skalierung nicht möglich ist bzw. die Fehlertoleranz ebenfalls auf einen Flaschenhals reduziert wird. Ein verteilter Einsatz und eine unabhängige Skalierung auf verschiedenen VMs oder unter Verwendung von Orchestrierungssystemen wie Kubernetes sind jedoch aufgrund der Microservice-Architektur und der Containerisierung problemlos möglich.

Um die Anwendung in der implementierten Struktur in einer neuen Umgebung in Betrieb zu nehmen, sind folgende Schritte notwendig:

1. Docker und Docker Compose installieren.
2. Ins Wurzelverzeichnis des Projekts wechseln.

```
cd ~/.../FleetControl
```

3. Infrastrukturdienste in den jeweiligen Unterverzeichnissen starten: PostgreSQL, InfluxDB, RabbitMQ und den MQTT-Broker.

```
cd Postgres && docker compose up -d && cd ..
cd InfluxDB && docker compose up -d && cd ..
cd EventBus && docker compose up -d && cd ..
cd MQTT && docker compose up -d && cd ..
```

4. Die Backend-Dienste in den jeweiligen Unterverzeichnissen starten: *FleetControlApi*, *DriveDataService*, *IngestionService* – *IngestionService* lädt beim eigenen Start den Zuordnungs-Cache per gRPC von *FleetControlApi* (siehe Abschnitt 5.0.2), muss also nach diesem gestartet werden.

```
cd FleetControlApi && docker compose up -d && cd ..
cd DriveDataService && docker compose up -d && cd ..
cd IngestionService && docker compose up -d && cd ..
```

5. *FleetControlUI* über `npm install` und das mitgelieferte Skript `run-frontend.sh` starten. Das Skript startet den Angular-Entwicklungsserver mit `-host 0.0.0.0`, damit dieser auch von außerhalb von `localhost` erreichbar ist.

```
cd FleetControlUI
npm install
./run-frontend.sh
```

6. Caddy starten. Bei einer öffentlich erreichbaren Domain wird automatisch ein TLS-Zertifikat bezogen, für eine rein lokale Umgebung kann stattdessen direkt unverschlüsselt auf die einzelnen Ports zugegriffen werden.

```
cd Caddy && docker compose up -d && cd ..
```

Die Anwendung ist damit unter
`https://fleetcontrol.polandcentral.cloudapp.azure.com` erreichbar. Für

den vollen Funktionsumfang, insbesondere die Registrierung neuer Telemetrieeinheiten über die Web Serial API, wird ein Chromium-basierter Browser (Chrome, Edge, Opera) benötigt.

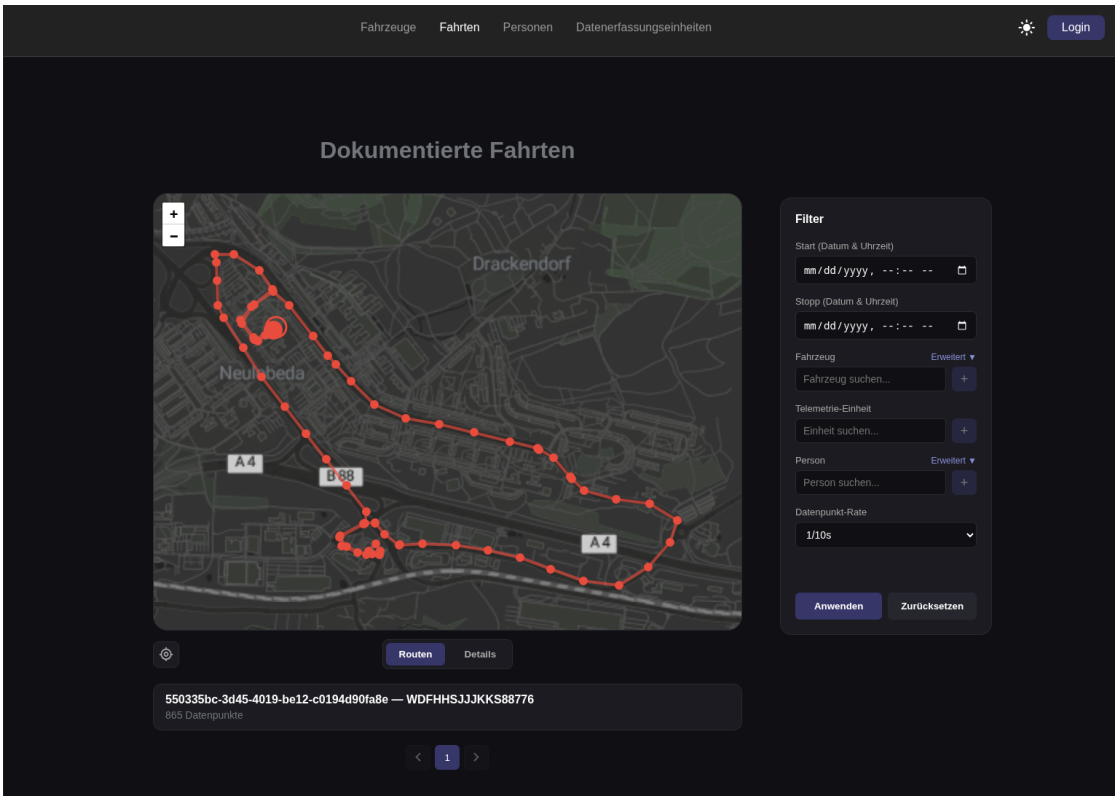


Abbildung 6.1: Fahrtenansicht von *FleetControl*

7 Evaluation

Um den Erfolg des Projekts beurteilen zu können, fand ein Abgleich der erreichten Ergebnisse mit den Projektzielen statt.

7.1 Erfüllung der Anforderungen

Die Evaluation erfolgte, indem geprüft wurde, wie sehr die erreichten Ergebnisse den erstellten Systemanforderungen entsprechen.

7.1.1 Funktionale Anforderungen

Im Folgenden wird bewertet, inwieweit die in Abschnitt 3.2 definierten funktionalen Anforderungen umgesetzt wurden.

- **FR 1 (Telemetrieeinheit, Hardware):** Erfüllt, mit Einschränkung. Die Telemetrieeinheit erfasst nach Einschaltung (Stromverbindung) Geschwindigkeit, Beschleunigung sowie die GPS-Position im Abstand von einer Sekunde (1.1). Die Einheit sendet Datenpunkte in einsekündigen Abständen über LTE (1.2). Die Telemetrieeinheit kann mit Fehlerzuständen wie aussetzendem GPS-Fix und ausfallender Internetverbindung umgehen (1.3), erhebt während eines GPS-Fix-Ausfalls allerdings keine Geschwindigkeits- und Beschleunigungsdaten (siehe Abschnitt 7.1.3).
- **FR 2 (Telemetrieeinheiten, System):** Erfüllt, mit Einschränkung. Eine Telemetrieeinheit kann mit der ausgegebenen Geräte-ID im System angelegt und wieder gelöscht werden, solange die zugehörigen Fahrten vorher gelöscht wurden (2.1), ein Fahrzeug kann der Einheit zugewiesen werden bzw. die Verbindung kann wieder getrennt werden (2.2). Angelegte Telemetrieeinheiten können nach Geräte-ID, verbundenem Fahrzeug und verbundenem Fahrer gefiltert werden. Detailansicht einer Telemetrieeinheit ist möglich, Filter funktioniert additiv und restriktiv (2.3). Die Geräte-ID-Auslesung für die Registrierung (2.1) benötigt dabei potentiell mehrere Anlaufversuche (siehe Abschnitt 5.1.5).

- **FR 3 (Fahrzeuge):** Vollständig erfüllt. Das System erlaubt die eindeutige Anlage sowie die Löschung von Fahrzeugen mit fahrzeugbezogenen Daten. Das Fahrzeug kann mit einem Fahrer und einer Telemetrieeinheit verbunden werden, die Verbindungen können wieder aufgehoben werden und ein Fahrzeug kann gelöscht werden, wenn keine zugehörigen Fahrten mehr existieren (3.1). Die Fahrzeugdaten können angesehen und nachbearbeitet werden mit Ausnahme der FIN, die als Identifikator fungiert (3.2). Auf einer Livekarte wird die letzte Position aller Fahrzeuge angezeigt mit Information aus dem zugehörigen Datenpunkt, die Positionen werden selbstständig aktualisiert und gerade aktive Fahrzeuge werden durch Pulsieren der aktuellen Markierung von statischen Fahrzeugen unterschieden (3.3). Eine Filterung der Fahrzeuge und eine Beschränkung der Karte auf diese (additiv und restriktiv) ist ebenfalls möglich (3.4).
- **FR 4 (Fahrer):** Vollständig erfüllt. Das System erlaubt die Anlage und Löschung von Personen mit personenbezogenen Daten (4.1). Fahrer können eindeutig einem Fahrzeug zugewiesen werden, die Zuweisung kann wieder aufgelöst werden (4.2). Eine Filterung (additiv und restriktiv) ist ebenfalls möglich (4.3).
- **FR 5 (Fahrten):** Vollständig umgesetzt. Erfasste Fahrten werden auf einer Karte dargestellt und die Datenpunkte zu Routen verbunden. Die Routendarstellung kann nach Anzahl der Datenpunkte angepasst werden (5.1). Jede Fahrt kann in einer Graphdarstellung begutachtet werden, die den Verlauf sowie Extremwerte leicht nachvollziehbar macht (5.2). Filtern (additiv/restriktiv) der Fahrten ist möglich (5.3). Eine Löschung der Fahrten ist ebenfalls möglich (5.4).

7.1.2 Nichtfunktionale Anforderungen

Die nichtfunktionalen Anforderungen wurden ebenfalls ausgewertet:

- **NFR 1 (Performance):** Erfüllt. Ein Datenpunkt pro Sekunde ist durch das feste Publish-Intervall der Telemetrieeinheit vorgegeben.
- **NFR 2 (Skalierbarkeit):** Erfüllt. *IngestionService* verwaltet für jedes Gerät einen unabhängigen Zustandsautomaten (siehe `TripReactor`) und kann so mehrere Telemetrieeinheiten gleichzeitig verarbeiten. Die Microservice-Architektur ermöglicht darüber hinaus horizontale Skalierung, die im Rahmen dieses Projekts aufgrund des aus Kostengründen gewählten Single-VM-Betriebs (siehe Kapitel Inbetriebnahme) in der tatsächlichen Bereitstellung nicht ausgenutzt wurde.
- **NFR 3 (Portabilität):** Größtenteils erfüllt, mit einer Einschränkung: Die

Registrierung neuer Telemetrieeinheiten über den Browser funktioniert nur in Chromium-basierten Browsern (Chrome, Edge, Opera), nicht in Firefox oder Safari – die Anwendung ist damit nicht vollständig browserunabhängig.

- **NFR 4 (Wartbarkeit):** Grundsätzlich erfüllt durch die Microservice- und Schichtenarchitekturen.

7.1.3 Bekannte Probleme

Im Folgenden wird kurz auf bekannte Probleme eingegangen.

- **GPS-Plausibilitätsfilter-Schwäche:** Der allererste Datenpunkt eines Geräts (bzw. der erste nach einer Pause von mehr als 5 Minuten) wird ungeprüft akzeptiert. Ist ausgerechnet dieser Punkt fehlerhaft, werden bis zu drei nachfolgende, tatsächlich korrekte Punkte fälschlich verworfen, bevor sich der Filter selbst korrigiert.
- **Keine Geschwindigkeits- und Beschleunigungsdaten bei GPS-Fix-Ausfall:** Da auf eine dedizierte IMU-Sensoreinheit verzichtet wurde (siehe Abschnitt 5.1.5) und Geschwindigkeitswerte stattdessen aus der GNSS-Positionsbestimmung des LTE/GNSS-Moduls abgeleitet werden, liegen während eines GPS-Fix-Ausfalls keine Geschwindigkeits- und Beschleunigungsdaten vor.
- **Mehrfache Geräte-ID-Anfragen bei angeschlossener Telemetrieeinheit nötig:** Auf der Benutzeroberfläche muss die Schaltfläche zur Geräte-ID-Ausgabe bei der Anlage von Telemetrieeinheiten potentiell mehrfach betätigt werden, um die ID in der Dropdown-Liste zu erhalten (siehe Abschnitt 5.1.5).
- **Stromversorgung der Telemetrieeinheit:** Die Telemetrieeinheit wurde im Rahmen dieser Arbeit für die Stromversorgung durch einen Zigarettenanzünder im PKW betrieben. Dabei besteht das Problem, dass eine dauerhaft an der Autoelektrik angeschlossene Telemetrieeinheit bei manchen PKWs auch bei ausgeschalteten Fahrzeugen Strom zieht. Wenn solche Fahrzeuge lange stehen, könnte es zu vorzeitig entladenen Autobatterien kommen.
- **Dauer bis zum GPS-Fix:** Bis ein gültiger GPS-Fix nach dem Einschalten einer Telemetrieeinheit gefunden wird, kann es zwischen ein und zwei Minuten dauern. Wenn die Telemetrieeinheit dauerhaft mit Spannung versorgt wird, wie es bei manchen Autos der Fall ist, ist dies nur bei einer ersten Fahrt direkt nach der Verbindung ein Problem. Fährt das Fahrzeug ohne gültigen GPS-Fix los, erzeugt dies keinen Fehler; sobald ein Fix gefunden wird, beginnt die Fahrt bei dem ersten gültigen Datenpunkt, der erfasst wird.
- **Eingeschränkte Browserunterstützung:** Die Registrierung neuer Teleme-

trieeinheiten über die *Web Serial API* funktioniert nur in Chromium-basierten Browsern (Chrome, Edge, Opera), nicht in Firefox oder Safari.

- **Kein unabhängiger Betrieb der Microservices:** Trotz Microservice-Architektur läuft die gesamte Anwendung aus Kostengründen aktuell auf einer einzelnen virtuellen Maschine – ein Ausfall betrifft damit alle Dienste gleichzeitig.
- **Abweichung von der angestrebten Hardwareunabhängigkeit:** `TelemetryPoint` in **App/** verarbeitet die Antwort des LTE/GNSS-Moduls direkt im Antwortformat des SIM7600, statt diesen direkten Hardware-Zugrif im dafür vorgesehenen Ordner **Platform/** zu kapseln.
- **FleetControlUI nicht containerisiert:** Im Gegensatz zu den übrigen Diensten liegt für *FleetControlUI* keine Dockerfile vor. Die Anwendung läuft produktiv über den Angular-Entwicklungsserver (`ng serve`), zudem ist die Backend-URL in `environment.ts` zur Build-Zeit fest einkompiliert.

8 Ausblick

Im Folgenden wird auf Ideen für die Weiterentwicklung von *FleetControl* eingegangen.

8.1 Erweiterbarkeit der Ergebnisse

Die naheliegendste Erweiterung ist die Behebung vorhandener Probleme, die in Abschnitt 7.1.3 (Bekannte Probleme) beschrieben wurde.

Darüber hinaus ergeben sich aus der bestehenden Architektur mehrere inhaltliche Erweiterungen:

- **Login und Rollenmodell:** Aktuell existiert keine Authentifizierung. Eine Erweiterung um Benutzerkonten mit Rollen (z. B. Administrator vs. einfacher Nutzer) würde sich in die bestehende Schichtenarchitektur einfügen, ohne die Struktur zu verändern.
- **Mehrere Fahrer pro Fahrzeug mit NFC-Authentifizierung:** Aktuell wird einem Fahrzeug höchstens ein Fahrer fest zugeordnet. Mit einem zusätzlichen NFC-Lesegerät an der Telemetrieeinheit könnte der tatsächlich fahrende Fahrer vor Fahrtbeginn erkannt und mit übertragen werden, sodass mehrere Fahrer dasselbe Fahrzeug nutzen können und jede Fahrt korrekt dem jeweiligen Fahrer statt nur dem Fahrzeug zugeordnet wird. Das Domänenmodell unterstützt eine fahrtbezogene Fahrer-Zuordnung bereits (`Trip.DriverId`), die feste Fahrzeug-Fahrer-Verknüpfung müsste dafür durch eine dynamische, pro Fahrt übermittelte Zuordnung ergänzt werden.
- **Sicherheits- und Verschleißscores:** Aus den bereits erfassten Telemetriedaten (Geschwindigkeit, Beschleunigung, siehe Abschnitt 5.0.1) ließen sich aggregierte Kennzahlen ableiten, etwa ein Sicherheitsscore aus der Häufigkeit starker Brems- und Beschleunigungsvorgänge sowie Geschwindigkeitsüberschreitungen, oder ein Verschleißscore aus gefahrener Strecke in Kombination mit der Fahrweise. Da die Rohdaten bereits in InfluxDB vorliegen, wäre dies primär eine zusätzliche Auswertungslogik ohne Änderungen an der Datenerfassung.

- **Automatische Fahrtenbücher:** Da Fahrten bereits vollständig mit Start- und Endzeitpunkt, Fahrer, Fahrzeug und Route erfasst werden, könnte *FleetControl* daraus automatisch klassische Fahrtenbücher pro Fahrzeug und Zeitraum generieren (z. B. als PDF- oder CSV-Export), wie sie für die steuerliche Nutzung von Firmenfahrzeugen benötigt werden.

8.2 Übertragbarkeit der Ergebnisse

Die entwickelte Architektur beschränkt sich nicht nur auf die Flottenüberwachung von Fahrzeugen, sondern lässt sich auch auf andere Anwendungsfälle mit ähnlichem Muster – verteilte Sensordatenerfassung, zentrale Datensammlung über einen Dienst, Bereitstellung in einer Benutzeroberfläche – übertragen, etwa Flottenmanagement für Maschinen, Zustandsüberwachung von Baustellen- oder Lagerequipment, oder andere IoT-Projekte.

9 Zusammenfassung und Fazit

Im Rahmen dieses Projekts wurde ein Fuhrparkverwaltungssystem als Fullstack-App inklusive einer Telemetrieeinheit für die Fahrtdatenerhebung entworfen, entwickelt, getestet und in Betrieb genommen.

Das Gesamtproblem wurde in Problembereiche aufgeteilt. Daraus wurden eigenständige Komponenten entwickelt, die im Microservice-Architekturstil entworfen wurden und sich somit für den Betrieb im Cloud-Umfeld eignen. Der Architekturstil unterstützt grundsätzlich unabhängige Skalierung einzelner Komponenten je nach Bedarf und vereinfacht eine parallele Weiterentwicklung des Systems.

Die Telemetrieeinheit kann Geschwindigkeit, Beschleunigung und Ortsposition eines Fahrzeugs bestimmen und lässt sich über eine Zigarettenanzünderbuchse im Fahrzeug betreiben. Erhobene Daten werden mit Zeitstempel über LTE an das System gesendet. Die Einheit lässt sich mit Betätigung des Tasters in einen ID-Ausgabe-Modus umstellen, sodass jede Telemetrieeinheit einzigartig im System angelegt werden kann.

Die Benutzeroberfläche kann Telemetrieeinheiten, Fahrer und Fahrzeuge registrieren, Verknüpfungen anlegen bzw. trennen und löschen.

Die letzte erfasste Position von Fahrzeugen wird auf einer Karte angezeigt, wobei sich gerade bewegende Fahrzeuge graphisch hervorgehoben werden.

Die Anwendung kann Fahrten erfassen, diese werden ebenfalls auf einer Karte dargestellt, wobei die Datenpunkte zu Fahrtrouten verbunden werden. Der Fahrverlauf einzelner Fahrten kann auf einem Graphen genauer untersucht werden.

Die Benutzeroberfläche läuft im Browser und kann in einem hellen und in einem dunklen Modus je nach Präferenz des Nutzers bedient werden.

Damit wurde das in Abschnitt 1.3 definierte Ziel – ein funktionsfähiges Minimum Viable Product mit allen notwendigen Komponenten – erreicht; die im Detail erfüllten Anforderungen sind in Kapitel 7 dokumentiert.

Das Projekt bot eine Möglichkeit, sich mit dem gesamten Spektrum von eingebetteter, hardwarenaher Softwareentwicklung bis hin zu einem verteilten, auf Microservices basierenden Full-Stack-System auseinanderzusetzen: von der Sensoransteuerung und Zustandsautomaten auf einem Echtzeitbetriebssystem über die asynchrone

Kommunikation mehrerer Dienste per MQTT, gRPC, REST und Eventbus bis hin zur Umsetzung einer reaktiven Weboberfläche und deren containerisierter Bereitstellung in einer Cloud-Umgebung. Diese fachliche Bandbreite deckt sich mit den in der Motivation genannten beruflichen Zielen und hat praktische Erfahrung in mehreren für die angestrebte berufliche Tätigkeit relevanten Bereichen ermöglicht.

Die in Kapitel 8 beschriebene Erweiterbarkeit und Übertragbarkeit der Ergebnisse zeigt, dass die entwickelte Lösung über den unmittelbaren Anwendungsfall der Fahrzeugflottenüberwachung hinaus als Grundlage für weiterführende Funktionen sowie für strukturell ähnliche Problemstellungen in anderen Domänen dienen kann.

KI-Nutzung

Zur Erstellung dieser Arbeit wurde durchgängig KI, genauer *ChatGPT*, verwendet. Im Folgenden soll beschrieben werden, wie KI zum Einsatz gekommen ist:

1. Vorschlag von Bibliotheken und Technologien sowie Generieren von Codebeispielen, die erklären, wie diese zu verwenden sind.
2. Erklärung von Fehlerausgaben bei der Entwicklung und Vorschlag von Lösungen, die dann auch zuerst umgesetzt wurden. So konnte viel Zeit bei der Entwicklung gespart werden.
3. Generierung von Terminalbefehlen für die Bedienung während der Entwicklung
4. Kontrolle und Berichtigung von Rechtschreibung, Grammatik, Form im Bericht.

Literatur

[1] 1a-anlasser.de. *Lichtmaschine – Beschreibung und Fehlersuche*. 2021. URL: https://www.1a-anlasser.de/downloads/Lichtmaschine-Diagnose-bei-Fehlfunktion.pdf (besucht am 24. 09. 2026).

[2] Frank Buschmann u. a. *Pattern-Oriented Software Architecture, Volume 1: A System of Patterns*. Chichester: John Wiley & Sons, 1996.

[3] Thomas H. Cormen u. a. *Introduction to Algorithms*. 3. Aufl. Cambridge, MA: MIT Press, 2009. ISBN: 978-0-262-03384-8.

[4] Docker Inc. *What is a Container?* 2026. URL: https://www.docker.com/resources/what-container/ (besucht am 24. 09. 2026).

[5] Thomas Huining Feng u. a. „System Design, Modeling, and Simulation using Ptolemy II“. In: Hrsg. von Claudius Ptolemaeus. Ptolemy.org, 2014. Kap. Finite State Machines. URL: https://ptolemy.berkeley.edu/books/Systems/chapters/FiniteStateMachines.pdf.

[6] Roy Thomas Fielding. „Architectural Styles and the Design of Network-based Software Architectures“. Diss. University of California, Irvine, 2000. URL: https://ics.uci.edu/~fielding/pubs/dissertation/fielding_dissertation.pdf.

[7] Gregor Grambow. „Data Engineering – Application Data Persistence, Vorlesungsfolien“. Unveröffentlichtes Vorlesungsmaterial, Hochschule Aalen. 2025.

[8] Gregor Grambow. „Datenbanksysteme – Vorlesungsskript“. Unveröffentlichtes Vorlesungsmaterial, Hochschule Aalen, Wintersemester 2024/25. 2024.

[9] InfluxData. *InfluxDB storage engine*. 2026. URL: https://docs.influxdata.com/influxdb/v2/reference/internals/storage-engine/ (besucht am 19. 09. 2026).

[10] Søren Kejser Jensen, Torben Bach Pedersen und Christian Thomsen. „Time Series Management Systems: A Survey“. In: *IEEE Transactions on Knowledge and Data Engineering* 29.11 (2017), S. 2581–2600. DOI: 10.1109/TKDE.2017.2740932.

[11] Microsoft. *Asynchronous programming - C#*. 2026. URL: https://learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/ (besucht am 24. 09. 2026).

[12] Mark Richards und Neal Ford. *Handbuch Moderner Softwarearchitektur: Architekturstile, Patterns und Best Practices*. Heidelberg: O'Reilly Verlag GmbH & Co. KG, 2026.

[13] Jürgen Schüle. „Embedded Systems 1 – Vorlesungsfolien". Unveröffentlichtes Vorlesungsmaterial, Hochschule Aalen. 2025.

[14] *STM32F103x8, STM32F103xB: Medium-density performance line ARM-based 32-bit MCU (Datasheet, DocID13587 Rev. 16)*. STMicroelectronics. 2013. URL: https://www.st.com/resource/en/datasheet/stm32f103c8.pdf (besucht am 24. 09. 2026).

[15] The gRPC Authors. *Introduction to gRPC*. 2026. URL: https://grpc.io/docs/what-is-grpc/introduction/ (besucht am 19. 09. 2026).

[16] Waveshare. *SIM7600E-H 4G HAT*. 2026. URL: https://www.waveshare.com/wiki/SIM7600E-H_4G_HAT (besucht am 24. 09. 2026).